过了喵 源程序前连续30页

```
================================================================
FILE: packages/core/src/error.rs
================================================================

use crate::parser::ParserError;
use std::error::Error;

#[derive(Debug)]
pub enum CoreError {
    Parser(ParserError),
    AI(String),
    Exam(String),
    Export(String),
    Config(String),
}

impl std::fmt::Display for CoreError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            CoreError::Parser(e) => write!(f, "Parser: {e}"),
            CoreError::AI(e) => write!(f, "AI: {e}"),
            CoreError::Exam(e) => write!(f, "Exam: {e}"),
            CoreError::Export(e) => write!(f, "Export: {e}"),
            CoreError::Config(e) => write!(f, "Config: {e}"),
        }
    }
}

impl From<ParserError> for CoreError {
    fn from(e: ParserError) -> Self { CoreError::Parser(e) }
}

impl From<reqwest::Error> for CoreError {
    fn from(e: reqwest::Error) -> Self {
        let mut msg = e.to_string();
        let mut source = e.source();
        while let Some(s) = source {
            msg.push_str(&format!("\n  caused by: {s}"));
            source = s.source();
        }
        CoreError::AI(msg)
    }
}

impl From<serde_json::Error> for CoreError {
    fn from(e: serde_json::Error) -> Self { CoreError::AI(e.to_string()) }
}


================================================================
FILE: packages/core/src/lib.rs
================================================================

pub mod parser;
pub mod ai;
pub mod exam;
pub mod export;
pub mod config;
pub mod error;


================================================================
FILE: packages/core/src/config/mod.rs
================================================================

mod store;

pub use store::{AIConfigData, ConfigStore};


================================================================
FILE: packages/core/src/config/store.rs
================================================================

use crate::error::CoreError;
use base64::{engine::general_purpose::STANDARD as BASE64, Engine};
use ring::aead::{Aad, LessSafeKey, Nonce, UnboundKey, AES_256_GCM};
use ring::rand::{SecureRandom, SystemRandom};
use serde::{Deserialize, Serialize};
use std::path::PathBuf;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AIConfigData {
    pub endpoint: String,
    pub api_key: String,
    pub model: String,
}

pub struct ConfigStore {
    config_path: PathBuf,
    key: [u8; 32],
}

fn seal(key: &[u8; 32], plaintext: &[u8]) -> Result<String, CoreError> {
    let rng = SystemRandom::new();
    let mut nonce_bytes = [0u8; 12];
    rng.fill(&mut nonce_bytes)
        .map_err(|_| CoreError::Config("nonce generation failed".to_string()))?;

    let unbound_key = UnboundKey::new(&AES_256_GCM, key)
        .map_err(|_| CoreError::Config("invalid key".to_string()))?;
    let sealing_key = LessSafeKey::new(unbound_key);

    let nonce = Nonce::assume_unique_for_key(nonce_bytes);
    let mut in_out = plaintext.to_vec();
    sealing_key
        .seal_in_place_append_tag(nonce, Aad::empty(), &mut in_out)
        .map_err(|_| CoreError::Config("encryption failed".to_string()))?;

    let mut combined = nonce_bytes.to_vec();
    combined.extend_from_slice(&in_out);
    Ok(BASE64.encode(&combined))
}
```

```
fn open_sealed(key: &[u8; 32], encoded: &str) -> Result<Vec<u8>, CoreError> {
    let combined = BASE64
        .decode(encoded)
        .map_err(|e| CoreError::Config(format!("decode error: {e}")))?;
    if combined.len() < 12 + 16 {
        return Err(CoreError::Config("config file corrupted".to_string()));
    }
    let nonce_bytes: [u8; 12] = combined[..12].try_into().unwrap();
    let unbound_key = UnboundKey::new(&AES_256_GCM, key)
        .map_err(|_| CoreError::Config("invalid key".to_string()))?;
    let opening_key = LessSafeKey::new(unbound_key);
    let nonce = Nonce::assume_unique_for_key(nonce_bytes);
    let mut in_out = combined[12..].to_vec();
    let plaintext = opening_key
        .open_in_place(nonce, Aad::empty(), &mut in_out)
        .map_err(|_| CoreError::Config("decryption failed — config may be corrupted".to_string()))?;
    Ok(plaintext.to_vec())
}

impl ConfigStore {
    pub fn new(app_name: &str) -> Result<Self, CoreError> {
        let config_dir = dirs_next().ok_or_else(|| {
            CoreError::Config("cannot determine config directory".to_string())
        })?;
        let app_dir = config_dir.join(app_name);
        std::fs::create_dir_all(&app_dir)
            .map_err(|e| CoreError::Config(format!("cannot create config dir: {e}")))?;

        let config_path = app_dir.join("config.enc");
        let key_path = app_dir.join("key.bin");

        let key = if key_path.exists() {
            let key_bytes = std::fs::read(&key_path)
                .map_err(|e| CoreError::Config(format!("cannot read key: {e}")))?;
            let mut key = [0u8; 32];
            key.copy_from_slice(&key_bytes[..32]);
            key
        } else {
            let rng = SystemRandom::new();
            let mut key = [0u8; 32];
            rng.fill(&mut key)
                .map_err(|_| CoreError::Config("key generation failed".to_string()))?;
            std::fs::write(&key_path, &key)
                .map_err(|e| CoreError::Config(format!("cannot write key: {e}")))?;

            #[cfg(unix)]
            {
                use std::os::unix::fs::PermissionsExt;
                let mut perms = std::fs::metadata(&key_path)
                    .map_err(|e| CoreError::Config(format!("cannot read key metadata: {e}")))?
                    .permissions();
                perms.set_mode(0o600);
                std::fs::set_permissions(&key_path, perms)
                    .map_err(|e| CoreError::Config(format!("cannot set key permissions: {e}")))?;
            }

            key
        };

        Ok(Self { config_path, key })
    }

    pub fn save(&self, endpoint: &str, api_key: &str, model: &str) -> Result<(), CoreError> {
        let config = AIConfigData {
            endpoint: endpoint.to_string(),
            api_key: api_key.to_string(),
            model: model.to_string(),
        };

        let plaintext = serde_json::to_vec(&config)
            .map_err(|e| CoreError::Config(format!("serialize error: {e}")))?;
        let encoded = seal(&self.key, &plaintext)?;
        std::fs::write(&self.config_path, encoded)
            .map_err(|e| CoreError::Config(format!("write error: {e}")))?;
        Ok(())
    }

    pub fn load(&self) -> Result<Option<AIConfigData>, CoreError> {
        if !self.config_path.exists() {
            return Ok(None);
        }

        let encoded = std::fs::read_to_string(&self.config_path)
            .map_err(|e| CoreError::Config(format!("read error: {e}")))?;
        let plaintext = open_sealed(&self.key, &encoded)?;
        let config: AIConfigData = serde_json::from_slice(&plaintext)
            .map_err(|e| CoreError::Config(format!("deserialize error: {e}")))?;
        Ok(Some(config))
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn seal_open_roundtrip() {
        let key = [7u8; 32];
        let sealed = seal(&key, b"hello vision").unwrap();
        let opened = open_sealed(&key, &sealed).unwrap();
        assert_eq!(opened, b"hello vision");
    }

    #[test]
    fn open_rejects_wrong_key() {
        let sealed = seal(&[7u8; 32], b"secret").unwrap();
        assert!(open_sealed(&[8u8; 32], &sealed).is_err());
    }
}

fn dirs_next() -> Option<PathBuf> {
    #[cfg(target_os = "macos")]
    {
        if let Ok(home) = std::env::var("HOME") {
            return Some(PathBuf::from(home).join("Library").join("Application Support"));
        }
    }
```

```
    #[cfg(target_os = "linux")]
    {
        if let Ok(data) = std::env::var("XDG_CONFIG_HOME") {
            if !data.is_empty() {
                return Some(PathBuf::from(data));
            }
        }
        if let Ok(home) = std::env::var("HOME") {
            return Some(PathBuf::from(home).join(".config"));
        }
    }
    #[cfg(target_os = "android")]
    {
        if let Ok(home) = std::env::var("HOME") {
            let p = PathBuf::from(home).join(".config");
            let _ = std::fs::create_dir_all(&p);
            return Some(p);
        }
        if let Ok(tmp) = std::env::var("TMPDIR") {
            return Some(PathBuf::from(tmp));
        }
    }
    #[cfg(target_os = "windows")]
    {
        if let Ok(appdata) = std::env::var("APPDATA") {
            return Some(PathBuf::from(appdata));
        }
    }
    None
}


================================================================
FILE: packages/core/src/parser/csv.rs
================================================================

use super::table::{clean_cell, rows_to_markdown};
use super::ParserError;

pub fn extract_csv(path: &str) -> Result<String, ParserError> {
    let raw = super::txt::read_text_lossy(path)?;
    let mut rdr = ::csv::ReaderBuilder::new()
        .has_headers(false)
        .flexible(true)
        .from_reader(raw.as_bytes());
    let mut rows: Vec<Vec<String>> = Vec::new();
    for rec in rdr.records() {
        let rec = rec.map_err(|e| ParserError::Parse(format!("csv error: {e}")))?;
        let cells: Vec<String> = rec.iter().map(clean_cell).collect();
        if cells.iter().all(|c| c.is_empty()) {
            continue;
        }
        rows.push(cells);
    }
    if rows.is_empty() {
        return Err(ParserError::Parse("file is empty".to_string()));
    }
    Ok(rows_to_markdown(&rows))
}


================================================================
FILE: packages/core/src/parser/docx.rs
================================================================

use super::xml_text::collect_texts;
use super::ParserError;
use std::io::Read;

pub fn extract_docx(path: &str) -> Result<String, ParserError> {
    let file = std::fs::File::open(path)?;
    let mut archive = zip::ZipArchive::new(file)
        .map_err(|e| ParserError::Parse(format!("invalid docx zip: {e}")))?;

    let mut doc_xml = String::new();
    archive
        .by_name("word/document.xml")
        .map_err(|e| ParserError::Parse(format!("missing document.xml: {e}")))?
        .read_to_string(&mut doc_xml)?;

    let texts = collect_texts(&doc_xml, b"t")?;
    let result = texts.join("\n");
    if result.trim().is_empty() {
        return Err(ParserError::Parse("no text found in docx".to_string()));
    }
    Ok(result)
}


================================================================
FILE: packages/core/src/parser/epub.rs
================================================================

use super::html::strip_html;
use super::ParserError;
use quick_xml::events::Event;
use quick_xml::Reader;
use std::collections::HashMap;
use std::io::Read;

pub fn extract_epub(path: &str) -> Result<String, ParserError> {
    let file = std::fs::File::open(path)?;
    let mut archive = zip::ZipArchive::new(file)
        .map_err(|e| ParserError::Parse(format!("invalid epub zip: {e}")))?;

    let mut chapters: Vec<String> = Vec::new();
    if let Ok(container) = read_zip_string(&mut archive, "META-INF/container.xml") {
        if let Some(opf_path) = container_opf_path(&container) {
            if let Ok(opf) = read_zip_string(&mut archive, &opf_path) {
                let base = opf_path
                    .rsplit_once('/')
                    .map(|(d, _)| format!("{d}/"))
                    .unwrap_or_default();
                chapters = spine_chapter_paths(&opf, &base);
            }
        }
    }
    if chapters.is_empty() {
```

```
        chapters = (0..archive.len())
            .filter_map(|i| archive.by_index(i).ok().map(|f| f.name().to_string()))
            .filter(|n| n.ends_with(".xhtml") || n.ends_with(".html") || n.ends_with(".htm"))
            .collect();
        chapters.sort();
    }

    let mut out = String::new();
    for ch in &chapters {
        if let Ok(xml) = read_zip_string(&mut archive, ch) {
            let t = strip_html(&xml);
            if !t.trim().is_empty() {
                out.push_str(t.trim());
                out.push_str("\n\n");
            }
        }
    }
    if out.trim().is_empty() {
        return Err(ParserError::Parse("no text found in epub".to_string()));
    }
    Ok(out.trim_end().to_string())
}

fn read_zip_string(
    archive: &mut zip::ZipArchive<std::fs::File>,
    name: &str,
) -> Result<String, ParserError> {
    let mut s = String::new();
    archive
        .by_name(name)
        .map_err(|e| ParserError::Parse(format!("zip error: {e}")))?
        .read_to_string(&mut s)?;
    Ok(s)
}

fn container_opf_path(xml: &str) -> Option<String> {
    let mut reader = Reader::from_str(xml);
    let mut buf = Vec::new();
    loop {
        match reader.read_event_into(&mut buf) {
            Ok(Event::Start(ref e)) | Ok(Event::Empty(ref e)) => {
                if e.local_name().as_ref() == b"rootfile" {
                    for attr in e.attributes().flatten() {
                        if attr.key.local_name().as_ref() == b"full-path" {
                            return Some(String::from_utf8_lossy(&attr.value).to_string());
                        }
                    }
                }
            }
            Ok(Event::Eof) | Err(_) => return None,
            _ => {}
        }
        buf.clear();
    }
}

fn spine_chapter_paths(opf: &str, base: &str) -> Vec<String> {
    let mut manifest: HashMap<String, String> = HashMap::new();
    let mut spine: Vec<String> = Vec::new();
    let mut reader = Reader::from_str(opf);
    let mut buf = Vec::new();
    loop {
        match reader.read_event_into(&mut buf) {
            Ok(Event::Start(ref e)) | Ok(Event::Empty(ref e)) => {
                match e.local_name().as_ref() {
                    b"item" => {
                        let mut id = None;
                        let mut href = None;
                        for attr in e.attributes().flatten() {
                            match attr.key.local_name().as_ref() {
                                b"id" => id = Some(String::from_utf8_lossy(&attr.value).to_string()),
                                b"href" => href = Some(String::from_utf8_lossy(&attr.value).to_string()),
                                _ => {}
                            }
                        }
                        if let (Some(id), Some(href)) = (id, href) {
                            manifest.insert(id, href);
                        }
                    }
                    b"itemref" => {
                        for attr in e.attributes().flatten() {
                            if attr.key.local_name().as_ref() == b"idref" {
                                spine.push(String::from_utf8_lossy(&attr.value).to_string());
                            }
                        }
                    }
                    _ => {}
                }
            }
            Ok(Event::Eof) | Err(_) => break,
            _ => {}
        }
        buf.clear();
    }
    spine
        .into_iter()
        .filter_map(|id| manifest.get(&id).map(|h| format!("{base}{h}")))
        .collect()
}


================================================================
FILE: packages/core/src/parser/excel.rs
================================================================

use super::table::{clean_cell, rows_to_markdown};
use super::ParserError;
use calamine::{open_workbook_auto, Data, Reader};

pub fn extract_excel(path: &str) -> Result<String, ParserError> {
    let mut wb = open_workbook_auto(path)
        .map_err(|e| ParserError::Parse(format!("spreadsheet open error: {e}")))?;
    let names: Vec<String> = wb.sheet_names().to_vec();
    let multi = names.len() > 1;
    let mut out = String::new();
    for name in names {
        let range = match wb.worksheet_range(&name) {
            Ok(r) => r,
```

```
            Err(_) => continue,
        };
        let mut rows: Vec<Vec<String>> = Vec::new();
        for row in range.rows() {
            let cells: Vec<String> = row
                .iter()
                .map(|d| clean_cell(&cell_to_string(d)))
                .collect();
            if cells.iter().all(|c| c.is_empty()) {
                continue;
            }
            rows.push(cells);
        }
        if rows.is_empty() {
            continue;
        }
        if multi {
            out.push_str(&format!("### {name}\n\n"));
        }
        out.push_str(&rows_to_markdown(&rows));
        out.push('\n');
    }
    if out.trim().is_empty() {
        return Err(ParserError::Parse("no data found in spreadsheet".to_string()));
    }
    Ok(out.trim_end().to_string())
}

fn cell_to_string(d: &Data) -> String {
    match d {
        Data::Empty => String::new(),
        other => other.to_string(),
    }
}


==========================================================================
FILE: packages/core/src/parser/html.rs
==========================================================================

use super::ParserError;

pub fn extract_html(path: &str) -> Result<String, ParserError> {
    let raw = super::txt::read_text_lossy(path)?;
    let text = strip_html(&raw);
    if text.trim().is_empty() {
        return Err(ParserError::Parse("no text found in html".to_string()));
    }
    Ok(text)
}

pub(crate) fn strip_html(html: &str) -> String {
    let s = remove_blocks(html, "<script", "</script>");
    let s = remove_blocks(&s, "<style", "</style>");
    let s = remove_blocks(&s, "<!--", "-->");
    let s = tags_to_text(&s);
    let s = decode_entities(&s);
    collapse_whitespace(&s)
}

fn find_ci(haystack: &str, needle: &str, from: usize) -> Option<usize> {
    let h = haystack.as_bytes();
    let n = needle.as_bytes();
    if n.is_empty() || h.len() < n.len() || from > h.len() - n.len() {
        return None;
    }
    (from..=h.len() - n.len()).find(|&i| h[i..i + n.len()].eq_ignore_ascii_case(n))
}

fn remove_blocks(input: &str, open: &str, close: &str) -> String {
    let mut out = String::with_capacity(input.len());
    let mut pos = 0;
    while let Some(start) = find_ci(input, open, pos) {
        out.push_str(&input[pos..start]);
        match find_ci(input, close, start + open.len()) {
            Some(end) => pos = end + close.len(),
            None => return out,
        }
    }
    out.push_str(&input[pos..]);
    out
}

fn tags_to_text(s: &str) -> String {
    const BLOCK_TAGS: [&str; 18] = [
        "p", "div", "br", "li", "ul", "ol", "tr", "table", "h1", "h2", "h3", "h4", "h5",
        "h6", "section", "article", "blockquote", "pre",
    ];
    let mut out = String::with_capacity(s.len());
    let mut rest = s;
    while let Some(idx) = rest.find('<') {
        out.push_str(&rest[..idx]);
        let after = &rest[idx..];
        match after.find('>') {
            Some(end) => {
                let inner = &after[1..end];
                let name: String = inner
                    .trim_start_matches('/')
                    .chars()
                    .take_while(|c| c.is_ascii_alphanumeric())
                    .collect::<String>()
                    .to_ascii_lowercase();
                if BLOCK_TAGS.contains(&name.as_str()) {
                    out.push('\n');
                } else if name == "td" || name == "th" {
                    out.push(' ');
                }
                rest = &after[end + 1..];
            }
            None => return out,
        }
    }
    out.push_str(rest);
    out
}

fn decode_numeric_entities(s: &str) -> String {
    let mut out = String::with_capacity(s.len());
```

```
    let mut rest = s;
    while let Some(idx) = rest.find("&#") {
        out.push_str(&rest[..idx]);
        let tail = &rest[idx + 2..];
        let semi = tail.find(';').filter(|&e| e > 0 && e <= 8);
        let decoded = semi.and_then(|end| {
            let code = &tail[..end];
            let parsed = if let Some(hex) = code.strip_prefix(['x', 'X']) {
                u32::from_str_radix(hex, 16).ok()
            } else {
                code.parse::<u32>().ok()
            };
            parsed.and_then(char::from_u32).map(|c| (c, end))
        });
        match decoded {
            Some((c, end)) => {
                out.push(c);
                rest = &tail[end + 1..];
            }
            None => {
                out.push_str("&#");
                rest = tail;
            }
        }
    }
    out.push_str(rest);
    out
}

fn decode_entities(s: &str) -> String {
    decode_numeric_entities(s)
        .replace(" ", " ")
        .replace("&lt;", "<")
        .replace("&gt;", ">")
        .replace("&quot;", "\"")
        .replace("&apos;", "'")
        .replace("&amp;", "&")
}

fn collapse_whitespace(s: &str) -> String {
    let mut out = String::with_capacity(s.len());
    let mut blank_pending = false;
    for line in s.lines() {
        let t = line.trim();
        if t.is_empty() {
            blank_pending = !out.is_empty();
        } else {
            if !out.is_empty() {
                out.push('\n');
                if blank_pending {
                    out.push('\n');
                }
            }
            out.push_str(t);
            blank_pending = false;
        }
    }
    out
}


======================================================================
FILE: packages/core/src/parser/mod.rs
======================================================================

mod txt;
mod docx;
mod pdf;
mod table;
mod csv;
mod excel;
mod epub;
mod html;
mod xml_text;
mod pptx;
mod odt;

use std::path::Path;

pub use txt::extract_txt;
pub use docx::extract_docx;
pub use pdf::extract_pdf;
pub use csv::extract_csv;
pub use excel::extract_excel;
pub use epub::extract_epub;
pub use html::extract_html;
pub use pptx::extract_pptx;
pub use odt::extract_odt;

#[derive(Debug)]
pub enum FileFormat {
    PlainText,
    Docx,
    Pdf,
    Odt,
    Pptx,
    Csv,
    Excel,
    Html,
    Epub,
}

impl FileFormat {
    pub fn from_extension(path: &str) -> Result<Self, ParserError> {
        let ext = Path::new(path)
            .extension()
            .and_then(|e| e.to_str())
            .unwrap_or("")
            .to_lowercase();
        match ext.as_str() {
            "docx" => Ok(FileFormat::Docx),
            "pdf" => Ok(FileFormat::Pdf),
            "csv" => Ok(FileFormat::Csv),
            "xlsx" | "xlsm" | "xls" | "ods" => Ok(FileFormat::Excel),
            "html" | "htm" => Ok(FileFormat::Html),
            "pptx" => Ok(FileFormat::Pptx),
            "odt" => Ok(FileFormat::Odt),
            "epub" => Ok(FileFormat::Epub),
```

```
            _ => Ok(FileFormat::PlainText),
        }
    }
}

#[derive(Debug)]
pub enum ParserError {
    Io(std::io::Error),
    Parse(String),
    Unsupported(String),
}

impl std::fmt::Display for ParserError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            ParserError::Io(e) => write!(f, "IO error: {e}"),
            ParserError::Parse(e) => write!(f, "Parse error: {e}"),
            ParserError::Unsupported(e) => write!(f, "Unsupported: {e}"),
        }
    }
}

impl From<std::io::Error> for ParserError {
    fn from(e: std::io::Error) -> Self { ParserError::Io(e) }
}

pub fn parse_file(path: &str) -> Result<String, ParserError> {
    let format = FileFormat::from_extension(path)?;
    match format {
        FileFormat::PlainText => extract_txt(path),
        FileFormat::Docx => extract_docx(path),
        FileFormat::Pdf => extract_pdf(path),
        FileFormat::Pptx => extract_pptx(path),
        FileFormat::Odt => extract_odt(path),
        FileFormat::Csv => extract_csv(path),
        FileFormat::Excel => extract_excel(path),
        FileFormat::Html => extract_html(path),
        FileFormat::Epub => extract_epub(path),
    }
}


======================================================================
FILE: packages/core/src/parser/odt.rs
======================================================================

use super::ParserError;
use quick_xml::events::Event;
use quick_xml::Reader;
use std::io::Read;

pub fn extract_odt(path: &str) -> Result<String, ParserError> {
    let file = std::fs::File::open(path)?;
    let mut archive = zip::ZipArchive::new(file)
        .map_err(|e| ParserError::Parse(format!("invalid odt zip: {e}")))?;

    let mut xml = String::new();
    archive
        .by_name("content.xml")
        .map_err(|e| ParserError::Parse(format!("missing content.xml: {e}")))?
        .read_to_string(&mut xml)?;

    let mut reader = Reader::from_str(&xml);
    let mut out = String::new();
    let mut buf = Vec::new();
    loop {
        match reader.read_event_into(&mut buf) {
            Ok(Event::Text(ref t)) => {
                let s = t.unescape().unwrap_or_default();
                if !s.trim().is_empty() {
                    out.push_str(&s);
                }
            }
            Ok(Event::End(ref e)) => {
                if matches!(e.local_name().as_ref(), b"p" | b"h") && !out.ends_with('\n') {
                    out.push('\n');
                }
            }
            Ok(Event::Eof) => break,
            Err(e) => return Err(ParserError::Parse(format!("xml error: {e}"))),
            _ => {}
        }
        buf.clear();
    }

    if out.trim().is_empty() {
        return Err(ParserError::Parse("no text found in odt".to_string()));
    }
    Ok(out.trim().to_string())
}


======================================================================
FILE: packages/core/src/parser/pdf.rs
======================================================================

use super::ParserError;
use lopdf::Document;
use unicode_normalization::UnicodeNormalization;

// macOS Quartz 生成的 PDF 常把汉字映射到 Kangxi 部首/CJK 兼容字符码点
// (如 人大网), 视觉相同但码点不同, 按 NFKC 归一化回标准汉字
fn normalize_compat_chars(s: &str) -> String {
    s.chars()
        .flat_map(|c| {
            let cp = c as u32;
            if (0x2E80..=0x2EFF).contains(&cp)
                || (0x2F00..=0x2FDF).contains(&cp)
                || (0xF900..=0xFAFF).contains(&cp)
            {
                c.nfkc().collect::<Vec<char>>()
            } else {
                vec![c]
            }
        })
        .collect()
}
```

```
pub fn extract_pdf(path: &str) -> Result<String, ParserError> {
    match extract_via_lopdf(path) {
        Ok(text) if !text.trim().is_empty() => return Ok(normalize_compat_chars(&text)),
        Ok(_) | Err(_) => {}
    }
    extract_via_pdf_extract(path).map(|t| normalize_compat_chars(&t))
}

fn extract_via_lopdf(path: &str) -> Result<String, ParserError> {
    let doc = Document::load(path)
        .map_err(|e| ParserError::Parse(format!("pdf load error: {e}")))?;

    let mut texts = Vec::new();
    let total = doc.get_pages().len();

    for page_num in 1..=total as u32 {
        match doc.extract_text(&[page_num]) {
            Ok(text) => {
                let cleaned = text.trim().to_string();
                if !cleaned.is_empty() {
                    texts.push(cleaned);
                }
            }
            Err(e) => {
                eprintln!("[Exameow] lopdf page {}/{} failed: {e}", page_num, total);
            }
        }
    }

    let result = texts.join("\n");
    if result.trim().is_empty() {
        return Err(ParserError::Parse("lopdf: no extractable text".to_string()));
    }
    Ok(result)
}

fn extract_via_pdf_extract(path: &str) -> Result<String, ParserError> {
    let data = std::fs::read(path)
        .map_err(|e| ParserError::Io(e))?;
    pdf_extract::extract_text_from_mem(&data)
        .map_err(|e| ParserError::Parse(format!("pdf-extract error: {e}")))
}


================================================================
FILE: packages/core/src/parser/pptx.rs
================================================================

use super::xml_text::collect_texts;
use super::ParserError;
use std::io::Read;

pub fn extract_pptx(path: &str) -> Result<String, ParserError> {
    let file = std::fs::File::open(path)?;
    let mut archive = zip::ZipArchive::new(file)
        .map_err(|e| ParserError::Parse(format!("invalid pptx zip: {e}")))?;

    let mut slide_names: Vec<String> = (0..archive.len())
        .filter_map(|i| archive.by_index(i).ok().map(|f| f.name().to_string()))
        .filter(|n| n.starts_with("ppt/slides/slide") && n.ends_with(".xml"))
        .collect();
    slide_names.sort_by_key(|n| slide_number(n));

    let mut out = String::new();
    for name in &slide_names {
        let mut xml = String::new();
        archive
            .by_name(name)
            .map_err(|e| ParserError::Parse(format!("zip error: {e}")))?
            .read_to_string(&mut xml)?;
        let texts = collect_texts(&xml, b"t")?;
        if texts.is_empty() {
            continue;
        }
        out.push_str(&format!("### Slide {}\n\n{}\n\n", slide_number(name), texts.join("\n")));
    }
    if out.trim().is_empty() {
        return Err(ParserError::Parse("no text found in pptx".to_string()));
    }
    Ok(out.trim_end().to_string())
}

fn slide_number(name: &str) -> u32 {
    name.trim_start_matches("ppt/slides/slide")
        .trim_end_matches(".xml")
        .parse()
        .unwrap_or(0)
}


================================================================
FILE: packages/core/src/parser/table.rs
================================================================

pub(crate) fn clean_cell(s: &str) -> String {
    s.replace('|', "\\|")
        .replace(['\r', '\n'], " ")
        .split_whitespace()
        .collect::<Vec<_>>()
        .join(" ")
}

pub(crate) fn rows_to_markdown(rows: &[Vec<String>]) -> String {
    let cols = rows.iter().map(|r| r.len()).max().unwrap_or(0);
    let mut out = String::new();
    for (i, row) in rows.iter().enumerate() {
        let mut cells = row.clone();
        cells.resize(cols, String::new());
        out.push_str("| ");
        out.push_str(&cells.join(" | "));
        out.push_str(" |\n");
        if i == 0 {
            out.push('|');
            out.push_str(&" --- |".repeat(cols));
            out.push('\n');
        }
    }
    out
```

```
}


================================================================
FILE: packages/core/src/parser/txt.rs
================================================================

use super::ParserError;

pub fn extract_txt(path: &str) -> Result<String, ParserError> {
    read_text_lossy(path)
}

pub(crate) fn read_text_lossy(path: &str) -> Result<String, ParserError> {
    let bytes = std::fs::read(path)?;
    read_text_from_bytes(&bytes)
}

pub(crate) fn read_text_from_bytes(bytes: &[u8]) -> Result<String, ParserError> {
    let text = if bytes.starts_with(&[0xFF, 0xFE]) {
        decode_or_reject(encoding_rs::UTF_16LE, bytes)?
    } else if bytes.starts_with(&[0xFE, 0xFF]) {
        decode_or_reject(encoding_rs::UTF_16BE, bytes)?
    } else if bytes.iter().take(8192).any(|&b| b == 0) {
        return Err(ParserError::Unsupported(
            "binary or unknown-encoding file".to_string(),
        ));
    } else {
        match std::str::from_utf8(bytes) {
            Ok(s) => s.to_string(),
            Err(_) => decode_or_reject(encoding_rs::GB18030, bytes)?,
        }
    };
    let text = text.trim_start_matches('\u{feff}').to_string();
    if text.trim().is_empty() {
        return Err(ParserError::Parse("file is empty".to_string()));
    }
    Ok(text)
}

fn decode_or_reject(
    encoding: &'static encoding_rs::Encoding,
    bytes: &[u8],
) -> Result<String, ParserError> {
    let (cow, _, had_errors) = encoding.decode(bytes);
    let replaced = cow.chars().filter(|&c| c == '\u{FFFD}').count();
    let total = cow.chars().count().max(1);
    if had_errors && replaced * 20 > total {
        return Err(ParserError::Unsupported(
            "binary or unknown-encoding file".to_string(),
        ));
    }
    Ok(cow.into_owned())
}


================================================================
FILE: packages/core/src/parser/xml_text.rs
================================================================

use super::ParserError;
use quick_xml::events::Event;
use quick_xml::Reader;

pub(crate) fn collect_texts(xml: &str, target: &[u8]) -> Result<Vec<String>, ParserError> {
    let mut reader = Reader::from_str(xml);
    reader.config_mut().trim_text(true);
    let mut texts = Vec::new();
    let mut buf = Vec::new();
    loop {
        match reader.read_event_into(&mut buf) {
            Ok(Event::Start(ref e)) => {
                if e.local_name().as_ref() == target {
                    if let Ok(Event::Text(ref t)) = reader.read_event_into(&mut buf) {
                        let text = t.unescape().unwrap_or_default();
                        if !text.trim().is_empty() {
                            texts.push(text.to_string());
                        }
                    }
                }
            }
            Ok(Event::Eof) => break,
            Err(e) => return Err(ParserError::Parse(format!("xml error: {e}"))),
            _ => {}
        }
        buf.clear();
    }
    Ok(texts)
}


================================================================
FILE: packages/core/src/ai/client.rs
================================================================

use crate::error::CoreError;
use super::models::{ModelInfo, ModelsResponse};
use reqwest::header::{AUTHORIZATION, CONTENT_TYPE};

pub struct AIClient {
    client: reqwest::Client,
    endpoint: String,
    api_key: String,
}

impl AIClient {
    pub fn new(endpoint: &str, api_key: &str) -> Self {
        let endpoint = endpoint.trim_end_matches('/').to_string();
        let client = reqwest::Client::builder()
            .no_proxy()
            .build()
            .unwrap_or_else(|_| reqwest::Client::new());
        Self {
            client,
            endpoint,
            api_key: api_key.to_string(),
        }
    }
```

```
    pub async fn fetch_models(&self) -> Result<Vec<ModelInfo>, CoreError> {
        let url = format!("{}/models", self.endpoint);
        let response = self
            .client
            .get(&url)
            .header(AUTHORIZATION, format!("Bearer {}", self.api_key))
            .timeout(std::time::Duration::from_secs(15))
            .send()
            .await?;

        if !response.status().is_success() {
            let status = response.status();
            let body = response.text().await.unwrap_or_default();
            return Err(CoreError::AI(format!("HTTP {status}: {body}")));
        }

        let models_response: ModelsResponse = response.json().await?;
        Ok(models_response.data)
    }

    pub async fn chat(
        &self,
        system_prompt: &str,
        user_prompt: &str,
        model: &str,
    ) -> Result<String, CoreError> {
        let body = serde_json::json!({
            "model": model,
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt}
            ],
            "temperature": 0.7,
            "max_tokens": 16384,
        });
        self.post_chat(body).await
    }

    pub async fn chat_with_image(
        &self,
        system_prompt: &str,
        user_text: &str,
        image_data_url: &str,
        model: &str,
    ) -> Result<String, CoreError> {
        let body = serde_json::json!({
            "model": model,
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": [
                    {"type": "text", "text": user_text},
                    {"type": "image_url", "image_url": {"url": image_data_url}}
                ]}
            ],
            "temperature": 0.2,
            "max_tokens": 16384,
        });
        self.post_chat(body).await
    }

    async fn post_chat(&self, body: serde_json::Value) -> Result<String, CoreError> {
        let url = format!("{}/chat/completions", self.endpoint);
        let response = self
            .client
            .post(&url)
            .header(AUTHORIZATION, format!("Bearer {}", self.api_key))
            .header(CONTENT_TYPE, "application/json")
            .json(&body)
            .timeout(std::time::Duration::from_secs(120))
            .send()
            .await?;

        if !response.status().is_success() {
            let status = response.status();
            let body = response.text().await.unwrap_or_default();
            return Err(CoreError::AI(format!("HTTP {status}: {body}")));
        }

        let json: serde_json::Value = response.json().await?;
        let content = json["choices"][0]["message"]["content"]
            .as_str()
            .unwrap_or("")
            .to_string();

        if content.is_empty() {
            return Err(CoreError::AI("empty response from AI".to_string()));
        }
        Ok(content)
    }
}


======================================================================
FILE: packages/core/src/ai/mod.rs
======================================================================

mod client;
mod models;

pub use client::AIClient;
pub use models::ModelInfo;


======================================================================
FILE: packages/core/src/ai/models.rs
======================================================================

use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Deserialize, Serialize)]
pub struct ModelInfo {
    pub id: String,
    #[serde(default)]
    pub object: Option<String>,
    #[serde(default)]
    pub created: Option<u64>,
    #[serde(default)]
```

```
    pub owned_by: Option<String>,
}

#[derive(Debug, Clone, Deserialize)]
pub struct ModelsResponse {
    pub data: Vec<ModelInfo>,
}


================================================================
FILE: packages/core/src/exam/answer.rs
================================================================

use crate::ai::AIClient;
use crate::error::CoreError;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AnswerResult {
    pub answer: String,
    pub analysis: String,
}

pub fn build_answer_system_prompt() -> String {
    r#"You are an expert exam-solving assistant. The user will give you an exam question (it may include options). Solve it.

## Output Rules
1. Respond ONLY with a valid JSON object — no explanation outside JSON, no markdown fences.
2. The JSON object MUST have exactly these fields:
   - "answer": the concise answer. For choice questions give the option letter(s) plus the option content; for true/false questions answer "True" or "False" (
   - "analysis": a clear step-by-step explanation of why this answer is correct.
3. Use the requested language for both fields.
4. If the question is ambiguous or unanswerable, still fill "answer" with your best attempt and explain the uncertainty in "analysis"."#
        .to_string()
}

pub fn build_answer_user_prompt(question: &str, language: &str) -> String {
    format!("Language: {language}\n\nQUESTION:\n{question}")
}

pub fn parse_answer(raw: &str) -> Result<AnswerResult, CoreError> {
    let cleaned = raw
        .trim()
        .trim_start_matches("```json")
        .trim_start_matches("```")
        .trim_end_matches("```")
        .trim();

    let start = cleaned.find('{');
    let end = cleaned.rfind('}');
    let json = match (start, end) {
        (Some(s), Some(e)) if e >= s => &cleaned[s..=e],
        _ => cleaned,
    };

    serde_json::from_str(json)
        .map_err(|e| CoreError::Exam(format!("Answer JSON parse error: {e}")))
}

pub async fn answer_question(
    client: &AIClient,
    question: &str,
    language: &str,
    model: &str,
) -> Result<AnswerResult, CoreError> {
    let system_prompt = build_answer_system_prompt();
    let user_prompt = build_answer_user_prompt(question, language);
    let response = client.chat(&system_prompt, &user_prompt, model).await?;
    parse_answer(&response)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_plain_json() {
        let r = parse_answer(r#"{"answer": "B", "analysis": "because"}"#).unwrap();
        assert_eq!(r.answer, "B");
        assert_eq!(r.analysis, "because");
    }

    #[test]
    fn parses_fenced_json_with_preamble() {
        let raw = "Here is the result:\n```json\n{\"answer\": \"True\", \"analysis\": \"x\"}\n```";
        let r = parse_answer(raw).unwrap();
        assert_eq!(r.answer, "True");
    }

    #[test]
    fn rejects_invalid_json() {
        assert!(parse_answer("not json at all").is_err());
    }
}


================================================================
FILE: packages/core/src/exam/judge.rs
================================================================

use crate::ai::AIClient;
use crate::error::CoreError;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct JudgeResult {
    pub correct: bool,
    pub feedback: String,
}

pub fn build_judge_system_prompt() -> String {
    r#"You are a strict but fair exam grader. You will be given an exam question, the reference answer, an optional reference analysis, and a student's answer

## Grading Rules
1. Judge by meaning, not wording: if the student's answer is semantically equivalent to the reference answer, it is correct even if phrased differently.
2. A partial answer that misses key points required by the reference answer is incorrect.
3. Extra correct information does not make the answer wrong, unless it contradicts the reference answer.
```

```
## Output Rules
1. Respond ONLY with a valid JSON object — no explanation outside JSON, no markdown fences.
2. The JSON object MUST have exactly these fields:
   - "correct": true or false
   - "feedback": one or two sentences explaining why the answer is correct, or what is missing or wrong.
3. Write "feedback" in the requested language."#
        .to_string()
}

pub fn build_judge_user_prompt(
    stem: &str,
    reference_answer: &str,
    analysis: &str,
    user_answer: &str,
    language: &str,
) -> String {
    let analysis_block = if analysis.trim().is_empty() {
        String::new()
    } else {
        format!("\n\nREFERENCE ANALYSIS:\n{analysis}")
    };
    format!(
        "Language: {language}\n\nQUESTION:\n{stem}\n\nREFERENCE ANSWER:\n{reference_answer}{analysis_block}\n\nSTUDENT ANSWER:\n{user_answer}"
    )
}

pub fn parse_judge(raw: &str) -> Result<JudgeResult, CoreError> {
    let cleaned = raw
        .trim()
        .trim_start_matches("```json")
        .trim_start_matches("```")
        .trim_end_matches("```")
        .trim();

    let start = cleaned.find('{');
    let end = cleaned.rfind('}');
    let json = match (start, end) {
        (Some(s), Some(e)) if e >= s => &cleaned[s..=e],
        _ => cleaned,
    };

    serde_json::from_str(json)
        .map_err(|e| CoreError::Exam(format!("Judge JSON parse error: {e}")))
}

pub async fn judge_answer(
    client: &AIClient,
    stem: &str,
    reference_answer: &str,
    analysis: &str,
    user_answer: &str,
    language: &str,
    model: &str,
) -> Result<JudgeResult, CoreError> {
    let system_prompt = build_judge_system_prompt();
    let user_prompt = build_judge_user_prompt(stem, reference_answer, analysis, user_answer, language);
    let response = client.chat(&system_prompt, &user_prompt, model).await?;
    parse_judge(&response)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_plain_json() {
        let r = parse_judge(r#"{"correct": true, "feedback": "well done"}"#).unwrap();
        assert!(r.correct);
        assert_eq!(r.feedback, "well done");
    }

    #[test]
    fn parses_fenced_json_with_preamble() {
        let raw = "Result:\n```json\n{\"correct\": false, \"feedback\": \"missing key point\"}\n```";
        let r = parse_judge(raw).unwrap();
        assert!(!r.correct);
    }

    #[test]
    fn rejects_invalid_json() {
        assert!(parse_judge("not json").is_err());
    }

    #[test]
    fn user_prompt_omits_empty_analysis() {
        let p = build_judge_user_prompt("stem", "ref", "", "mine", "Chinese");
        assert!(!p.contains("REFERENCE ANALYSIS"));
        let p2 = build_judge_user_prompt("stem", "ref", "because", "mine", "Chinese");
        assert!(p2.contains("REFERENCE ANALYSIS:\nbecause"));
    }
}


======================================================================
FILE: packages/core/src/exam/mod.rs
======================================================================

mod answer;
mod judge;
mod types;
mod prompt;

pub use answer::*;
pub use judge::*;
pub use types::*;
pub use prompt::*;


======================================================================
FILE: packages/core/src/exam/prompt.rs
======================================================================

use crate::exam::{Difficulty, ExamParams, QuestionType};
use crate::error::CoreError;
use crate::ai::AIClient;
use crate::exam::Question;

pub fn build_system_prompt() -> String {
```

```
    format!(
        r#"You are an expert exam question generator. Generate questions based on the provided document content.

## Critical Rules (MUST follow)
- EVERY question MUST be unique — do NOT generate two questions that test the same concept, fact, or sentence.
- Cover DIFFERENT parts of the document for each question. Avoid clustering questions on the same paragraph.
- Vary question wording, angles, and tested knowledge points.
- When the question stem or analysis refers to the document, ALWAYS use the specific document name provided — NEVER use vague phrases like "the document", "th

## Output Rules
1. Respond ONLY with a valid JSON array — no explanation, no markdown fences.
2. Each question object MUST have exactly these fields:
   - "id": a short unique identifier string
   - "type": one of [{}]
   - "stem": the question text
   - "options": array of option strings (required for single_choice/multi_choice/true_false; empty array for others)
   - "answer": the correct answer
   - "analysis": brief explanation of the answer (can be empty string for fill_blank/short_answer)
3. For single_choice: exactly 4 options, one correct.
4. For multi_choice: exactly 4 options, at least one correct (list correct letters separated by comma in answer).
5. For true_false: options ["True", "False"], answer is "True" or "False".
6. For fill_blank: answer is the exact word/phrase to fill in.
7. For short_answer: answer is a concise reference answer.
8. All questions must be based on the document content.
9. Use the specified language for questions.
"#,
        vec![
            QuestionType::SingleChoice,
            QuestionType::MultiChoice,
            QuestionType::TrueFalse,
            QuestionType::FillBlank,
            QuestionType::ShortAnswer,
        ]
        .iter()
        .map(|t| t.to_string())
        .collect::<Vec<_>>()
        .join(", ")
    )
}

pub fn build_user_prompt(text: &str, params: &ExamParams) -> String {
    let difficulty_str = match params.difficulty {
        Difficulty::Easy => "easy questions suitable for beginners",
        Difficulty::Medium => "moderate difficulty questions requiring understanding",
        Difficulty::Hard => "challenging questions requiring deep analysis",
    };

    let topic_note = match &params.topic_filter {
        Some(topic) => format!("\nFocus on this topic: {topic}"),
        None => String::new(),
    };

    let batch_note = match params.batch_index {
        Some(idx) if params.batch_total.unwrap_or(1) > 1 => {
            format!(
                "\nThis is batch {}/{} of the document. Focus on different content than other batches would.",
                idx, params.batch_total.unwrap_or(1)
            )
        }
        _ => String::new(),
    };

    let doc_name = match &params.source_name {
        Some(name) if name.contains('、') => format!("\nThe documents are collectively titled: {name}\nWhen questions need to reference a specific document, u
        Some(name) => format!("\nThe document title is: {name}\nWhen questions need to reference this document, use \"{name}\" — do NOT say \"the document\" o
        None => String::new(),
    };

    let max_chars = 32000;
    let text_section = if text.len() > max_chars {
        let head_size = max_chars * 6 / 10;
        let tail_size = max_chars - head_size;
        let head_len = safe_char_boundary(text, head_size);
        let head = &text[..head_len];
        let tail_start = safe_char_boundary(text, text.len().saturating_sub(tail_size));
        let tail = if tail_start > head_len + 100 {
            format!("\n\n...(middle omitted)...\n\n{}", &text[tail_start..])
        } else {
            text[head_len..].to_string()
        };
        format!("{}{}", head, tail)
    } else {
        text.to_string()
    };

    let count_instruction = if let Some(ref tc) = params.type_counts {
        let mut parts: Vec<String> = vec![];
        let mut total: u32 = 0;
        for (key, &cnt) in tc {
            if cnt > 0 {
                parts.push(format!("{cnt} {key} questions"));
                total += cnt;
            }
        }
        format!(
            "Generate exactly the following breakdown of {total} questions:\n{per_type}",
            total = total,
            per_type = parts.join("\n")
        )
    } else {
        let types_list = params
            .question_types
            .iter()
            .map(|t| t.to_string())
            .collect::<Vec<_>>()
            .join(", ");
        format!(
            "Generate {count} questions.\nQuestion types: {types}",
            count = params.count,
            types = types_list,
        )
    };

    format!(
        r#"{count_instruction}
Difficulty: {difficulty_str}
Language: {language}{topic_note}{batch_note}{doc_name}
```

```
DOCUMENT CONTENT:
{text_content}
"#,
        count_instruction = count_instruction,
        difficulty_str = difficulty_str,
        language = params.language,
        topic_note = topic_note,
        batch_note = batch_note,
        doc_name = doc_name,
        text_content = text_section,
    )
}

pub fn parse_questions(json_str: &str) -> Result<Vec<Question>, CoreError> {
    let cleaned = json_str
        .trim()
        .trim_start_matches("```json")
        .trim_start_matches("```")
        .trim_end_matches("```")
        .trim();

    let questions: Vec<Question> = serde_json::from_str(cleaned)
        .map_err(|e| CoreError::Exam(format!("JSON parse error: {e}")))?;

    if questions.is_empty() {
        return Err(CoreError::Exam("AI returned empty questions array".to_string()));
    }

    Ok(questions)
}

fn safe_char_boundary(s: &str, mut index: usize) -> usize {
    if index >= s.len() {
        return s.len();
    }
    while index > 0 && !s.is_char_boundary(index) {
        index -= 1;
    }
    index
}

pub async fn generate_exam(
    client: &AIClient,
    text: &str,
    params: &ExamParams,
    model: &str,
) -> Result<Vec<Question>, CoreError> {
    let doc_text = params.text.as_deref().unwrap_or(text);
    let system_prompt = build_system_prompt();
    let user_prompt = build_user_prompt(doc_text, params);
    let response = client.chat(&system_prompt, &user_prompt, model).await?;
    parse_questions(&response)
}


======================================================================
FILE: packages/core/src/exam/types.rs
======================================================================

use serde::{Deserialize, Serialize};
use std::fmt;

#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum QuestionType {
    SingleChoice,
    MultiChoice,
    TrueFalse,
    FillBlank,
    ShortAnswer,
}

impl fmt::Display for QuestionType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            QuestionType::SingleChoice => write!(f, "single_choice"),
            QuestionType::MultiChoice => write!(f, "multi_choice"),
            QuestionType::TrueFalse => write!(f, "true_false"),
            QuestionType::FillBlank => write!(f, "fill_blank"),
            QuestionType::ShortAnswer => write!(f, "short_answer"),
        }
    }
}

impl QuestionType {
    pub fn to_label_cn(&self) -> &'static str {
        match self {
            QuestionType::SingleChoice => "单选题",
            QuestionType::MultiChoice => "多选题",
            QuestionType::TrueFalse => "判断题",
            QuestionType::FillBlank => "填空题",
            QuestionType::ShortAnswer => "简答题",
        }
    }
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub enum Difficulty {
    Easy,
    Medium,
    Hard,
}

impl fmt::Display for Difficulty {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Difficulty::Easy => write!(f, "easy"),
            Difficulty::Medium => write!(f, "medium"),
            Difficulty::Hard => write!(f, "hard"),
        }
    }
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Question {
    pub id: String,
    #[serde(rename = "type")]
```

```
    pub qtype: QuestionType,
    pub stem: String,
    #[serde(default)]
    pub options: Vec<String>,
    pub answer: String,
    #[serde(default)]
    pub analysis: String,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ExamParams {
    pub question_types: Vec<QuestionType>,
    pub count: u32,
    pub difficulty: Difficulty,
    pub language: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub topic_filter: Option<String>,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub type_counts: Option<std::collections::HashMap<String, u32>>,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub text: Option<String>,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub batch_index: Option<u32>,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub batch_total: Option<u32>,
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub source_name: Option<String>,
}


================================================================
FILE: packages/core/src/export/mod.rs
================================================================

mod writer;
mod xlsx;

pub use writer::{export_csv, export_csv_to_writer};
pub use xlsx::{export_xlsx, export_xlsx_to_writer};


================================================================
FILE: packages/core/src/export/writer.rs
================================================================

use crate::error::CoreError;
use crate::exam::Question;
use csv::WriterBuilder;
use std::io::Write;

use super::xlsx::to_chinese_type;

pub fn export_csv(questions: &[Question], path: &str) -> Result<(), CoreError> {
    let mut wtr = WriterBuilder::new()
        .from_path(path)
        .map_err(|e| CoreError::Export(format!("cannot create CSV file: {e}")))?;

    write_csv_records(questions, &mut wtr)?;

    Ok(())
}

pub fn export_csv_to_writer<W: Write>(questions: &[Question], writer: W) -> Result<(), CoreError> {
    let mut wtr = WriterBuilder::new()
        .from_writer(writer);

    write_csv_records(questions, &mut wtr)?;

    Ok(())
}

fn write_csv_records<W: Write>(questions: &[Question], wtr: &mut csv::Writer<W>) -> Result<(), CoreError> {
    wtr.write_record(["题干", "题型", "选项A", "选项B", "选项C", "选项D", "选项E", "选项F", "选项G", "选项H", "正确答案", "解析", "章节", "难度"])
        .map_err(|e| CoreError::Export(format!("write error: {e}")))?;

    for q in questions {
        let qtype_str = q.qtype.to_string();
        let qtype = to_chinese_type(&qtype_str);
        let mut row: Vec<String> = vec![
            q.stem.clone(),
            qtype.to_string(),
        ];
        for i in 0..8 {
            row.push(q.options.get(i).cloned().unwrap_or_default());
        }
        row.push(q.answer.clone());
        row.push(q.analysis.clone());
        row.push(String::new());
        row.push(String::new());

        wtr.write_record(&row)
            .map_err(|e| CoreError::Export(format!("write error: {e}")))?;
    }

    wtr.flush()
        .map_err(|e| CoreError::Export(format!("flush error: {e}")))?;

    Ok(())
}


================================================================
FILE: packages/core/src/export/xlsx.rs
================================================================

use crate::error::CoreError;
use crate::exam::Question;
use std::io::{Cursor, Write};
use zip::write::FileOptions;
use zip::CompressionMethod;

const XLSX_TYPE_MAP: &[(&str, &str)] = &[
    ("single_choice", "单选题"),
    ("multi_choice", "多选题"),
    ("true_false", "判断题"),
    ("fill_blank", "填空题"),
    ("short_answer", "简答题"),
];
```

```
pub(super) fn to_chinese_type(qtype: &str) -> &str {
    for (key, label) in XLSX_TYPE_MAP {
        if qtype == *key {
            return label;
        }
    }
    "简答题"
}

struct SharedStringWriter {
    strings: Vec<String>,
    index_map: std::collections::HashMap<String, usize>,
}

impl SharedStringWriter {
    fn new() -> Self {
        SharedStringWriter {
            strings: Vec::new(),
            index_map: std::collections::HashMap::new(),
        }
    }

    fn add(&mut self, s: &str) -> usize {
        if let Some(idx) = self.index_map.get(s) {
            return *idx;
        }
        let idx = self.strings.len();
        self.strings.push(s.to_string());
        self.index_map.insert(s.to_string(), idx);
        idx
    }

    fn to_xml(&self) -> String {
        let count = self.strings.len();
        let mut xml = format!(
            r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<sst xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main" count="{}" uniqueCount="{}">"#,
            count, count
        );
        for s in &self.strings {
            let escaped = escape_xml(s);
            xml.push_str(&format!("<si><t>{}</t></si>", escaped));
        }
        xml.push_str("</sst>");
        xml
    }
}

fn escape_xml(s: &str) -> String {
    s.replace('&', "&amp;")
        .replace('<', "&lt;")
        .replace('>', "&gt;")
        .replace('"', "&quot;")
        .replace('\'', "&apos;")
}

fn col_letter(idx: usize) -> String {
    let mut n = idx;
    let mut result = Vec::new();
    loop {
        let rem = n % 26;
        result.push((b'A' + rem as u8) as char);
        if n < 26 {
            break;
        }
        n = n / 26 - 1;
    }
    result.reverse();
    result.into_iter().collect()
}

fn answer_letter(answer: &str, options: &[String]) -> String {
    if answer.len() == 1 && answer.chars().all(|c| c.is_ascii_uppercase() && c >= 'A' && c <= 'H') {
        return answer.to_string();
    }

    let mut letters = String::new();
    for ch in answer.chars() {
        if ch.is_ascii_uppercase() && ch >= 'A' && ch <= 'H' {
            letters.push(ch);
        }
    }
    if !letters.is_empty() {
        return letters;
    }

    let mut result = String::new();
    for a_part in answer.split(',') {
        let a_part = a_part.trim();
        if let Some(pos) = options.iter().position(|o| o.trim() == a_part) {
            result.push((b'A' + pos as u8) as char);
        }
    }

    if result.is_empty() {
        answer.to_string()
    } else {
        result
    }
}

fn true_false_answer(answer: &str) -> &str {
    match answer.trim() {
        "对" | "正确" | "√" | "是" | "true" | "True" | "TRUE" => "√",
        "错" | "错误" | "×" | "否" | "false" | "False" | "FALSE" => "×",
        _ => answer,
    }
}

impl Default for SharedStringWriter {
    fn default() -> Self {
        Self::new()
    }
}

#[derive(Default)]
struct SheetDataWriter {
```

```
    rows: Vec<String>,
    current_row: usize,
    shared_strings: SharedStringWriter,
}

impl SheetDataWriter {
    fn new_row(&mut self) {
        self.current_row = self.rows.len();
        let row_num = self.current_row + 1;
        self.rows.push(format!(r#"<row r="{}" spans="1:14">"#, row_num));
    }

    fn finish_row(&mut self) {
        if let Some(last) = self.rows.last_mut() {
            last.push_str("</row>");
        }
    }

    fn add_string_cell(&mut self, col: usize, value: &str) {
        let si = self.shared_strings.add(value);
        let row_num = self.current_row + 1;
        let col_letter = col_letter(col);
        let cell = format!(
            r#"<c r="{}{}" t="s"><v>{}</v></c>"#,
            col_letter, row_num, si
        );
        if let Some(last) = self.rows.last_mut() {
            last.push_str(&cell);
        }
    }

    fn to_xml(&self) -> String {
        let mut xml = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<worksheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"
           xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships"><sheetData>"#
            .to_string();

        for row in &self.rows {
            xml.push_str(row);
        }

        xml.push_str("</sheetData></worksheet>");
        xml
    }
}

pub fn export_xlsx(questions: &[Question], path: &str) -> Result<(), CoreError> {
    let data = generate_xlsx(questions)?;
    std::fs::write(path, data)
        .map_err(|e| CoreError::Export(format!("cannot write file: {e}")))
}

pub fn export_xlsx_to_writer(questions: &[Question]) -> Result<Vec<u8>, CoreError> {
    generate_xlsx(questions)
}

fn generate_xlsx(questions: &[Question]) -> Result<Vec<u8>, CoreError> {
    let mut writer = SheetDataWriter::default();

    // Row 1: Header
    writer.new_row();
    writer.add_string_cell(0, "题干（必填）");
    writer.add_string_cell(1, "题型 （必填）");
    for i in 0..8 {
        writer.add_string_cell(2 + i, &format!("选项 {}", (b'A' + i as u8) as char));
    }
    writer.add_string_cell(10, "正确答案\n（必填）");
    writer.add_string_cell(11, "解析\n（勿删）");
    writer.add_string_cell(12, "章节\n（勿删）");
    writer.add_string_cell(13, "难度");
    writer.finish_row();

    for q in questions {
        writer.new_row();

        let stem = q.stem.trim();
        let qtype_str = q.qtype.to_string();
        let qtype = to_chinese_type(&qtype_str);
        let analysis = q.analysis.trim();
        let answer = q.answer.trim();
        let options: Vec<String> = q.options.iter().map(|o| o.trim().to_string()).collect();

        let k_answer = match &q.qtype {
            crate::exam::QuestionType::SingleChoice | crate::exam::QuestionType::MultiChoice => {
                answer_letter(answer, &options)
            }
            crate::exam::QuestionType::TrueFalse => true_false_answer(answer).to_string(),
            crate::exam::QuestionType::FillBlank | crate::exam::QuestionType::ShortAnswer => {
                answer.to_string()
            }
        };

        writer.add_string_cell(0, stem);
        writer.add_string_cell(1, qtype);
        for (i, opt) in options.iter().enumerate() {
            if i < 8 {
                writer.add_string_cell(2 + i, opt);
            }
        }

        if let crate::exam::QuestionType::FillBlank = &q.qtype {
            if answer.contains('|') {
                let parts: Vec<&str> = answer.split('|').map(|s| s.trim()).collect();
                for (i, part) in parts.iter().enumerate() {
                    if i < 8 && !part.is_empty() {
                        writer.add_string_cell(2 + i, part);
                    }
                }
            }
        }

        writer.add_string_cell(10, &k_answer);
        writer.add_string_cell(11, if analysis.is_empty() { "" } else { analysis });
        writer.add_string_cell(12, "");
        writer.add_string_cell(13, "");

        writer.finish_row();
    }
```

```
    let sheet_xml = writer.to_xml();
    let sst_xml = writer.shared_strings.to_xml();

    let styles_xml = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<styleSheet xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main">
  <fonts count="1">
    <font><sz val="11"/><name val="Heiti SC Light"/></font>
  </fonts>
  <fills count="1">
    <fill><patternFill patternType="none"/></fill>
  </fills>
  <borders count="1">
    <border><left/><right/><top/><bottom/><diagonal/></border>
  </borders>
  <cellStyleXfs count="1">
    <xf numFmtId="0" fontId="0" fillId="0" borderId="0"/>
  </cellStyleXfs>
  <cellXfs count="1">
    <xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/>
  </cellXfs>
</styleSheet>"#;

    let content_types = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
  <Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>
  <Default Extension="xml" ContentType="application/xml"/>
  <Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>
  <Override PartName="/xl/worksheets/sheet1.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>
  <Override PartName="/xl/sharedStrings.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sharedStrings+xml"/>
  <Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>
</Types>"#;

    let root_rels = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="xl/workbook.xml"/>
</Relationships>"#;

    let workbook_xml = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<workbook xmlns="http://schemas.openxmlformats.org/spreadsheetml/2006/main"
          xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships">
  <sheets>
    <sheet name="Sheet1" sheetId="1" r:id="rId1"/>
  </sheets>
</workbook>"#;

    let workbook_rels = r#"<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/worksheet" Target="worksheets/sheet1.xml"/>
  <Relationship Id="rId2" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/sharedStrings" Target="sharedStrings.xml"/>
  <Relationship Id="rId3" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/styles" Target="styles.xml"/>
</Relationships>"#;

    let buf = Cursor::new(Vec::new());
    let mut zip_writer = zip::ZipWriter::new(buf);
    let opts = FileOptions::<()>::default().compression_method(CompressionMethod::Deflated);

    zip_writer
        .start_file("[Content_Types].xml", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(content_types.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("_rels/.rels", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(root_rels.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("xl/workbook.xml", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(workbook_xml.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("xl/_rels/workbook.xml.rels", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(workbook_rels.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("xl/worksheets/sheet1.xml", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(sheet_xml.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("xl/sharedStrings.xml", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(sst_xml.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    zip_writer
        .start_file("xl/styles.xml", opts)
        .map_err(|e| CoreError::Export(format!("zip error: {e}")))?;
    zip_writer
        .write_all(styles_xml.as_bytes())
        .map_err(|e| CoreError::Export(format!("zip write error: {e}")))?;

    let result = zip_writer
        .finish()
        .map_err(|e| CoreError::Export(format!("zip finalize error: {e}")))?;
    Ok(result.into_inner())
}


================================================================
FILE: packages/server/src/main.rs
================================================================

mod routes;
```

```
use axum::{routing::{get, post}, Router};
use std::sync::Arc;
use routes::AppState;
use exameow_core::config::ConfigStore;
use tower_http::cors::{CorsLayer, Any};
use tower_http::services::ServeDir;

#[tokio::main]
async fn main() {
    let config_store = ConfigStore::new("ExameowServer").unwrap_or_else(|_| {
        eprintln!("Warning: could not init config store, using transient store");
        ConfigStore::new("ExameowServerTransient").unwrap()
    });

    let state = Arc::new(AppState { config_store });

    let static_dir =
        std::env::var("STATIC_DIR").unwrap_or_else(|_| "../frontend/dist".to_string());

    let app = Router::new()
        .route("/api/models", get(routes::get_models))
        .route("/api/generate", post(routes::generate_exam_handler))
        .route("/api/answer", post(routes::answer_handler))
        .route("/api/judge", post(routes::judge_handler))
        .route("/api/export", get(routes::export_handler))
        .route("/api/export/xlsx", post(routes::export_xlsx_handler))
        .route("/api/config/save", post(routes::save_config_handler))
        .route("/api/config/load", get(routes::load_config_handler))
        .fallback_service(ServeDir::new(&static_dir))
        .layer(CorsLayer::new().allow_origin(Any).allow_methods(Any).allow_headers(Any))
        .with_state(state);

    let port: u16 = std::env::var("PORT")
        .ok()
        .and_then(|p| p.parse().ok())
        .unwrap_or(3000);

    let listener = tokio::net::TcpListener::bind(format!("0.0.0.0:{port}"))
        .await
        .unwrap();
    println!("Exameow server running on http://localhost:{port}");
    axum::serve(listener, app).await.unwrap();
}


====================================================================
FILE: packages/server/src/routes.rs
====================================================================

use axum::{
    extract::{Multipart, Query, State},
    http::{header, StatusCode},
    response::Response,
    Json,
};
use exameow_core::ai::{AIClient, ModelInfo};
use exameow_core::config::{AIConfigData, ConfigStore};
use exameow_core::exam::{
    answer_question, generate_exam, judge_answer, AnswerResult, ExamParams, JudgeResult, Question,
};
use exameow_core::parser::parse_file;
use serde::{Deserialize, Serialize};
use std::io::Write;
use std::sync::Arc;

pub struct AppState {
    pub config_store: ConfigStore,
}

#[derive(Deserialize)]
pub struct ModelsQuery {
    pub endpoint: Option<String>,
    pub api_key: Option<String>,
}

#[derive(Serialize)]
pub struct GenerateResult {
    pub questions: Vec<Question>,
}

#[derive(Deserialize)]
pub struct ExportQuery {
    pub questions: String,
}

fn ai_endpoint() -> String { std::env::var("AI_ENDPOINT").unwrap_or_default() }
fn ai_api_key() -> String { std::env::var("AI_API_KEY").unwrap_or_default() }
fn ai_model() -> String { std::env::var("AI_MODEL").unwrap_or_default() }

pub async fn get_models(
    Query(params): Query<ModelsQuery>,
) -> Result<Json<Vec<ModelInfo>>, (StatusCode, String)> {
    let endpoint = params.endpoint.as_deref().unwrap_or("");
    let api_key = params.api_key.as_deref().unwrap_or("");

    let (endpoint, api_key) = if endpoint.is_empty() || api_key.is_empty() {
        let e = ai_endpoint();
        let k = ai_api_key();
        if e.is_empty() || k.is_empty() {
            return Err((StatusCode::BAD_REQUEST, "No AI config (set AI_ENDPOINT/AI_API_KEY env vars)".to_string()));
        }
        (e, k)
    } else {
        (endpoint.to_string(), api_key.to_string())
    };

    let client = AIClient::new(&endpoint, &api_key);
    let models = client
        .fetch_models()
        .await
        .map_err(|e| (StatusCode::BAD_GATEWAY, format!("AI error: {e}")))?;
    Ok(Json(models))
}

pub async fn generate_exam_handler(
    State(_state): State<Arc<AppState>>,
    mut multipart: Multipart,
```

```
) -> Result<Json<GenerateResult>, (StatusCode, String)> {
    let mut file_data: Option<Vec<u8>> = None;
    let mut file_name = String::new();
    let mut params_json = String::new();
    let mut endpoint = String::new();
    let mut api_key = String::new();
    let mut model = String::new();

    while let Ok(Some(field)) = multipart.next_field().await {
        let name = field.name().unwrap_or("").to_string();
        match name.as_str() {
            "file" => {
                file_name = field.file_name().unwrap_or("unknown").to_string();
                file_data = Some(
                    field
                        .bytes()
                        .await
                        .map_err(|e| (StatusCode::BAD_REQUEST, e.to_string()))?
                        .to_vec(),
                );
            }
            "params" => {
                params_json = field
                    .text()
                    .await
                    .map_err(|e| (StatusCode::BAD_REQUEST, e.to_string()))?
            }
            "endpoint" => {
                endpoint = field
                    .text()
                    .await
                    .map_err(|e| (StatusCode::BAD_REQUEST, e.to_string()))?
            }
            "api_key" => {
                api_key = field
                    .text()
                    .await
                    .map_err(|e| (StatusCode::BAD_REQUEST, e.to_string()))?
            }
            "model" => {
                model = field
                    .text()
                    .await
                    .map_err(|e| (StatusCode::BAD_REQUEST, e.to_string()))?
            }
            _ => {}
        }
    }

    let file_data =
        file_data.ok_or((StatusCode::BAD_REQUEST, "No file uploaded".to_string()))?;

    let endpoint = if endpoint.is_empty() { ai_endpoint() } else { endpoint };
    let api_key = if api_key.is_empty() { ai_api_key() } else { api_key };
    let model = if model.is_empty() { ai_model() } else { model };

    if endpoint.is_empty() || api_key.is_empty() {
        return Err((StatusCode::BAD_REQUEST, "No AI config (set AI_ENDPOINT/AI_API_KEY env vars)".to_string()));
    }

    let params: ExamParams = serde_json::from_str(&params_json)
        .map_err(|e| (StatusCode::BAD_REQUEST, format!("Invalid params: {e}")))?;

    let ext = file_name.rsplit_once('.').map(|(_, e)| e).unwrap_or("txt");
    let mut temp_file = tempfile::Builder::new()
        .suffix(&format!(".{ext}"))
        .tempfile()
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;
    temp_file
        .write_all(&file_data)
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;

    let (_, temp_path) = temp_file
        .keep()
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;
    let temp_path_str = temp_path.to_string_lossy().to_string();

    let text = parse_file(&temp_path_str)
        .map_err(|e| (StatusCode::BAD_REQUEST, format!("Parse error: {e}")))?;

    let _ = std::fs::remove_file(&temp_path_str);

    let client = AIClient::new(&endpoint, &api_key);
    let questions = generate_exam(&client, &text, &params, &model)
        .await
        .map_err(|e| (StatusCode::BAD_GATEWAY, format!("AI error: {e}")))?;

    Ok(Json(GenerateResult { questions }))
}

pub async fn export_handler(
    Query(params): Query<ExportQuery>,
) -> Result<Response, (StatusCode, String)> {
    let questions: Vec<Question> = serde_json::from_str(&params.questions)
        .map_err(|e| (StatusCode::BAD_REQUEST, format!("Invalid questions JSON: {e}")))?;

    let mut buf = vec![];
    exameow_core::export::export_csv_to_writer(&questions, &mut buf)
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;

    Response::builder()
        .header(header::CONTENT_TYPE, "text/csv; charset=utf-8")
        .header(
            header::CONTENT_DISPOSITION,
            "attachment; filename=\"questions.csv\"",
        )
        .body(axum::body::Body::from(buf))
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))
}

pub async fn export_xlsx_handler(
    Json(questions): Json<Vec<Question>>,
) -> Result<Response, (StatusCode, String)> {
    let data = exameow_core::export::export_xlsx_to_writer(&questions)
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;

    Response::builder()
        .header(
```

```
            header::CONTENT_TYPE,
            "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        )
        .header(
            header::CONTENT_DISPOSITION,
            "attachment; filename=\"exameow_questions.xlsx\"",
        )
        .body(axum::body::Body::from(data))
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))
}

pub async fn save_config_handler(
    State(_state): State<Arc<AppState>>,
    Json(config): Json<AIConfigData>,
) -> Result<StatusCode, (StatusCode, String)> {
    _state
        .config_store
        .save(&config.endpoint, &config.api_key, &config.model)
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, format!("Save error: {e}")))?;
    Ok(StatusCode::OK)
}

pub async fn load_config_handler(
    State(_state): State<Arc<AppState>>,
) -> Result<Json<Option<AIConfigData>>, (StatusCode, String)> {
    let config = _state
        .config_store
        .load()
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, format!("Load error: {e}")))?;
    Ok(Json(config))
}

#[derive(Deserialize)]
pub struct AnswerRequest {
    pub question: String,
    pub language: Option<String>,
    pub endpoint: Option<String>,
    pub api_key: Option<String>,
    pub model: Option<String>,
}

pub async fn answer_handler(
    Json(req): Json<AnswerRequest>,
) -> Result<Json<AnswerResult>, (StatusCode, String)> {
    if req.question.trim().is_empty() {
        return Err((StatusCode::BAD_REQUEST, "Question is empty".to_string()));
    }

    let endpoint = req
        .endpoint
        .filter(|s| !s.is_empty())
        .unwrap_or_else(ai_endpoint);
    let api_key = req
        .api_key
        .filter(|s| !s.is_empty())
        .unwrap_or_else(ai_api_key);
    let model = req.model.filter(|s| !s.is_empty()).unwrap_or_else(ai_model);

    if endpoint.is_empty() || api_key.is_empty() {
        return Err((
            StatusCode::BAD_REQUEST,
            "No AI config (set AI_ENDPOINT/AI_API_KEY env vars)".to_string(),
        ));
    }

    let language = req.language.filter(|s| !s.is_empty()).unwrap_or_else(|| "Chinese".to_string());

    let client = AIClient::new(&endpoint, &api_key);
    let result = answer_question(&client, &req.question, &language, &model)
        .await
        .map_err(|e| (StatusCode::BAD_GATEWAY, format!("AI error: {e}")))?;
    Ok(Json(result))
}

#[derive(Deserialize)]
pub struct JudgeRequest {
    pub stem: String,
    pub reference_answer: String,
    pub analysis: Option<String>,
    pub user_answer: String,
    pub language: Option<String>,
    pub endpoint: Option<String>,
    pub api_key: Option<String>,
    pub model: Option<String>,
}

pub async fn judge_handler(
    Json(req): Json<JudgeRequest>,
) -> Result<Json<JudgeResult>, (StatusCode, String)> {
    if req.user_answer.trim().is_empty() {
        return Err((StatusCode::BAD_REQUEST, "User answer is empty".to_string()));
    }

    let endpoint = req
        .endpoint
        .filter(|s| !s.is_empty())
        .unwrap_or_else(ai_endpoint);
    let api_key = req
        .api_key
        .filter(|s| !s.is_empty())
        .unwrap_or_else(ai_api_key);
    let model = req.model.filter(|s| !s.is_empty()).unwrap_or_else(ai_model);

    if endpoint.is_empty() || api_key.is_empty() {
        return Err((
            StatusCode::BAD_REQUEST,
            "No AI config (set AI_ENDPOINT/AI_API_KEY env vars)".to_string(),
        ));
    }

    let language = req.language.filter(|s| !s.is_empty()).unwrap_or_else(|| "Chinese".to_string());
    let analysis = req.analysis.unwrap_or_default();

    let client = AIClient::new(&endpoint, &api_key);
    let result = judge_answer(
        &client,
        &req.stem,
        &req.reference_answer,
```

```
        &analysis,
        &req.user_answer,
        &language,
        &model,
    )
    .await
    .map_err(|e| (StatusCode::BAD_GATEWAY, format!("AI error: {e}")))?;
    Ok(Json(result))
}


================================================================
FILE: src-tauri/src/lib.rs
================================================================

use exameow_core::ai::{AIClient, ModelInfo};
use exameow_core::config::{AIConfigData, ConfigStore};
use exameow_core::exam::{
    answer_question as core_answer_question,
    generate_exam as core_generate_exam,
    judge_answer as core_judge_answer, AnswerResult, ExamParams, JudgeResult, Question,
};
use exameow_core::export::export_csv as core_export_csv;
use exameow_core::export::export_xlsx as core_export_xlsx;
use exameow_core::parser::parse_file;
use serde::Serialize;
use std::fmt;
#[cfg(target_os = "macos")]
fn make_webview_transparent(win: &tauri::WebviewWindow) {
    use objc::runtime::{Class, Object, NO};
    use objc::{msg_send, sel, sel_impl};
    if let Ok(ptr) = win.ns_window() {
        unsafe {
            let ns_window = ptr as *mut Object;
            let clear: *mut Object = msg_send![Class::get("NSColor").unwrap(), clearColor];
            let _: () = msg_send![ns_window, setOpaque: NO];
            let _: () = msg_send![ns_window, setBackgroundColor: clear];
        }
    }
}

use tauri::Manager;
use base64::Engine;

const APP_NAME: &str = "Exameow";

#[derive(Debug, Serialize)]
pub struct CommandError(String);

impl fmt::Display for CommandError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        self.0.fmt(f)
    }
}

impl std::error::Error for CommandError {}

#[derive(Serialize)]
pub struct GenerateResult {
    pub questions: Vec<Question>,
}

#[tauri::command]
async fn get_models(endpoint: String, api_key: String) -> Result<Vec<ModelInfo>, CommandError> {
    let client = AIClient::new(&endpoint, &api_key);
    client
        .fetch_models()
        .await
        .map_err(|e| CommandError(format!("Failed to fetch models: {e}")))
}

#[tauri::command]
async fn generate_exam(
    file_path: String,
    params_json: String,
    endpoint: String,
    api_key: String,
    model: String,
) -> Result<GenerateResult, CommandError> {
    let params: ExamParams = serde_json::from_str(&params_json)
        .map_err(|e| CommandError(format!("Invalid params JSON: {e}")))?;

    let text = if params.text.is_some() {
        params.text.clone().unwrap()
    } else {
        parse_file(&file_path).map_err(|e| CommandError(format!("File parse error: {e}")))?
    };

    let client = AIClient::new(&endpoint, &api_key);
    let questions = core_generate_exam(&client, &text, &params, &model)
        .await
        .map_err(|e| CommandError(format!("Exam generation error: {e}")))?;

    Ok(GenerateResult { questions })
}

#[tauri::command]
async fn answer_question(
    question: String,
    language: String,
    endpoint: String,
    api_key: String,
    model: String,
) -> Result<AnswerResult, CommandError> {
    if question.trim().is_empty() {
        return Err(CommandError("Question is empty".to_string()));
    }
    let client = AIClient::new(&endpoint, &api_key);
    core_answer_question(&client, &question, &language, &model)
        .await
        .map_err(|e| CommandError(format!("Answer error: {e}")))
}

#[tauri::command]
async fn judge_answer(
    stem: String,
    reference_answer: String,
    analysis: String,
```

```
    user_answer: String,
    language: String,
    endpoint: String,
    api_key: String,
    model: String,
) -> Result<JudgeResult, CommandError> {
    if user_answer.trim().is_empty() {
        return Err(CommandError("User answer is empty".to_string()));
    }
    let client = AIClient::new(&endpoint, &api_key);
    core_judge_answer(
        &client,
        &stem,
        &reference_answer,
        &analysis,
        &user_answer,
        &language,
        &model,
    )
    .await
    .map_err(|e| CommandError(format!("Judge error: {e}")))
}

#[tauri::command]
fn parse_file_text(file_path: String) -> Result<String, CommandError> {
    parse_file(&file_path).map_err(|e| CommandError(format!("File parse error: {e}")))
}

#[tauri::command]
fn parse_file_bytes(base64_data: String, file_ext: String) -> Result<String, CommandError> {
    let bytes = base64::engine::general_purpose::STANDARD
        .decode(&base64_data)
        .map_err(|e| CommandError(format!("Base64 decode error: {e}")))?;

    let dir = std::env::temp_dir();
    let file_name = format!("exameow_upload.{}", file_ext);
    let file_path = dir.join(&file_name);

    std::fs::write(&file_path, &bytes)
        .map_err(|e| CommandError(format!("Write temp file error: {e}")))?;

    let result = parse_file(file_path.to_string_lossy().as_ref());
    let _ = std::fs::remove_file(&file_path);
    result.map_err(|e| CommandError(format!("File parse error: {e}")))
}

#[tauri::command]
fn export_csv(questions_json: String, save_path: String) -> Result<(), CommandError> {
    let questions: Vec<Question> = serde_json::from_str(&questions_json)
        .map_err(|e| CommandError(format!("Invalid questions JSON: {e}")))?;
    core_export_csv(&questions, &save_path)
        .map_err(|e| CommandError(format!("CSV export error: {e}")))
}

#[tauri::command]
fn export_xlsx(questions_json: String, save_path: String) -> Result<(), CommandError> {
    let questions: Vec<Question> = serde_json::from_str(&questions_json)
        .map_err(|e| CommandError(format!("Invalid questions JSON: {e}")))?;
    core_export_xlsx(&questions, &save_path)
        .map_err(|e| CommandError(format!("XLSX export error: {e}")))
}

#[tauri::command]
fn export_xlsx_data(questions_json: String) -> Result<String, CommandError> {
    let questions: Vec<Question> = serde_json::from_str(&questions_json)
        .map_err(|e| CommandError(format!("Invalid questions JSON: {e}")))?;
    let data = exameow_core::export::export_xlsx_to_writer(&questions)
        .map_err(|e| CommandError(format!("Export XLSX error: {e}")))?;
    Ok(base64::engine::general_purpose::STANDARD.encode(&data))
}

#[tauri::command]
fn save_to_downloads(filename: String, content_base64: String) -> Result<String, CommandError> {
    let bytes = base64::engine::general_purpose::STANDARD
        .decode(&content_base64)
        .map_err(|e| CommandError(format!("Base64 decode error: {e}")))?;
    let dir = std::path::Path::new("/storage/emulated/0/Download/Exameow");
    std::fs::create_dir_all(dir)
        .map_err(|e| CommandError(format!("Create dir error: {e}")))?;
    let path = dir.join(&filename);
    std::fs::write(&path, &bytes)
        .map_err(|e| CommandError(format!("Write error: {e}")))?;
    Ok(path.to_string_lossy().to_string())
}

#[tauri::command]
fn save_config(endpoint: String, api_key: String, model: String) -> Result<(), CommandError> {
    let store = ConfigStore::new(APP_NAME)
        .map_err(|e| CommandError(format!("Config init error: {e}")))?;
    store
        .save(&endpoint, &api_key, &model)
        .map_err(|e| CommandError(format!("Config save error: {e}")))
}

#[tauri::command]
fn load_config() -> Result<Option<AIConfigData>, CommandError> {
    let store = ConfigStore::new(APP_NAME)
        .map_err(|e| CommandError(format!("Config init error: {e}")))?;
    store
        .load()
        .map_err(|e| CommandError(format!("Config load error: {e}")))
}

#[tauri::command]
fn open_app_settings() -> Result<(), CommandError> {
    #[cfg(target_os = "android")]
    {
        // On Android handled via @tauri-apps/plugin-opener on the frontend side
    }
    #[cfg(target_os = "ios")]
    {
        use objc::runtime::{Class, Object};
        use objc::{msg_send, sel, sel_impl};
        unsafe {
            let ns_url: *mut Object = {
                let cls = Class::get("NSURL").unwrap();
                let ns_str: *mut Object = msg_send![Class::get("NSString").unwrap(), stringWithUTF8String: "app-settings:\0".as_ptr() as *const i8];
                msg_send![cls, URLWithString: ns_str]
```

```
            };
            let app: *mut Object = msg_send![Class::get("UIApplication").unwrap(), sharedApplication];
            let _: () = msg_send![app, openURL: ns_url options: 0_usize completionHandler: 0_usize];
        }
    }
    Ok(())
}


#[cfg(target_os = "macos")]
mod screen_capture_access {
    #[link(name = "CoreGraphics", kind = "framework")]
    extern "C" {
        fn CGPreflightScreenCaptureAccess() -> bool;
        fn CGRequestScreenCaptureAccess() -> bool;
    }

    pub fn preflight() -> bool {
        unsafe { CGPreflightScreenCaptureAccess() }
    }

    pub fn request() -> bool {
        unsafe { CGRequestScreenCaptureAccess() }
    }
}

#[tauri::command]
fn check_screen_permission() -> bool {
    #[cfg(target_os = "macos")]
    {
        screen_capture_access::preflight()
    }
    #[cfg(not(target_os = "macos"))]
    {
        true
    }
}

#[tauri::command]
fn request_screen_permission() -> bool {
    #[cfg(target_os = "macos")]
    {
        screen_capture_access::request()
    }
    #[cfg(not(target_os = "macos"))]
    {
        true
    }
}

#[tauri::command]
fn open_screen_recording_settings() -> Result<(), CommandError> {
    #[cfg(target_os = "macos")]
    {
        std::process::Command::new("open")
            .arg("x-apple.systempreferences:com.apple.preference.security?Privacy_ScreenCapture")
            .spawn()
            .map_err(|e| CommandError(format!("Failed to open settings: {e}")))?;
    }
    Ok(())
}

#[cfg(desktop)]
#[tauri::command]
fn capture_screen(x: i32, y: i32, w: i32, h: i32, force: Option<bool>) -> Result<tauri::ipc::Response, CommandError> {
    std::panic::catch_unwind(|| capture_screen_inner(x, y, w, h, force.unwrap_or(false)))
        .map_err(|_| CommandError("Screen capture panicked".to_string()))?
}

#[cfg(not(desktop))]
#[tauri::command]
fn capture_screen(_x: i32, _y: i32, _w: i32, _h: i32, _force: Option<bool>) -> Result<tauri::ipc::Response, CommandError> {
    Err(CommandError(
        "Screen capture is only supported on desktop".to_string(),
    ))
}

#[cfg(desktop)]
fn capture_screen_inner(x: i32, y: i32, w: i32, h: i32, force: bool) -> Result<tauri::ipc::Response, CommandError> {
    #[cfg(target_os = "macos")]
    if !screen_capture_access::preflight() {
        return Err(CommandError("screen_recording_permission_denied".to_string()));
    }

    let t0 = std::time::Instant::now();
    let monitors = xcap::Monitor::all()
        .map_err(|e| CommandError(format!("Failed to enumerate monitors: {e}")))?;

    // Pick the monitor whose bounds contain the capture region center,
    // falling back to the primary monitor.
    let cx = x + w / 2;
    let cy = y + h / 2;
    let monitor = monitors
        .iter()
        .find(|m| {
            let mx = m.x().unwrap_or(0);
            let my = m.y().unwrap_or(0);
            let mw = m.width().unwrap_or(0) as i32;
            let mh = m.height().unwrap_or(0) as i32;
            cx >= mx && cx < mx + mw && cy >= my && cy < my + mh
        })
        .unwrap_or(&monitors[0]);

    let captured = monitor
        .capture_image()
        .map_err(|e| CommandError(format!("Failed to capture screen: {e}")))?;
    let mut dyn_img = image::DynamicImage::ImageRgba8(captured);
    let img_w = dyn_img.width();
    let img_h = dyn_img.height();

    // Convert global coordinates to monitor-local coordinates
    let mon_x = monitor.x().unwrap_or(0);
    let mon_y = monitor.y().unwrap_or(0);
    let local_x = x - mon_x;
    let local_y = y - mon_y;
    let cx = (local_x.max(0) as u32).min(img_w.saturating_sub(1));
    let cy = (local_y.max(0) as u32).min(img_h.saturating_sub(1));
    let cw = (w.max(1) as u32).min(img_w - cx);
```

```
    let ch = (h.max(1) as u32).min(img_h - cy);
    let cropped = dyn_img.crop(cx, cy, cw, ch);
    let changed = frame_changed(&cropped);
    if !force && !changed {
        eprintln!("[capture_screen] unchanged, skipped, elapsed={:?}", t0.elapsed());
        return Ok(tauri::ipc::Response::new(Vec::new()));
    }
    eprintln!("[capture_screen] crop=({cx},{cy},{cw}x{ch}) of {img_w}x{img_h} on monitor({mon_x},{mon_y}), elapsed={:?}", t0.elapsed());
    let mut buf = std::io::Cursor::new(Vec::new());
    cropped
        .write_to(&mut buf, image::ImageFormat::Jpeg)
        .map_err(|e| CommandError(format!("Failed to encode JPEG: {e}")))?;
    Ok(tauri::ipc::Response::new(buf.into_inner()))
}

// 降采样整图对比相邻两帧：静态画面直接跳过 JPEG 编码和 IPC 传输。
// 缩略图全像素比较，阈值 0.5%：光标闪烁不计入，文字变化必然超过。
// 前端另有 ~5s 心跳强制 OCR 兜底，即使漏检也不会永久卡住。
#[cfg(desktop)]
static LAST_FRAME: std::sync::Mutex<Option<Vec<u8>>> = std::sync::Mutex::new(None);

#[cfg(desktop)]
fn frame_changed(img: &image::DynamicImage) -> bool {
    const THUMB_W: u32 = 128;
    const THUMB_H: u32 = 64;
    let thumb = image::imageops::resize(img, THUMB_W, THUMB_H, image::imageops::FilterType::Triangle);
    let gray = image::imageops::grayscale(&thumb);
    let data = gray.into_raw();
    let mut guard = LAST_FRAME.lock().unwrap();
    let changed = match guard.as_ref() {
        Some(prev) if prev.len() == data.len() => {
            let mut diff = 0usize;
            for (a, b) in prev.iter().zip(data.iter()) {
                if (*a as i32 - *b as i32).abs() > 16 {
                    diff += 1;
                }
            }
            diff * 1000 / data.len().max(1) > 5
        }
        _ => true,
    };
    if changed {
        *guard = Some(data);
    }
    changed
}

#[tauri::command]
fn frontend_log(msg: String) {
    eprintln!("[frontend] {msg}");
}

#[tauri::command]
fn greet(name: &str) -> String {
    format!("Hello, {}! Exameow is ready.", name)
}

#[cfg(desktop)]
fn place_window(win: &tauri::WebviewWindow, x: f64, y: f64, w: f64, h: f64) -> Result<(), CommandError> {
    use tauri::{PhysicalPosition, PhysicalSize};
    win.set_position(PhysicalPosition::new(x as i32, y as i32))
        .map_err(|e| CommandError(format!("Failed to position window: {e}")))?;
    win.set_size(PhysicalSize::new(w.max(1.0) as u32, h.max(1.0) as u32))
        .map_err(|e| CommandError(format!("Failed to size window: {e}")))?;
    Ok(())
}

#[cfg(desktop)]
#[tauri::command]
async fn create_record_windows(app: tauri::AppHandle) -> Result<(), CommandError> {
    if let Some(w) = app.get_webview_window("record-overlay") { w.close().ok(); }
    if let Some(w) = app.get_webview_window("answer-float") { w.close().ok(); }

    *LAST_FRAME.lock().unwrap() = None;

    // Geometry is computed here (not in JS) on the monitor hosting the main
    // window, in physical pixels: window.screen in the webview only covers the
    // current monitor and Tauri's builder position/inner_size are logical, so
    // JS-side math breaks on multi-monitor or scaled (125%/150%) displays.
    let main_win = app.get_webview_window("main");
    let monitor = main_win
        .as_ref()
        .and_then(|w| w.current_monitor().ok().flatten())
        .or_else(|| main_win.as_ref().and_then(|w| w.primary_monitor().ok().flatten()))
        .ok_or_else(|| CommandError("No monitor found".to_string()))?;

    let sf = monitor.scale_factor();
    let mx = monitor.position().x as f64;
    let my = monitor.position().y as f64;
    let mw = monitor.size().width as f64;
    let mh = monitor.size().height as f64;

    let overlay_w = (mw * 0.6).round();
    let overlay_h = (mh * 0.4).round();
    let overlay_x = (mx + (mw - overlay_w) / 2.0).round();
    let overlay_y = (my + mh * 0.08).round();

    let float_w = (340.0 * sf).round();
    let float_h = (320.0 * sf).round();
    let float_x = (mx + mw - float_w - 20.0 * sf).round();
    let float_y = (my + mh - float_h - 40.0 * sf).round();

    #[cfg(debug_assertions)]
    let overlay_url = tauri::WebviewUrl::External("http://localhost:5273/#/src-windows/record-overlay".parse().unwrap());
    #[cfg(not(debug_assertions))]
    let overlay_url = tauri::WebviewUrl::App("/index.html#/src-windows/record-overlay".into());

    let _overlay = tauri::WebviewWindowBuilder::new(
        &app,
        "record-overlay",
        overlay_url,
    )
    // Windows quirk: a transparent window created hidden renders an opaque
    // white background when shown later (until moved/resized), so create it
    // visible with logical geometry, then correct with physical values below.
    .position(overlay_x / sf, overlay_y / sf)
    .inner_size(overlay_w / sf, overlay_h / sf)
    .min_inner_size(240.0, 120.0)
```

```
    .decorations(false)
    .transparent(true)
    .always_on_top(true)
    .resizable(true)
    .visible(true)
    .build()
    .map_err(|e| CommandError(format!("Failed to create record-overlay: {e}")))?;

    #[cfg(target_os = "macos")]
    make_webview_transparent(&_overlay);

    place_window(&_overlay, overlay_x, overlay_y, overlay_w, overlay_h)?;

    #[cfg(debug_assertions)]
    let float_url = tauri::WebviewUrl::External("http://localhost:5273/#/src-windows/answer-float".parse().unwrap());
    #[cfg(not(debug_assertions))]
    let float_url = tauri::WebviewUrl::App("/index.html#/src-windows/answer-float".into());

    let _float = tauri::WebviewWindowBuilder::new(
        &app,
        "answer-float",
        float_url,
    )
    .position(float_x / sf, float_y / sf)
    .inner_size(float_w / sf, float_h / sf)
    .min_inner_size(280.0, 200.0)
    .decorations(false)
    .transparent(true)
    .always_on_top(true)
    .resizable(true)
    .visible(true)
    .build()
    .map_err(|e| CommandError(format!("Failed to create answer-float: {e}")))?;

    #[cfg(target_os = "macos")]
    make_webview_transparent(&_float);

    place_window(&_float, float_x, float_y, float_w, float_h)?;

    Ok(())
}

#[cfg(not(desktop))]
#[tauri::command]
async fn create_record_windows(_app: tauri::AppHandle) -> Result<(), CommandError> {
    Err(CommandError(
        "Recording windows are only supported on desktop".to_string(),
    ))
}

#[cfg(desktop)]
#[tauri::command]
fn close_record_windows(app: tauri::AppHandle) -> Result<(), CommandError> {
    if let Some(w) = app.get_webview_window("record-overlay") {
        w.close().ok();
    }
    if let Some(w) = app.get_webview_window("answer-float") {
        w.close().ok();
    }
    Ok(())
}

#[cfg(not(desktop))]
#[tauri::command]
fn close_record_windows(_app: tauri::AppHandle) -> Result<(), CommandError> {
    Ok(())
}

#[cfg(desktop)]
#[tauri::command]
fn resize_record_overlay(app: tauri::AppHandle, w: f64, h: f64) -> Result<(), CommandError> {
    if let Some(win) = app.get_webview_window("record-overlay") {
        use tauri::Size;
        win.set_size(Size::Logical((w, h).into()))
            .map_err(|e| CommandError(format!("Failed to resize: {e}")))?;
    }
    Ok(())
}

#[cfg(not(desktop))]
#[tauri::command]
fn resize_record_overlay(_app: tauri::AppHandle, _w: f64, _h: f64) -> Result<(), CommandError> {
    Ok(())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_fs::init())
        .plugin(tauri_plugin_opener::init())
        .plugin(tauri_plugin_sharekit::init())
        .plugin(tauri_plugin_screenrecord::init())
        .setup(|app| {
            let config = &app.config().app.windows[0];
            let mut builder = tauri::WebviewWindowBuilder::from_config(app.handle(), config)?;

            #[cfg(target_os = "macos")]
            {
                builder = builder
                    .title_bar_style(tauri::TitleBarStyle::Overlay)
                    .hidden_title(true);
            }

            #[cfg(any(target_os = "windows", target_os = "linux"))]
            {
                builder = builder.decorations(false);
            }

            let window = builder.build()?;

            #[cfg(not(target_os = "android"))]
            window.show()?;

            Ok(())
        })
        .invoke_handler(tauri::generate_handler![
            greet,
```

```
            get_models,
            generate_exam,
            answer_question,
            judge_answer,
            parse_file_text,
            parse_file_bytes,
            export_csv,
            export_xlsx,
            export_xlsx_data,
            save_to_downloads,
            save_config,
            load_config,
            open_app_settings,
            capture_screen,
            check_screen_permission,
            request_screen_permission,
            open_screen_recording_settings,
            frontend_log,
            create_record_windows,
            close_record_windows,
            resize_record_overlay,
        ])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}


========================================================================
FILE: src-tauri/src/main.rs
========================================================================

#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
    exameow_lib::run()
}


========================================================================
FILE: workers/src/ai.ts
========================================================================

import { Ai } from '@cloudflare/workers-types'
import { DEFAULT_MODEL } from './types'

interface AIChatInput {
  model?: string
  systemPrompt: string
  userPrompt: string
}

export async function aiChat(
  ai: Ai,
  input: AIChatInput
): Promise<string> {
  const model = input.model || DEFAULT_MODEL

  const result: any = await ai.run(model as any, {
    messages: [
      { role: 'system', content: input.systemPrompt },
      { role: 'user', content: input.userPrompt },
    ],
    temperature: 0.7,
    max_tokens: 16384,
    stream: false,
  })

  console.log('AI result type:', typeof result, 'keys:', result ? Object.keys(result) : 'null')

  // CF Workers AI returns { response: string } for text generation
  if (typeof result === 'string') {
    if (!result.trim()) throw new Error('AI returned empty response')
    return result
  }

  if (result && typeof result === 'object') {
    // Handle { response: "..." }
    if (result.response && typeof result.response === 'string') {
      if (!result.response.trim()) throw new Error('AI returned empty response')
      return result.response
    }

    // Handle streaming response (ReadableStream)
    if (result.response && typeof result.response === 'object') {
      throw new Error('Unexpected streaming response from AI model')
    }

    // Some models might return { choices: [{ message: { content: "..." } }] }
    if (result.choices?.[0]?.message?.content) {
      return result.choices[0].message.content
    }
  }

  throw new Error(`AI returned unexpected response: type=${typeof result} keys=${result ? Object.keys(result).join(',') : 'null'}`)
}


========================================================================
FILE: workers/src/answer.ts
========================================================================

import { Ai } from '@cloudflare/workers-types'
import { aiChat } from './ai'
import { AnswerResult } from './types'

export async function answerQuestion(
  ai: Ai,
  question: string,
  language: string,
  model?: string
): Promise<AnswerResult> {
  const systemPrompt = buildAnswerSystemPrompt()
  const userPrompt = buildAnswerUserPrompt(question, language)
  const response = await aiChat(ai, { model, systemPrompt, userPrompt })
  return parseAnswer(response)
}

function buildAnswerSystemPrompt(): string {
```

```
  return `You are an expert exam-solving assistant. The user will give you an exam question (it may include options). Solve it.

## Output Rules
1. Respond ONLY with a valid JSON object — no explanation outside JSON, no markdown fences.
2. The JSON object MUST have exactly these fields:
   - "answer": the concise answer. For choice questions give the option letter(s) plus the option content; for true/false questions answer "True" or "False" (
   - "analysis": a clear step-by-step explanation of why this answer is correct.
3. Use the requested language for both fields.
4. If the question is ambiguous or unanswerable, still fill "answer" with your best attempt and explain the uncertainty in "analysis".`
}

function buildAnswerUserPrompt(question: string, language: string): string {
  return `Language: ${language}\n\nQUESTION:\n${question}`
}

export function parseAnswer(raw: string): AnswerResult {
  let cleaned = raw.trim()
  cleaned = cleaned.replace(/^```(?:json)?\s*\n?/i, '')
  cleaned = cleaned.replace(/\n?```\s*$/i, '')

  const start = cleaned.indexOf('{')
  const end = cleaned.lastIndexOf('}')
  const json = start >= 0 && end >= start ? cleaned.slice(start, end + 1) : cleaned

  const parsed = JSON.parse(json)
  if (typeof parsed?.answer !== 'string' || typeof parsed?.analysis !== 'string') {
    throw new Error('AI answer missing "answer"/"analysis" fields')
  }
  return { answer: parsed.answer, analysis: parsed.analysis }
}


======================================================================
FILE: workers/src/exam.ts
======================================================================

import { Ai } from '@cloudflare/workers-types'
import { aiChat } from './ai'
import { ExamParams, Question, QuestionType, Difficulty } from './types'

export async function generateExam(
  ai: Ai,
  text: string,
  params: ExamParams,
  model?: string
): Promise<Question[]> {
  const systemPrompt = buildSystemPrompt()
  const docText = params.text || text
  const userPrompt = buildUserPrompt(docText, params)
  const response = await aiChat(ai, { model, systemPrompt, userPrompt })
  console.log('AI response preview:', response.substring(0, 200))
  return parseQuestions(response)
}

function buildSystemPrompt(): string {
  const questionTypes = [
    QuestionType.SingleChoice,
    QuestionType.MultiChoice,
    QuestionType.TrueFalse,
    QuestionType.FillBlank,
    QuestionType.ShortAnswer,
  ].join(', ')

  return `You are an expert exam question generator. Generate questions based on the provided document content.

## Critical Rules (MUST follow)
- EVERY question MUST be unique — do NOT generate two questions that test the same concept, fact, or sentence.
- Cover DIFFERENT parts of the document for each question. Avoid clustering questions on the same paragraph.
- Vary question wording, angles, and tested knowledge points.
- When the question stem or analysis refers to the document, ALWAYS use the specific document name provided — NEVER use vague phrases like "the document", "th

## Output Rules
1. Respond ONLY with a valid JSON array — no explanation, no markdown fences.
2. Each question object MUST have exactly these fields:
   - "id": a short unique identifier string
   - "type": one of [${questionTypes}]
   - "stem": the question text
   - "options": array of option strings (required for single_choice/multi_choice/true_false; empty array for others)
   - "answer": the correct answer
   - "analysis": brief explanation of the answer (can be empty string for fill_blank/short_answer)
3. For single_choice: exactly 4 options, one correct.
4. For multi_choice: exactly 4 options, at least one correct (list correct letters separated by comma in answer).
5. For true_false: options ["True", "False"], answer is "True" or "False".
6. For fill_blank: answer is the exact word/phrase to fill in.
7. For short_answer: answer is a concise reference answer.
8. All questions must be based on the document content.
9. Use the specified language for questions.`
}

function buildUserPrompt(text: string, params: ExamParams): string {
  const difficultyMap: Record<Difficulty, string> = {
    [Difficulty.Easy]: 'easy questions suitable for beginners',
    [Difficulty.Medium]: 'moderate difficulty questions requiring understanding',
    [Difficulty.Hard]: 'challenging questions requiring deep analysis',
  }

  const difficultyStr = difficultyMap[params.difficulty] || difficultyMap[Difficulty.Medium]

  const topicNote = params.topic_filter
    ? `\nFocus on this topic: ${params.topic_filter}`
    : ''

  const batchNote =
    params.batch_index !== undefined &&
    params.batch_total &&
    params.batch_total > 1
      ? `\nThis is batch ${params.batch_index}/${params.batch_total} of the document. Focus on different content than other batches would.`
      : ''

  const docName = params.source_name
    ? params.source_name.includes('、')
      ? `\nThe documents are collectively titled: ${params.source_name}\nWhen questions need to reference a specific document, use its individual title above
      : `\nThe document title is: ${params.source_name}\nWhen questions need to reference this document, use "${params.source_name}" — do NOT say "the documen
    : ''

  const maxChars = 32000
  const textSection =
    text.length > maxChars
```

```
      ? text.slice(0, (maxChars * 6) / 10) +
        '\n\n...(middle omitted)...\n\n' +
        text.slice(text.length - (maxChars * 4) / 10)
      : text

  const countInstruction = params.type_counts
    ? (() => {
        const parts: string[] = []
        let total = 0
        for (const [key, cnt] of Object.entries(params.type_counts)) {
          if (cnt > 0) {
            parts.push(`${cnt} ${key} questions`)
            total += cnt
          }
        }
        return `Generate exactly the following breakdown of ${total} questions:\n${parts.join('\n')}`
      })()
    : `Generate ${params.count} questions.\nQuestion types: ${params.question_types.join(', ')}`

  return `${countInstruction}
Difficulty: ${difficultyStr}
Language: ${params.language}${topicNote}${batchNote}${docName}

DOCUMENT CONTENT:
${textSection}`
}

function parseQuestions(jsonStr: unknown): Question[] {
  if (typeof jsonStr !== 'string') {
    throw new Error(`Expected string response from AI, got ${typeof jsonStr}`)
  }

  let cleaned = jsonStr.trim()
  cleaned = cleaned.replace(/^```(?:json)?\s*\n?/i, '')
  cleaned = cleaned.replace(/\n?```\s*$/i, '')

  // Strip AI preamble (e.g. "Here is the JSON:", "The answer is:", etc.)
  const jsonStart = cleaned.search(/[\[\{]/)
  if (jsonStart > 0) {
    cleaned = cleaned.substring(jsonStart)
  }

  cleaned = cleaned.trim()

  const parsed = JSON.parse(cleaned)

  // Handle { "questions": [...] } wrapper
  let questions: Question[]
  if (Array.isArray(parsed)) {
    questions = parsed
  } else if (parsed && typeof parsed === 'object' && Array.isArray(parsed.questions)) {
    questions = parsed.questions
  } else {
    console.error('Unexpected AI response structure:', JSON.stringify(parsed).substring(0, 200))
    throw new Error('AI response is neither an array nor { questions: [...] }')
  }

  if (!questions.length) {
    throw new Error('AI returned empty questions array')
  }

  return questions
}


================================================================
FILE: workers/src/export.ts
================================================================

const TYPE_MAP: Record<string, string> = {
  single_choice: '单选题',
  multi_choice: '多选题',
  true_false: '判断题',
  fill_blank: '填空题',
  short_answer: '简答题',
}

function chineseType(qtype: string): string {
  return TYPE_MAP[qtype] || '简答题'
}

function answerLetter(answer: string, options: string[]): string {
  if (/^[A-H]$/.test(answer)) return answer
  const letters = answer.replace(/[^A-H]/g, '')
  if (letters) return letters
  const result: string[] = []
  for (const part of answer.split(',')) {
    const trimmed = part.trim()
    const idx = options.findIndex((o) => o.trim() === trimmed)
    if (idx >= 0) result.push(String.fromCharCode(65 + idx))
  }
  return result.length ? result.join(',') : answer
}

function trueFalseAnswer(answer: string): string {
  const a = answer.trim()
  if (['对', '正确', '√', '是', 'true', 'True', 'TRUE'].includes(a)) return '√'
  if (['错', '错误', '×', '否', 'false', 'False', 'FALSE'].includes(a)) return '×'
  return a
}

function escapeXml(s: string): string {
  return s
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&apos;')
}

function colLetter(idx: number): string {
  let n = idx
  const result: string[] = []
  while (true) {
    result.push(String.fromCharCode(65 + (n % 26)))
    if (n < 26) break
    n = Math.floor(n / 26) - 1
  }
```

```
  return result.reverse().join('')
}

export function generateXlsxBuffer(questions: import('./types').Question[]): Uint8Array {
  const strings: Map<string, number> = new Map()
  const sst: string[] = []

  function addSharedString(s: string): number {
    if (strings.has(s)) return strings.get(s)!
    const idx = sst.length
    sst.push(s)
    strings.set(s, idx)
    return idx
  }

  let sheetRows = ''
  let rowNum = 0

  function addRow(cells: { col: number; value: string }[]) {
    rowNum++
    const cellsXml = cells
      .map((c) => {
        const si = addSharedString(c.value)
        return `<c r="${colLetter(c.col)}${rowNum}" t="s"><v>${si}</v></c>`
      })
      .join('')
    sheetRows += `<row r="${rowNum}" spans="1:14">${cellsXml}</row>`
  }

  const headers = [
    { col: 0, value: '题干（必填）' },
    { col: 1, value: '题型 （必填）' },
    ...Array.from({ length: 8 }, (_, i) => ({
      col: 2 + i,
      value: `选项 ${String.fromCharCode(65 + i)}`,
    })),
    { col: 10, value: '正确答案\n（必填）' },
    { col: 11, value: '解析\n（勿删）' },
    { col: 12, value: '章节\n（勿删）' },
    { col: 13, value: '难度' },
  ]
  addRow(headers)

  for (const q of questions) {
    const stem = q.stem.trim()
    const qtype = chineseType(q.type)
    const analysis = q.analysis.trim()
    const answer = q.answer.trim()
    const options = q.options.map((o) => o.trim())

    const finalAnswer =
      q.type === 'single_choice' || q.type === 'multi_choice'
        ? answerLetter(answer, options)
        : q.type === 'true_false'
          ? trueFalseAnswer(answer)
          : answer

    const cells: { col: number; value: string }[] = [
      { col: 0, value: stem },
      { col: 1, value: qtype },
    ]
```

过了喵 源程序后连续30页

```
}

function toggleType(qtype: string) {
  if (props.config.typeCounts[qtype]) {
    setCount(qtype, 0)
  } else {
    setCount(qtype, Math.min(5, props.availableTypes.find(t => t.type === qtype)?.count ?? 5))
  }
}
</script>

<template>
  <div class="space-y-4">
    <h3 class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
      {{ i18n.t('practiceMockConfigTitle') }}
    </h3>

    <div v-for="item in props.availableTypes" :key="item.type" class="flex items-center gap-3">
      <button
        class="chip-filter flex-shrink-0"
        :class="{ 'chip-filter-active': !!props.config.typeCounts[item.type] }"
        @click="toggleType(item.type)"
      >
        {{ item.label }}
        <span class="text-body-sm" :style="{ color: 'rgb(var(--md-on-surface-muted))' }">
          ({{ item.count }})
        </span>
      </button>

      <Transition name="scale">
        <div v-if="props.config.typeCounts[item.type]" class="flex items-center gap-2">
          <input
            type="number"
            :value="props.config.typeCounts[item.type]"
            min="1"
            :max="item.count"
            class="input-outlined !w-16 !px-2 !py-2 text-center text-sm"
            @input="setCount(item.type, Number(($event.target as HTMLInputElement).value))"
          />
          <span class="text-body-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceQuestions') }}
          </span>
        </div>
      </Transition>
    </div>

    <button class="btn-filled w-full" :disabled="!canGenerate" @click="emit('generate')">
      {{ i18n.t('practiceMockGenerate') }}
    </button>
  </div>
</template>


================================================================
FILE: frontend/src/components/practice/ModeSelector.vue
================================================================

<script setup lang="ts">
import { computed } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import type { PracticeMode } from '@exameow/shared'
import {
  QueueListIcon,
  ArrowPathRoundedSquareIcon,
  ClockIcon,
  ExclamationTriangleIcon,
  PlayIcon,
} from '@heroicons/vue/24/outline'

const props = defineProps<{
  modelValue: PracticeMode | null
  hasWrongQuestions?: boolean
}>()

const emit = defineEmits<{
  (e: 'update:modelValue', v: PracticeMode): void
  (e: 'confirm', v: PracticeMode): void
}>()

const i18n = useI18nStore()

const modes = computed(() => {
  const all = [
    {
      value: 'sequential' as PracticeMode,
      title: i18n.t('practiceModeSequential'),
      desc: i18n.t('practiceModeSequentialDesc'),
      icon: QueueListIcon,
    },
    {
      value: 'random' as PracticeMode,
      title: i18n.t('practiceModeRandom'),
      desc: i18n.t('practiceModeRandomDesc'),
      icon: ArrowPathRoundedSquareIcon,
    },
    {
      value: 'mock' as PracticeMode,
      title: i18n.t('practiceModeMock'),
      desc: i18n.t('practiceModeMockDesc'),
      icon: ClockIcon,
    },
    {
      value: 'wrong' as PracticeMode,
      title: i18n.t('wrongModeTitle'),
      desc: i18n.t('wrongModeDesc'),
      icon: ExclamationTriangleIcon,
    },
  ]
  if (!props.hasWrongQuestions) {
    return all.filter(m => m.value !== 'wrong')
  }
  return all
})
```

```
function select(mode: PracticeMode) {
  emit('update:modelValue', mode)
}

function start() {
  if (props.modelValue) {
    emit('confirm', props.modelValue)
  }
}
</script>

<template>
  <div class="space-y-3">
    <h3 class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
      {{ i18n.t('practiceSelectModeTitle') }}
    </h3>
    <button
      v-for="m in modes"
      :key="m.value"
      class="w-full text-left p-4 rounded-xl border transition-all duration-200 flex items-start gap-4"
      :class="[
        props.modelValue === m.value
          ? 'border-[rgb(var(--md-primary))] bg-[rgba(var(--md-primary),0.08)]'
          : ''
      ]"
      :style="
        props.modelValue === m.value
          ? {}
          : {
              borderColor: 'rgb(var(--md-outline-variant))',
              backgroundColor: 'rgb(var(--md-surface))',
            }
      "
      @click="select(m.value)"
    >
      <div
        class="w-10 h-10 rounded-xl flex items-center justify-center shrink-0 mt-0.5"
        :style="{
          backgroundColor:
            props.modelValue === m.value
              ? 'rgb(var(--md-primary-container))'
              : 'rgb(var(--md-surface-container-high))',
        }"
      >
        <component
          :is="m.icon"
          class="w-5 h-5"
          :style="{
            color:
              props.modelValue === m.value
                ? 'rgb(var(--md-on-primary-container))'
                : 'rgb(var(--md-on-surface-variant))',
          }"
        />
      </div>
      <div class="flex-1 min-w-0">
        <div class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ m.title }}</div>
        <div class="text-body-sm mt-0.5" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">{{ m.desc }}</div>
      </div>
      <div
        class="w-5 h-5 rounded-full border-2 flex items-center justify-center shrink-0 mt-1.5"
        :style="{
          borderColor:
            props.modelValue === m.value
              ? 'rgb(var(--md-primary))'
              : 'rgb(var(--md-outline-variant))',
        }"
      >
        <div
          v-if="props.modelValue === m.value"
          class="w-2.5 h-2.5 rounded-full"
          :style="{ backgroundColor: 'rgb(var(--md-primary))' }"
        />
      </div>
    </button>
    <button
      class="btn-filled w-full mt-4"
      :disabled="!props.modelValue"
      @click="start"
    >
      <PlayIcon class="w-4 h-4" />
      {{ props.modelValue === 'mock' ? i18n.t('practiceMockConfigTitle') : i18n.t('practiceStartBtn') }}
    </button>
  </div>
</template>


======================================================================
FILE: frontend/src/components/practice/PracticeModeToggle.vue
======================================================================

<script setup lang="ts">
import { computed } from 'vue'
import { useI18nStore } from '@/stores/i18n'

const props = defineProps<{
  modelValue: 'exam' | 'flashcard'
}>()

const emit = defineEmits<{
  (e: 'update:modelValue', v: 'exam' | 'flashcard'): void
}>()

const i18n = useI18nStore()

const activeIndex = computed(() => props.modelValue === 'exam' ? 0 : 1)

function selectExam() {
  emit('update:modelValue', 'exam')
}

function selectFlashcard() {
  emit('update:modelValue', 'flashcard')
}
</script>

<template>
```

```
  <div
    class="mode-toggle-track"
    :style="{
      backgroundColor: 'rgb(var(--md-surface-container-high))',
      borderColor: 'rgb(var(--md-outline-variant))',
    }"
  >
    <div
      class="mode-toggle-thumb"
      :style="{
        backgroundColor: 'rgb(var(--md-primary-container))',
        color: 'rgb(var(--md-on-primary-container))',
        transform: `translateX(${activeIndex * 100}%)`,
      }"
    />
    <button
      class="mode-toggle-option"
      :class="{ 'mode-toggle-option--active': modelValue === 'exam' }"
      @click="selectExam"
    >
      <span class="text-xs font-medium truncate">{{ i18n.t('practiceModeExam') }}</span>
    </button>
    <button
      class="mode-toggle-option"
      :class="{ 'mode-toggle-option--active': modelValue === 'flashcard' }"
      @click="selectFlashcard"
    >
      <span class="text-xs font-medium truncate">{{ i18n.t('practiceModeFlashcard') }}</span>
    </button>
  </div>
</template>


================================================================
FILE: frontend/src/components/practice/PracticeResult.vue
================================================================

<script setup lang="ts">
import { computed } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import type { PracticeSession } from '@exameow/shared'
import {
  CheckCircleIcon,
  XCircleIcon,
  ClockIcon,
} from '@heroicons/vue/24/outline'

const props = defineProps<{
  session: PracticeSession
  score: number
  autoGradedCount: number
  elapsedText: string
}>()

const emit = defineEmits<{
  (e: 'retry'): void
  (e: 'home'): void
}>()

const i18n = useI18nStore()

const total = computed(() => props.session.questions.length)
const accuracy = computed(() => {
  if (props.autoGradedCount === 0) return 0
  return Math.round((props.score / props.autoGradedCount) * 100)
})

const wrongQuestions = computed(() => {
  return props.session.questions
    .map((q, i) => ({ ...q, originalIndex: i }))
    .filter(q => q.isCorrect === false)
})

const correctCount = computed(() => {
  return props.session.questions.filter(q => q.isCorrect === true).length
})

const wrongCount = computed(() => {
  return wrongQuestions.value.length
})

const typeLabel = (type: string): string => {
  const labels: Record<string, string> = {
    single_choice: i18n.t('typeSingle'),
    multi_choice: i18n.t('typeMulti'),
    true_false: i18n.t('typeTrueFalse'),
    fill_blank: i18n.t('typeFillBlank'),
    short_answer: i18n.t('typeShortAnswer'),
  }
  return labels[type] ?? type
}
</script>

<template>
  <div class="space-y-5">
    <!-- Score Card -->
    <div class="card-filled p-5 sm:p-6 text-center">
      <div
        class="w-16 h-16 rounded-full flex items-center justify-center mx-auto mb-3 elevation-1"
        :style="{ backgroundColor: 'rgb(var(--md-primary))' }"
      >
        <span class="text-2xl font-bold" :style="{ color: 'rgb(var(--md-on-primary))' }">{{ score }}</span>
      </div>
      <h2 class="text-headline-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface))' }">
        {{ i18n.t('practiceResultTitle') }}
      </h2>
      <p class="text-body-md" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
        {{ score }} / {{ total }}
      </p>
    </div>

    <!-- Stats -->
    <div class="grid grid-cols-2 gap-3">
      <div class="card-outlined p-4 text-center">
        <CheckCircleIcon class="w-6 h-6 mx-auto mb-1" :style="{ color: 'rgb(var(--md-primary))' }" />
        <div class="text-title-md" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ correctCount }}</div>
        <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ i18n.t('practiceCorrect') }}
```

```
        </div>
      </div>
      <div class="card-outlined p-4 text-center">
        <XCircleIcon class="w-6 h-6 mx-auto mb-1" :style="{ color: 'rgb(var(--md-error))' }" />
        <div class="text-title-md" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ wrongCount }}</div>
        <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ i18n.t('practiceIncorrect') }}
        </div>
      </div>
    </div>

    <!-- Accuracy & Time -->
    <div class="card-outlined p-4 flex items-center justify-between">
      <div class="text-center flex-1">
        <div class="text-display-sm font-bold" :style="{ color: 'rgb(var(--md-primary))' }">
          {{ autoGradedCount > 0 ? accuracy + '%' : '--' }}
        </div>
        <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ i18n.t('practiceAccuracy') }}
        </div>
      </div>
      <div class="w-px h-10 mx-2" :style="{ backgroundColor: 'rgb(var(--md-outline-variant))' }" />
      <div class="text-center flex-1 flex items-center justify-center gap-2">
        <ClockIcon class="w-5 h-5" :style="{ color: 'rgb(var(--md-on-surface-variant))' }" />
        <div>
          <div class="text-title-md" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ elapsedText }}</div>
          <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceElapsed') }}
          </div>
        </div>
      </div>
    </div>

    <!-- Wrong Questions Review -->
    <div v-if="wrongQuestions.length > 0" class="space-y-3">
      <h3 class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
        {{ i18n.t('practiceReviewTitle') }} ({{ wrongQuestions.length }})
      </h3>

      <div
        v-for="item in wrongQuestions"
        :key="item.originalIndex"
        class="card-outlined p-4"
      >
        <div class="flex items-start gap-2 mb-2">
          <span
            class="inline-flex items-center justify-center w-6 h-6 rounded-full text-[10px] font-bold shrink-0 mt-0.5"
            :style="{
              backgroundColor: 'rgb(var(--md-error-container))',
              color: 'rgb(var(--md-on-error-container))',
            }"
          >{{ item.originalIndex + 1 }}</span>
          <div class="flex-1">
            <div class="text-sm mb-2" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ item.question.stem }}</div>

            <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
              {{ i18n.t('practiceReviewYourAnswer') }}
            </div>
            <div class="text-sm mb-2" :style="{ color: 'rgb(var(--md-error))' }">
              {{ item.userAnswer || i18n.t('practiceUnanswered') }}
            </div>

            <div class="text-label-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
              {{ i18n.t('practiceReviewCorrectAnswer') }}
            </div>
            <div class="text-sm mb-2" :style="{ color: 'rgb(var(--md-primary))' }">
              {{ item.question.answer }}
            </div>

            <div v-if="item.question.analysis" class="text-xs mt-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
              {{ item.question.analysis }}
            </div>
          </div>
        </div>
      </div>
    </div>

    <!-- Actions -->
    <div class="flex gap-3 pt-2">
      <button class="btn-tonal flex-1" @click="emit('retry')">
        {{ i18n.t('practiceRetryBtn') }}
      </button>
      <button class="btn-outlined flex-1" @click="emit('home')">
        {{ i18n.t('practiceBackToBanks') }}
      </button>
    </div>
  </div>
</template>


======================================================================
FILE: frontend/src/components/practice/ProgressBar.vue
======================================================================

<script setup lang="ts">
import { useI18nStore } from '@/stores/i18n'
import { usePracticeStore } from '@/stores/practice'
import { TableCellsIcon } from '@heroicons/vue/24/outline'

defineProps<{
  mode: string
  current: number
  total: number
  answeredCount: number
}>()

const emit = defineEmits<{
  (e: 'openSheet'): void
}>()

const i18n = useI18nStore()
</script>

<template>
  <div class="flex items-center gap-3">
    <button
      class="flex items-center gap-1.5 px-2.5 py-1.5 rounded-full text-xs sm:text-sm font-medium transition-all duration-200 shrink-0"
      :style="{
```

```
        backgroundColor: 'rgb(var(--md-surface-container-highest))',
        color: 'rgb(var(--md-on-surface))',
      }"
      @click="emit('openSheet')"
    >
      <TableCellsIcon class="w-3.5 h-3.5 sm:w-4 sm:h-4" />
      <span class="hidden sm:inline">{{ i18n.t('practiceAnswerSheet') }}</span>
      <span class="tabular-nums text-xs" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
        {{ current }}/{{ total }}
      </span>
    </button>

    <div class="flex-1 min-w-0">
      <div class="h-1.5 rounded-full overflow-hidden w-full" :style="{ backgroundColor: 'rgba(var(--md-primary) / 0.12)' }">
        <div
          class="h-full rounded-full transition-all duration-500 ease-out"
          :style="{
            backgroundColor: 'rgb(var(--md-primary))',
            width: ((current / total) * 100) + '%',
          }"
        />
      </div>
    </div>
  </div>
</template>


======================================================================
FILE: frontend/src/components/practice/QuestionCard.vue
======================================================================

<script setup lang="ts">
import { ref, computed } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import type { Question, PracticeMode } from '@exameow/shared'
import {
  CheckCircleIcon,
  XCircleIcon,
  XMarkIcon,
  SparklesIcon,
} from '@heroicons/vue/24/outline'

const props = defineProps<{
  question: Question
  mode: PracticeMode
  userAnswer: string | null
  isCorrect: boolean | null
  submitted: boolean
  questionNumber: number
  autoAdvancing: boolean
  wrongCount?: number
  isWrongMode?: boolean
  flashcardMode?: boolean
  aiConfigured?: boolean
  aiJudging?: boolean
  aiFeedback?: string | null
  aiJudgeError?: string | null
}>()

const emit = defineEmits<{
  (e: 'submit', answer: string | null): void
  (e: 'selfCheck', correct: boolean): void
  (e: 'select', answer: string | null): void
  (e: 'removeWrong'): void
  (e: 'aiJudge'): void
  (e: 'aiCancel'): void
  (e: 'regrade', correct: boolean): void
}>()

const i18n = useI18nStore()

const answerRevealed = ref(false)

const isChoiceType = computed(() => {
  return props.question.type === 'single_choice' ||
    props.question.type === 'multi_choice' ||
    props.question.type === 'true_false'
})

const isSelfCheckType = computed(() => {
  return props.question.type === 'fill_blank' || props.question.type === 'short_answer'
})

const showFlashcardPreview = computed(() => {
  return props.flashcardMode === true && !props.submitted
})

const interactive = computed(() => {
  return !props.submitted && !props.flashcardMode
})

const optionLabels = 'ABCDEFGH'.split('')

const correctAnswerSet = computed(() => {
  if (props.question.type === 'true_false') {
    const a = props.question.answer.trim()
    const isTrue = ['A', '√', '对', '正确', 'TRUE', 'T', '是', 'YES', 'Y', '1'].some(
      v => a.toUpperCase() === v.toUpperCase() || a.includes(v)
    )
    return new Set<string>(isTrue ? ['A'] : ['B'])
  }
  return new Set(props.question.answer.trim().toUpperCase().replace(/[^A-H]/g, '').split(''))
})

const selectedSet = computed(() => {
  if (!props.userAnswer) return new Set<string>()
  return new Set(props.userAnswer.trim().toUpperCase().replace(/[^A-H]/g, '').split(''))
})

const canSubmit = computed(() => {
  if (props.submitted) return false
  if (props.question.type === 'multi_choice') return selectedSet.value.size > 0
  if (isChoiceType.value) return selectedSet.value.size > 0
  return false
})

const typeLabel = computed(() => {
  const labels: Record<string, string> = {
```

```
    single_choice: i18n.t('typeSingle'),
    multi_choice: i18n.t('typeMulti'),
    true_false: i18n.t('typeTrueFalse'),
    fill_blank: i18n.t('typeFillBlank'),
    short_answer: i18n.t('typeShortAnswer'),
  }
  return labels[props.question.type] ?? props.question.type
})

const trueFalseOptions = computed(() => {
  const locale = i18n.locale
  return [
    { label: locale === 'zh' ? 'A. 对 (True)' : 'A. True', value: 'A' },
    { label: locale === 'zh' ? 'B. 错 (False)' : 'B. False', value: 'B' },
  ]
})

function selectOption(opt: string) {
  if (!interactive.value) return

  if (props.question.type === 'single_choice' || props.question.type === 'true_false') {
    emit('submit', opt)
    return
  }
  const current = new Set(selectedSet.value)
  if (current.has(opt)) {
    current.delete(opt)
  } else {
    current.add(opt)
  }
  const ans = Array.from(current).sort().join('')
  emit('select', ans || null)
}

function confirmMultiChoice() {
  emit('submit', props.userAnswer)
}

function handleTextInput(e: Event) {
  const val = (e.target as HTMLInputElement | HTMLTextAreaElement).value
  emit('select', val || null)
}

function handleSelfCheck(correct: boolean) {
  if (props.submitted) return
  emit('selfCheck', correct)
}

function getOptionStyle(opt: string, idx: number) {
  if (showFlashcardPreview.value) {
    const isCorrect = correctAnswerSet.value.has(opt)
    if (isCorrect) {
      return {
        borderColor: 'rgb(var(--md-primary))',
        backgroundColor: 'rgba(var(--md-primary), 0.12)',
      }
    }
    return {
      borderColor: 'rgb(var(--md-outline-variant))',
      backgroundColor: 'transparent',
    }
  }

  if (!props.submitted) {
    const isSelected = selectedSet.value.has(opt)
    return {
      borderColor: isSelected ? 'rgb(var(--md-primary))' : 'rgb(var(--md-outline-variant))',
      backgroundColor: isSelected ? 'rgba(var(--md-primary), 0.08)' : 'transparent',
    }
  }

  const isCorrect = correctAnswerSet.value.has(opt)
  const wasChosen = selectedSet.value.has(opt)

  if (isCorrect && wasChosen) {
    return {
      borderColor: 'rgb(var(--md-primary))',
      backgroundColor: 'rgba(var(--md-primary), 0.12)',
    }
  }
  if (isCorrect && !wasChosen) {
    return {
      borderColor: 'rgb(var(--md-primary))',
      backgroundColor: 'rgba(var(--md-primary), 0.06)',
    }
  }
  if (!isCorrect && wasChosen) {
    return {
      borderColor: 'rgb(var(--md-error))',
      backgroundColor: 'rgba(var(--md-error), 0.08)',
    }
  }
  return {
    borderColor: 'rgb(var(--md-outline-variant))',
    backgroundColor: 'transparent',
  }
}

function getBadgeStyle(opt: string) {
  if (showFlashcardPreview.value) {
    const isCorrect = correctAnswerSet.value.has(opt)
    return {
      backgroundColor: isCorrect ? 'rgb(var(--md-primary))' : 'rgb(var(--md-surface-container-highest))',
      color: isCorrect ? 'rgb(var(--md-on-primary))' : 'rgb(var(--md-on-surface-variant))',
    }
  }

  if (!props.submitted) {
    const isSelected = selectedSet.value.has(opt)
    return {
      backgroundColor: isSelected ? 'rgb(var(--md-primary))' : 'rgb(var(--md-surface-container-highest))',
      color: isSelected ? 'rgb(var(--md-on-primary))' : 'rgb(var(--md-on-surface-variant))',
    }
  }
  const isCorrect = correctAnswerSet.value.has(opt)
  const wasChosen = selectedSet.value.has(opt)

  if (isCorrect && wasChosen) {
```

```
    return { backgroundColor: 'rgb(var(--md-primary))', color: 'rgb(var(--md-on-primary))' }
  }
  if (isCorrect && !wasChosen) {
    return { backgroundColor: 'rgb(var(--md-primary-container))', color: 'rgb(var(--md-on-primary-container))' }
  }
  if (!isCorrect && wasChosen) {
    return { backgroundColor: 'rgb(var(--md-error))', color: 'rgb(var(--md-on-error))' }
  }
  return { backgroundColor: 'rgb(var(--md-surface-container-highest))', color: 'rgb(var(--md-on-surface-variant))' }
}
</script>

<template>
  <div class="card-elevated p-4 sm:p-6">
    <!-- Header -->
    <div class="flex items-center justify-between mb-4">
      <div class="flex items-center gap-2">
        <span
          class="inline-flex items-center justify-center w-7 h-7 rounded-full text-xs font-bold"
          :style="{
            backgroundColor: 'rgb(var(--md-primary-container))',
            color: 'rgb(var(--md-on-primary-container))',
          }"
        >{{ questionNumber }}</span>
        <span
          class="inline-block px-2.5 py-0.5 rounded-full text-[11px] font-medium"
          :style="{
            backgroundColor: 'rgb(var(--md-secondary-container))',
            color: 'rgb(var(--md-on-secondary-container))',
          }"
        >{{ typeLabel }}</span>
        <span
          v-if="flashcardMode"
          class="inline-block px-2.5 py-0.5 rounded-full text-[11px] font-medium"
          :style="{
            backgroundColor: 'rgb(var(--md-tertiary-container))',
            color: 'rgb(var(--md-on-tertiary-container))',
          }"
        >{{ i18n.t('practiceModeFlashcard') }}</span>
      </div>

      <!-- Result badge after submit -->
      <div v-if="submitted && isCorrect !== null" class="flex items-center gap-1.5">
        <template v-if="isCorrect">
          <CheckCircleIcon class="w-5 h-5" :style="{ color: 'rgb(var(--md-primary))' }" />
          <span class="text-sm font-bold" :style="{ color: 'rgb(var(--md-primary))' }">
            {{ i18n.t('practiceCorrect') }}
          </span>
        </template>
        <template v-else>
          <XCircleIcon class="w-5 h-5" :style="{ color: 'rgb(var(--md-error))' }" />
          <span class="text-sm font-bold" :style="{ color: 'rgb(var(--md-error))' }">
            {{ i18n.t('practiceIncorrect') }}
          </span>
        </template>
      </div>

      <!-- Remove from wrong questions button -->
      <button
        v-if="isWrongMode && !submitted"
        class="btn-tonal !h-8 text-xs !px-3 shrink-0"
        :style="{ borderColor: 'rgb(var(--md-error))', color: 'rgb(var(--md-error))' }"
        @click="emit('removeWrong')"
      >
        <XMarkIcon class="w-3.5 h-3.5" />
        {{ i18n.t('practiceRemoveWrong') }}
      </button>
    </div>

    <!-- Stem -->
    <div class="text-body-lg mb-5" :style="{ color: 'rgb(var(--md-on-surface))' }">
      {{ question.stem }}
    </div>

    <!-- Wrong count badge -->
    <div
      v-if="wrongCount !== undefined && wrongCount > 0"
      class="inline-flex items-center gap-1 px-2.5 py-1 rounded-full text-xs font-medium mb-4"
      :style="{
        backgroundColor: 'rgba(var(--md-error), 0.12)',
        color: 'rgb(var(--md-error))',
      }"
    >
      <XCircleIcon class="w-3.5 h-3.5" />
      {{ i18n.t('wrongTimesCount', { n: wrongCount }) }}
    </div>

    <!-- Choice / TrueFalse Options -->
    <div v-if="isChoiceType" class="space-y-2 mb-4">
      <template v-if="question.type === 'true_false'">
        <button
          v-for="opt in trueFalseOptions"
          :key="opt.value"
          class="w-full text-left p-3 rounded-xl border transition-all duration-200 flex items-center gap-3"
          :disabled="!interactive"
          :style="getOptionStyle(opt.value, 0)"
          @click="selectOption(opt.value)"
        >
          <div
            class="w-8 h-8 rounded-full flex items-center justify-center text-xs font-bold shrink-0"
            :style="getBadgeStyle(opt.value)"
          >{{ opt.value }}</div>
          <span class="text-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
            {{ opt.label.replace(/^[A-Z]\.\s*/, '') }}
          </span>
          <CheckCircleIcon
            v-if="(submitted || showFlashcardPreview) && correctAnswerSet.has(opt.value)"
            class="w-5 h-5 ml-auto"
            :style="{ color: 'rgb(var(--md-primary))' }"
          />
          <XCircleIcon
            v-if="submitted && !correctAnswerSet.has(opt.value) && selectedSet.has(opt.value)"
            class="w-5 h-5 ml-auto"
            :style="{ color: 'rgb(var(--md-error))' }"
          />
        </button>
      </template>
      <template v-else>
```

```
    <button
      v-for="(opt, idx) in question.options"
      :key="idx"
      class="w-full text-left p-3 rounded-xl border transition-all duration-200 flex items-center gap-3"
      :disabled="!interactive"
      :style="getOptionStyle(optionLabels[idx]!, idx)"
      @click="selectOption(optionLabels[idx]!)"
    >
      <div
        class="w-8 h-8 rounded-full flex items-center justify-center text-xs font-bold shrink-0"
        :style="getBadgeStyle(optionLabels[idx]!)"
      >{{ optionLabels[idx] }}</div>
      <span class="text-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ opt }}</span>
      <CheckCircleIcon
        v-if="(submitted || showFlashcardPreview) && correctAnswerSet.has(optionLabels[idx]!)"
        class="w-5 h-5 ml-auto"
        :style="{ color: 'rgb(var(--md-primary))' }"
      />
      <XCircleIcon
        v-if="submitted && !correctAnswerSet.has(optionLabels[idx]!) && selectedSet.has(optionLabels[idx]!)"
        class="w-5 h-5 ml-auto"
        :style="{ color: 'rgb(var(--md-error))' }"
      />
    </button>
  </template>

  <!-- Multi-choice confirm button -->
  <div v-if="question.type === 'multi_choice' && interactive" class="mt-3">
    <button
      class="btn-filled w-full"
      :disabled="!canSubmit"
      @click="confirmMultiChoice"
    >
      {{ i18n.t('practiceConfirmAnswer') }}
    </button>
  </div>

  <div v-if="question.type === 'single_choice' && interactive" class="text-xs mt-2 text-center" :style="{ color: 'rgb(var(--md-on-surface-muted))' }">
    {{ i18n.t('practiceClickToSubmit') }}
  </div>
  <div v-if="question.type === 'true_false' && interactive" class="text-xs mt-2 text-center" :style="{ color: 'rgb(var(--md-on-surface-muted))' }">
    {{ i18n.t('practiceClickToSubmit') }}
  </div>
  <div v-if="autoAdvancing" class="text-xs mt-1 text-center font-medium" :style="{ color: 'rgb(var(--md-primary))' }">
    {{ i18n.t('practiceCorrectAdvancing') }}
  </div>
</div>

<!-- Fill Blank / Short Answer -->
<div v-if="isSelfCheckType" class="mb-4">
  <textarea
    v-if="question.type === 'short_answer'"
    class="input-outlined !min-h-[100px]"
    :value="userAnswer ?? ''"
    :placeholder="i18n.t('practiceInputAnswer')"
    :disabled="!interactive"
    @input="handleTextInput"
  />
  <input
    v-else
    class="input-outlined"
    :value="userAnswer ?? ''"
    :placeholder="i18n.t('practiceInputAnswerShort')"
    :disabled="!interactive"
    @input="handleTextInput"
  />

  <!-- Action buttons (before reveal, before submit) -->
  <template v-if="userAnswer && interactive && !answerRevealed">
    <div class="flex gap-2 mt-3">
      <button
        v-if="question.type === 'fill_blank'"
        class="btn-filled !h-9 text-sm flex-1"
        @click="emit('submit', userAnswer)"
      >
        {{ i18n.t('practiceSubmitAuto') }}
      </button>
      <button
        v-if="question.type === 'short_answer'"
        class="btn-filled !h-9 text-sm flex-1"
        :disabled="!aiConfigured || aiJudging"
        @click="emit('aiJudge')"
      >
        <SparklesIcon class="w-4 h-4" />
        {{ aiJudging ? i18n.t('practiceAiJudging') : i18n.t('practiceAiJudge') }}
      </button>
      <button
        v-if="aiJudging"
        class="btn-outlined !h-9 text-sm shrink-0"
        @click="emit('aiCancel')"
      >
        {{ i18n.t('searchCancel') }}
      </button>
      <button
        v-else
        class="btn-outlined !h-9 text-sm flex-1"
        @click="answerRevealed = true"
      >
        {{ i18n.t('practiceRevealAnswer') }}
      </button>
    </div>
    <div class="text-xs mt-2 text-center" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
      {{ question.type === 'fill_blank' ? i18n.t('practiceSubmitAutoHint') : i18n.t('practiceAiJudgeHint') }}
      · {{ i18n.t('practiceRevealHint') }}
    </div>
    <div
      v-if="question.type === 'short_answer' && !aiConfigured"
      class="text-xs mt-1 text-center"
      :style="{ color: 'rgb(var(--md-error))' }"
    >
      {{ i18n.t('searchNotConfigured') }}
    </div>
    <div
      v-if="aiJudgeError"
      class="mt-2 p-3 rounded-xl text-sm flex items-center justify-between gap-2"
      :style="{ backgroundColor: 'rgba(var(--md-error), 0.08)', color: 'rgb(var(--md-error))' }"
    >
      <span class="min-w-0 break-words">{{ aiJudgeError }}</span>
```

```
            <button class="btn-tonal !h-7 !px-3 text-xs shrink-0" @click="emit('aiJudge')">
              {{ i18n.t('searchRetry') }}
            </button>
          </div>
        </template>

        <!-- Revealed answer panel (before submit) -->
        <div
          v-if="answerRevealed && interactive"
          class="mt-3 p-3 rounded-xl"
          :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))' }"
        >
          <div class="text-label-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceReviewCorrectAnswer') }}
          </div>
          <div class="text-sm" :style="{ color: 'rgb(var(--md-primary))' }">
            {{ question.answer }}
          </div>
          <div v-if="question.analysis" class="mt-2 text-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            <span class="text-label-sm">{{ i18n.t('practiceReviewAnalysis') }}: </span>{{ question.analysis }}
          </div>
        </div>

        <!-- Self-check buttons (only after reveal) -->
        <div v-if="userAnswer && interactive && answerRevealed" class="flex gap-2 mt-3">
          <button class="btn-tonal !h-9 text-sm flex-1" @click="handleSelfCheck(true)">
            <CheckCircleIcon class="w-4 h-4" />
            {{ i18n.t('practiceSelfCheckCorrect') }}
          </button>
          <button class="btn-outlined !h-9 text-sm flex-1" @click="handleSelfCheck(false)">
            {{ i18n.t('practiceSelfCheckWrong') }}
          </button>
        </div>

        <!-- Result after submit -->
        <div v-if="submitted" class="mt-3 p-3 rounded-xl" :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))' }">
          <div class="text-label-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceReviewYourAnswer') }}
          </div>
          <div class="text-sm mb-3" :style="{ color: isCorrect ? 'rgb(var(--md-primary))' : 'rgb(var(--md-error))' }">
            {{ userAnswer || i18n.t('practiceUnanswered') }}
          </div>
          <div class="text-label-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceReviewCorrectAnswer') }}
          </div>
          <div class="text-sm" :style="{ color: 'rgb(var(--md-primary))' }">
            {{ question.answer }}
          </div>
          <div v-if="question.analysis" class="mt-2 text-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            <span class="text-label-sm">{{ i18n.t('practiceReviewAnalysis') }}: </span>{{ question.analysis }}
          </div>
          <div v-if="aiFeedback" class="mt-2 text-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            <span class="text-label-sm">{{ i18n.t('practiceAiFeedback') }}: </span>{{ aiFeedback }}
          </div>
        </div>

        <!-- Regrade (short answer, after submit) -->
        <div v-if="submitted && question.type === 'short_answer' && !flashcardMode" class="mt-3">
          <div class="text-xs mb-2 text-center" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceRegradeHint') }}
          </div>
          <div class="flex gap-2">
            <button class="btn-tonal !h-9 text-sm flex-1" @click="emit('regrade', true)">
              <CheckCircleIcon class="w-4 h-4" />
              {{ i18n.t('practiceSelfCheckCorrect') }}
            </button>
            <button class="btn-outlined !h-9 text-sm flex-1" @click="emit('regrade', false)">
              {{ i18n.t('practiceSelfCheckWrong') }}
            </button>
          </div>
        </div>

        <!-- Flashcard preview for self-check types -->
        <div v-if="showFlashcardPreview" class="mt-3 p-3 rounded-xl" :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))' }">
          <div class="text-label-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceReviewCorrectAnswer') }}
          </div>
          <div class="text-sm" :style="{ color: 'rgb(var(--md-primary))' }">
            {{ question.answer }}
          </div>
          <div v-if="question.analysis" class="mt-2 text-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            <span class="text-label-sm">{{ i18n.t('practiceReviewAnalysis') }}: </span>{{ question.analysis }}
          </div>
        </div>
      </div>

      <!-- Correct answer display for choice types (when wrong) -->
      <div
        v-if="submitted && isCorrect === false && isChoiceType"
        class="mt-4 p-3 rounded-xl"
        :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))' }"
      >
        <div class="text-label-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ i18n.t('practiceReviewCorrectAnswer') }}
        </div>
        <div class="text-sm font-bold" :style="{ color: 'rgb(var(--md-primary))' }">
          {{ question.answer }}
        </div>
      </div>

      <!-- Analysis for choice types (flashcard preview or submitted) -->
      <div v-if="showFlashcardPreview && isChoiceType && question.analysis" class="mt-3">
        <div class="px-3 py-2.5 rounded-xl text-sm" :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))', color: 'rgb(var(--md-on-surface-variant))
          <span class="text-label-sm mr-2" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">{{ i18n.t('practiceReviewAnalysis') }}: </span>
          {{ question.analysis }}
        </div>
      </div>

      <!-- Analysis for choice types (normal submitted) -->
      <div v-if="submitted && question.analysis && !showFlashcardPreview" class="mt-3">
        <div class="px-3 py-2.5 rounded-xl text-sm" :style="{ backgroundColor: 'rgb(var(--md-surface-container-low))', color: 'rgb(var(--md-on-surface-variant))
          <span class="text-label-sm mr-2" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">{{ i18n.t('practiceReviewAnalysis') }}: </span>
          {{ question.analysis }}
        </div>
      </div>
    </div>
  </template>
```

```
================================================================
FILE: frontend/src/components/practice/WrongQuestionManager.vue
================================================================

<script setup lang="ts">
import { computed, ref } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import { usePracticeStore } from '@/stores/practice'
import { useWrongQuestionsStore } from '@/stores/wrongQuestions'
import type { WrongSort, WrongQuestionEntry } from '@exameow/shared'
import {
  ArrowDownIcon,
  ArrowUpIcon,
  ClockIcon,
  XMarkIcon,
  ExclamationTriangleIcon,
  TrashIcon,
  DocumentTextIcon,
} from '@heroicons/vue/24/outline'

const props = defineProps<{
  bankId: string
}>()

const emit = defineEmits<{
  (e: 'close'): void
}>()

const i18n = useI18nStore()
const practiceStore = usePracticeStore()
const wrongStore = useWrongQuestionsStore()

const sort = ref<WrongSort>('count-desc')
const showClearConfirm = ref(false)

const bank = computed(() => practiceStore.getBank(props.bankId))
const entries = computed(() => {
  const map = wrongStore.getBankEntryMap(props.bankId)
  return Object.values(map)
    .filter(e => bank.value?.questions.some(q => q.id === e.questionId))
    .sort((a, b) => {
      switch (sort.value) {
        case 'count-desc': return b.wrongCount - a.wrongCount
        case 'count-asc': return a.wrongCount - b.wrongCount
        case 'time-desc': return b.lastWrongAt - a.lastWrongAt
        case 'time-asc': return a.lastWrongAt - b.lastWrongAt
      }
    })
})

function getQuestionStem(entry: WrongQuestionEntry): string {
  const q = bank.value?.questions.find(q => q.id === entry.questionId)
  return q?.stem ?? i18n.t('practiceQuestionGone')
}

function formatDate(ts: number): string {
  const d = new Date(ts)
  return `${d.getFullYear()}/${String(d.getMonth() + 1).padStart(2, '0')}/${String(d.getDate()).padStart(2, '0')}`
}

function handleRemove(questionId: string) {
  wrongStore.removeWrong(props.bankId, questionId)
}

function handleClear() {
  wrongStore.clearBank(props.bankId)
  showClearConfirm.value = false
}

const sortOptions: { value: WrongSort; label: string; icon: any }[] = [
  { value: 'count-desc', label: i18n.t('wrongSortCountDesc'), icon: ArrowDownIcon },
  { value: 'count-asc', label: i18n.t('wrongSortCountAsc'), icon: ArrowUpIcon },
  { value: 'time-desc', label: i18n.t('wrongSortTimeDesc'), icon: ClockIcon },
  { value: 'time-asc', label: i18n.t('wrongSortTimeAsc'), icon: ClockIcon },
]
</script>

<template>
  <div class="scrim flex items-center justify-center p-4" @click.self="emit('close')">
    <div class="card-elevated w-full max-w-lg max-h-[80vh] flex flex-col">
      <!-- Header -->
      <div class="flex items-center justify-between p-4 border-b" :style="{ borderColor: 'rgb(var(--md-outline-variant))' }">
        <div>
          <div class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
            {{ i18n.t('wrongManagerTitle') }} - {{ bank?.name ?? '' }}
          </div>
          <div class="text-body-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ entries.length }} {{ i18n.t('wrongCount') }}
          </div>
        </div>
        <button class="btn-icon !w-8 !h-8" @click="emit('close')">
          <XMarkIcon class="w-5 h-5" />
        </button>
      </div>

      <!-- Sort bar -->
      <div v-if="entries.length > 0" class="flex items-center gap-2 px-4 py-2 border-b" :style="{ borderColor: 'rgb(var(--md-outline-variant))' }">
        <template v-for="opt in sortOptions" :key="opt.value">
          <button
            class="text-xs px-2.5 py-1 rounded-full transition-all duration-200"
            :style="{
              backgroundColor: sort === opt.value ? 'rgb(var(--md-secondary-container))' : 'transparent',
              color: sort === opt.value ? 'rgb(var(--md-on-secondary-container))' : 'rgb(var(--md-on-surface-variant))',
            }"
            @click="sort = opt.value"
          >
            {{ opt.label }}
          </button>
        </template>
        <button
          class="btn-icon !w-7 !h-7 ml-auto"
          :style="{ color: 'rgb(var(--md-error))' }"
          @click="showClearConfirm = true"
        >
          <TrashIcon class="w-4 h-4" />
        </button>
      </div>
```

```
      <!-- List -->
      <div class="overflow-y-auto flex-1 p-4">
        <div
          v-if="entries.length === 0"
          class="text-center py-8"
        >
          <DocumentTextIcon class="w-10 h-10 mx-auto mb-3" :style="{ color: 'rgb(var(--md-on-surface-muted))' }" />
          <div class="text-body-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('wrongManagerEmpty') }}
          </div>
        </div>

        <div v-else class="space-y-2">
          <div
            v-for="entry in entries"
            :key="entry.questionId"
            class="card-outlined p-3 flex items-start gap-3"
          >
            <div class="flex-1 min-w-0">
              <div class="text-sm line-clamp-2" :style="{ color: 'rgb(var(--md-on-surface))' }">
                {{ getQuestionStem(entry) }}
              </div>
              <div class="flex items-center gap-3 mt-1.5 text-xs" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
                <span class="flex items-center gap-1">
                  <ExclamationTriangleIcon class="w-3 h-3" :style="{ color: 'rgb(var(--md-error))' }" />
                  {{ i18n.t('wrongTimesCount', { n: entry.wrongCount }) }}
                </span>
                <span class="flex items-center gap-1">
                  <ClockIcon class="w-3 h-3" />
                  {{ formatDate(entry.lastWrongAt) }}
                </span>
              </div>
            </div>
            <button
              class="btn-icon !w-7 !h-7 shrink-0"
              :style="{ color: 'rgb(var(--md-on-surface-variant))' }"
              @click="handleRemove(entry.questionId)"
            >
              <XMarkIcon class="w-4 h-4" />
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>

  <!-- Clear Confirm Dialog -->
  <Transition name="scale">
    <div v-if="showClearConfirm" class="scrim flex items-center justify-center p-4" @click.self="showClearConfirm = false">
      <div class="card-elevated w-full max-w-sm p-5 text-center">
        <TrashIcon class="w-10 h-10 mx-auto mb-3" :style="{ color: 'rgb(var(--md-error))' }" />
        <div class="text-title-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface))' }">
          {{ i18n.t('wrongClearConfirm') }}
        </div>
        <p class="text-body-sm mb-4" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ i18n.t('wrongClearConfirmMsg') }}
        </p>
        <div class="flex gap-3">
          <button class="btn-outlined flex-1" @click="showClearConfirm = false">{{ i18n.t('btnBack') }}</button>
          <button
            class="btn-filled flex-1"
            :style="{ backgroundColor: 'rgb(var(--md-error))', color: 'rgb(var(--md-on-error))' }"
            @click="handleClear"
          >
            {{ i18n.t('wrongClearBtn') }}
          </button>
        </div>
      </div>
    </div>
  </Transition>
</template>


======================================================================
FILE: frontend/src/components/practice/WrongQuestionsSortDialog.vue
======================================================================

<script setup lang="ts">
import { ref } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import type { WrongSort } from '@exameow/shared'
import {
  ArrowDownIcon,
  ArrowUpIcon,
  ClockIcon,
  ExclamationTriangleIcon,
  PlayIcon,
} from '@heroicons/vue/24/outline'

const props = defineProps<{
  wrongCount: number
}>()

const emit = defineEmits<{
  (e: 'start', sort: WrongSort): void
  (e: 'close'): void
}>()

const i18n = useI18nStore()

const sorts: { value: WrongSort; label: string; icon: any }[] = [
  { value: 'count-desc', label: i18n.t('wrongSortCountDesc'), icon: ArrowDownIcon },
  { value: 'count-asc', label: i18n.t('wrongSortCountAsc'), icon: ArrowUpIcon },
  { value: 'time-desc', label: i18n.t('wrongSortTimeDesc'), icon: ClockIcon },
  { value: 'time-asc', label: i18n.t('wrongSortTimeAsc'), icon: ClockIcon },
]

const selected = ref<WrongSort>('count-desc')
</script>

<template>
  <div class="scrim flex items-center justify-center p-4" @click.self="emit('close')">
    <div class="card-elevated w-full max-w-sm p-5">
      <div class="flex items-center gap-3 mb-4">
        <div
          class="w-10 h-10 rounded-xl flex items-center justify-center shrink-0"
          :style="{ backgroundColor: 'rgb(var(--md-error-container))' }"
```

```
        >
          <ExclamationTriangleIcon class="w-5 h-5" :style="{ color: 'rgb(var(--md-on-error-container))' }" />
        </div>
        <div>
          <div class="text-title-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">
            {{ i18n.t('wrongModeTitle') }}
          </div>
          <div class="text-body-sm" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ wrongCount }} {{ i18n.t('wrongCount') }}
          </div>
        </div>
      </div>

      <div class="text-label-sm mb-2" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
        {{ i18n.t('wrongSortTitle') }}
      </div>

      <div class="space-y-1.5 mb-4">
        <button
          v-for="s in sorts"
          :key="s.value"
          class="w-full text-left p-3 rounded-xl border transition-all duration-200 flex items-center gap-3"
          :style="{
            borderColor: selected === s.value ? 'rgb(var(--md-primary))' : 'rgb(var(--md-outline-variant))',
            backgroundColor: selected === s.value ? 'rgba(var(--md-primary), 0.08)' : 'transparent',
          }"
          @click="selected = s.value"
        >
          <component :is="s.icon" class="w-4 h-4" :style="{ color: 'rgb(var(--md-on-surface-variant))' }" />
          <span class="text-sm" :style="{ color: 'rgb(var(--md-on-surface))' }">{{ s.label }}</span>
          <div
            class="w-4 h-4 rounded-full border-2 flex items-center justify-center ml-auto"
            :style="{
              borderColor: selected === s.value ? 'rgb(var(--md-primary))' : 'rgb(var(--md-outline-variant))',
            }"
          >
            <div
              v-if="selected === s.value"
              class="w-2 h-2 rounded-full"
              :style="{ backgroundColor: 'rgb(var(--md-primary))' }"
            />
          </div>
        </button>
      </div>

      <div class="flex gap-3">
        <button class="btn-outlined flex-1" @click="emit('close')">
          {{ i18n.t('btnBack') }}
        </button>
        <button class="btn-filled flex-1" @click="emit('start', selected)">
          <PlayIcon class="w-4 h-4" />
          {{ i18n.t('practiceStartBtn') }}
        </button>
      </div>
    </div>
  </div>
</template>


======================================================================
FILE: frontend/src/views/CameraLiveView.vue
======================================================================

<script setup lang="ts">
import { ref, onUnmounted, nextTick } from 'vue'
import { useRouter } from 'vue-router'
import { useI18nStore } from '@/stores/i18n'
import { useCameraLiveStore } from '@/stores/cameraLive'
import { useCameraLive } from '@/composables/useCameraLive'
import { PauseIcon, XMarkIcon, MagnifyingGlassIcon, CheckIcon, Cog6ToothIcon, ArrowLeftIcon, AdjustmentsHorizontalIcon } from '@heroicons/vue/24/outline'
import { PlayIcon } from '@heroicons/vue/24/solid'
import { api } from '@/api'
import SearchSettingsPanel from '@/components/search/SearchSettingsPanel.vue'

const router = useRouter()
const i18n = useI18nStore()
const store = useCameraLiveStore()
const { startCamera, stopCamera, pauseCamera, resumeCamera } = useCameraLive()

const videoRef = ref<HTMLVideoElement | null>(null)
const cameraReady = ref(false)
const cameraError = ref('')
const permissionDenied = ref(false)
const showSettings = ref(false)

async function handleStart() {
  cameraError.value = ''
  permissionDenied.value = false
  try {
    store.startScanning()
    await nextTick()
    if (!videoRef.value) return
    await startCamera(videoRef.value)
    cameraReady.value = true
  } catch (e: any) {
    store.stopScanning()
    const msg = e?.message || String(e)
    if (msg.includes('Permission') || msg.includes('NotAllowedError') || msg.includes('denied')) {
      cameraError.value = i18n.t('searchCameraLivePermissionDenied')
      permissionDenied.value = true
    } else {
      cameraError.value = i18n.t('searchCameraLiveStartFailed')
    }
    console.warn('[拍屏搜题] start failed:', e)
  }
}

import { isAndroid } from '@/utils/platform'

async function openSettings() {
  try {
    await api.openAppSettings()
  } catch {}
  if (isAndroid()) {
    try {
      const { openUrl } = await import('@tauri-apps/plugin-opener')
      await openUrl('package:com.exameow.app')
    } catch {}
```

```
  }
}

async function handlePause() {
  await pauseCamera()
  store.pauseScanning()
}

async function handleResume() {
  store.resumeScanning()
  await nextTick()
  if (!videoRef.value) return
  await resumeCamera(videoRef.value)
}

async function handleExit() {
  stopCamera()
  store.stopScanning()
  cameraReady.value = false
  router.push('/search')
}

onUnmounted(() => {
  stopCamera()
  store.stopScanning()
})

function isCorrect(idx: number): boolean {
  const r = store.currentResult
  if (!r) return false
  const q = r.question
  if (q.type !== 'single_choice' && q.type !== 'multi_choice') return false
  const letters = (q.answer ?? '').trim().toUpperCase().replace(/[^A-H]/g, '')
  return letters.includes(String.fromCharCode(65 + idx))
}
</script>

<template>
  <div>
    <div class="flex items-center gap-2 mb-1">
      <button class="btn-icon" @click="handleExit">
        <ArrowLeftIcon class="w-5 h-5" />
      </button>
      <h1 class="text-display-sm flex-1">{{ i18n.t('searchModeCameraLive') }}</h1>
    </div>
    <p class="text-body-lg mb-4" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('searchModeCameraLiveDesc') }}</p>

    <div v-if="!cameraReady && store.status === 'idle'" class="card-filled p-6 flex flex-col gap-4">
      <div class="flex items-start justify-end gap-2">
        <button
          class="btn-tonal text-sm shrink-0 !px-3 !py-2"
          :style="{ color: showSettings ? 'rgb(var(--md-primary))' : 'rgb(var(--md-on-surface-variant))' }"
          @click="showSettings = !showSettings"
          :title="i18n.t('searchSettings')"
        >
          <AdjustmentsHorizontalIcon class="w-4 h-4" />
          <span class="hidden sm:inline">{{ i18n.t('searchSettings') }}</span>
        </button>
      </div>

      <div class="flex flex-col items-center gap-4 text-center">
        <div
          class="w-16 h-16 rounded-full flex items-center justify-center"
          :style="{ backgroundColor: 'rgb(var(--md-primary-container))', color: 'rgb(var(--md-on-primary-container))' }"
        >
          <MagnifyingGlassIcon class="w-8 h-8" />
        </div>
        <button
          class="px-8 py-3 rounded-full text-title-md font-medium transition-all duration-200 cursor-pointer"
          :style="{ backgroundColor: 'rgb(var(--md-primary))', color: 'rgb(var(--md-on-primary))' }"
          @click="handleStart"
        >
          {{ i18n.t('searchCameraLiveStart') }}
        </button>
        <p v-if="cameraError" class="text-body-sm" :style="{ color: 'rgb(var(--md-error))' }">
          {{ cameraError }}
        </p>
        <div v-if="permissionDenied" class="flex gap-3 flex-wrap justify-center">
          <button class="btn-tonal text-sm !px-5 !py-2.5" @click="openSettings">
            <Cog6ToothIcon class="w-4 h-4" />
            {{ i18n.t('searchCameraLiveOpenSettings') }}
          </button>
          <button class="btn-outlined text-sm !px-5 !py-2.5" @click="handleStart">
            {{ i18n.t('searchRetry') }}
          </button>
        </div>
      </div>

      <div v-if="showSettings" class="pt-4" style="border-top: 1px solid rgb(var(--md-outline-variant) / 0.4)">
        <SearchSettingsPanel />
      </div>
    </div>

    <template v-else>
      <div class="flex flex-col gap-3" style="height: calc(100vh - 8rem)">
        <div class="relative flex-1 min-h-0 rounded-2xl overflow-hidden" style="background: rgb(var(--md-surface-container-high));">
          <video
            ref="videoRef"
            class="w-full h-full object-cover"
            playsinline
            muted
            autoplay
          />

          <div
            v-if="store.status === 'scanning'"
            class="absolute top-4 right-4 w-3 h-3 rounded-full"
            style="background: rgb(var(--md-error)); animation: cameraLive-pulse 1.5s ease-in-out infinite;"
          />

          <div
            v-if="store.status === 'paused'"
            class="absolute inset-0 flex items-center justify-center"
            style="background: rgba(0,0,0,0.4);"
          >
            <div class="text-center">
              <PauseIcon class="w-10 h-10 mx-auto mb-2" style="color: white;" />
              <p class="text-white text-title-md font-semibold">{{ i18n.t('searchCameraLivePausedLabel') }}</p>
```

```
            </div>
          </div>
        </div>

        <div class="shrink-0 flex items-center justify-center gap-4">
          <template v-if="store.status === 'scanning'">
            <button class="btn-tonal !px-6 !py-3" @click="handlePause">
              <PauseIcon class="w-5 h-5" />
              {{ i18n.t('searchCameraLivePause') }}
            </button>
            <button
              class="btn-outlined !px-6 !py-3"
              :style="{ color: 'rgb(var(--md-error))' }"
              @click="handleExit"
            >
              <XMarkIcon class="w-5 h-5" />
              {{ i18n.t('searchCameraLiveExit') }}
            </button>
          </template>
          <template v-else-if="store.status === 'paused'">
            <button class="btn-filled !px-6 !py-3" @click="handleResume">
              <PlayIcon class="w-5 h-5" />
              {{ i18n.t('searchCameraLiveResume') }}
            </button>
            <button
              class="btn-outlined !px-6 !py-3"
              :style="{ color: 'rgb(var(--md-error))' }"
              @click="handleExit"
            >
              <XMarkIcon class="w-5 h-5" />
              {{ i18n.t('searchCameraLiveExit') }}
            </button>
          </template>
        </div>

        <div class="flex-1 min-h-0 overflow-y-auto">
          <div v-if="store.ocrError" class="card-outlined p-4 text-center">
            <p class="text-body-md" :style="{ color: 'rgb(var(--md-error))' }">OCR 初始化失败</p>
            <p class="text-body-sm mt-1" style="color: rgb(var(--md-on-surface-variant));">{{ store.ocrError }}</p>
          </div>

          <div v-else-if="store.currentResult" :key="store.currentResult.question.id" class="card-filled p-4 space-y-2">
            <div class="flex items-center gap-2">
              <CheckIcon class="w-5 h-5 shrink-0" style="color: rgb(var(--md-primary));" />
              <span class="text-title-md font-bold">
                {{ i18n.t('searchScreenRecordAnswer') }}: {{ store.currentResult.question.answer }}
              </span>
            </div>

            <p class="text-body-md leading-snug">
              {{ store.currentResult.question.stem }}
            </p>

            <div v-if="store.currentResult.question.options?.length" class="space-y-1.5">
              <div
                v-for="(opt, idx) in store.currentResult.question.options"
                :key="idx"
                class="flex items-center gap-2 p-2 rounded-xl"
                :style="isCorrect(idx) ? {
                  backgroundColor: 'rgb(var(--md-primary-container))',
                  color: 'rgb(var(--md-on-primary-container))',
                  fontWeight: 600,
                } : {
                  backgroundColor: 'rgb(var(--md-surface-container-highest))',
                  color: 'rgb(var(--md-on-surface-variant))',
                }"
              >
                <span
                  class="w-6 h-6 rounded-full flex items-center justify-center text-xs font-bold shrink-0"
                  :style="isCorrect(idx) ? {
                    backgroundColor: 'rgb(var(--md-primary))',
                    color: 'rgb(var(--md-on-primary))',
                  } : {
                    backgroundColor: 'rgb(var(--md-surface-container))',
                    color: 'rgb(var(--md-on-surface-variant))',
                  }"
                >
                  {{ String.fromCharCode(65 + idx) }}
                </span>
                <span class="text-sm truncate">{{ opt }}</span>
              </div>
            </div>

            <p class="text-xs" style="color: rgb(var(--md-on-surface-variant));">
              {{ store.currentResult.bankName }}
            </p>
          </div>

          <div v-else class="card-outlined p-4 flex flex-col items-center gap-2">
            <MagnifyingGlassIcon class="w-8 h-8" style="color: rgb(var(--md-on-surface-variant));" />
            <p class="text-body-sm" style="color: rgb(var(--md-on-surface-variant));">
              {{ i18n.t('searchCameraLiveNoMatch') }}
            </p>
          </div>
        </div>
      </div>
    </template>
  </div>
</template>

<style scoped>
@keyframes cameraLive-pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.3; }
}
</style>


================================================================
FILE: frontend/src/views/ConfigView.vue
================================================================

<script setup lang="ts">
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { useConfigStore } from '@/stores/config'

import { useI18nStore } from '@/stores/i18n'
```

```
import { isCloudflare } from '@/utils/platform'
import { ServerIcon, KeyIcon, CloudArrowDownIcon, CpuChipIcon, CheckCircleIcon, EyeIcon, EyeSlashIcon, CheckIcon, ArrowRightIcon, CloudIcon } from '@heroicons

const configStore = useConfigStore()
const router = useRouter()
const i18n = useI18nStore()
const version = import.meta.env.VITE_APP_VERSION

const showKey = ref(false)
const saveSuccess = ref(false)
const saveError = ref('')

const configFetchError = ref('')
const configFetching = ref(false)


async function handleFetchModels() {
  configFetchError.value = ''
  configFetching.value = true
  try { await configStore.fetchModels() } catch (e: any) { configFetchError.value = e.message || String(e) } finally { configFetching.value = false }
}


async function handleSave() {
  saveError.value = ''
  try {
    await configStore.save()
    saveSuccess.value = true
    setTimeout(() => saveSuccess.value = false, 2500)
  } catch (e: any) { saveError.value = e.message || String(e) }
}
</script>

<template>
  <div>
    <div class="flex items-center justify-between mb-1">
      <h1 class="text-display-sm">{{ i18n.t('configTitle') }}</h1>
      <span class="text-body-sm" style="color: rgb(var(--md-on-surface-variant))">v{{ version }}</span>
    </div>
    <p class="text-body-lg mb-6" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('configSubtitle') }}</p>

    <!-- CF: Provider toggle -->
    <div v-if="isCloudflare()" class="card-filled p-4 mb-4">
      <label class="text-label-md block mb-3" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('configAiProvider') }}</label>
      <div class="flex items-center gap-3">
        <button
          class="btn-tonal text-sm !px-4 !py-2"
          :class="{ 'btn-filled': configStore.aiProvider === 'cf-free' }"
          @click="configStore.setProvider('cf-free')"
        >
          <CloudIcon class="w-4 h-4" />
          {{ i18n.t('configCfFree') }}
        </button>
        <button
          class="btn-tonal text-sm !px-4 !py-2"
          :class="{ 'btn-filled': configStore.aiProvider === 'custom' }"
          @click="configStore.setProvider('custom')"
        >
          <ServerIcon class="w-4 h-4" />
          {{ i18n.t('configCustomApi') }}
        </button>
      </div>
      <p v-if="configStore.aiProvider === 'cf-free'" class="text-body-sm mt-2" style="color: rgb(var(--md-on-surface-variant))">
        {{ i18n.t('configCfFreeDesc') }}
      </p>
      <p v-else class="text-body-sm mt-2" style="color: rgb(var(--md-on-surface-variant))">
        {{ i18n.t('configCustomApiDesc') }}
      </p>
    </div>

    <!-- Endpoint (custom API or non-CF) -->
    <div v-if="!isCloudflare() || configStore.aiProvider === 'custom'" class="card-filled p-5 mb-4">
      <label class="text-label-md block mb-3" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('configSectionEndpoint') }}</label>
      <div class="relative">
        <ServerIcon class="absolute left-3 top-1/2 -translate-y-1/2 w-5 h-5 z-10" style="color: rgb(var(--md-on-surface-variant))" />
        <input
          v-model="configStore.endpoint"
          :placeholder="i18n.t('configApiUrl')"
          class="input-outlined !pl-10"
        />
      </div>
    </div>

    <!-- Auth (custom API or non-CF) -->
    <div v-if="!isCloudflare() || configStore.aiProvider === 'custom'" class="card-filled p-5 mb-4">
      <label class="text-label-md block mb-3" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('configSectionAuth') }}</label>
      <div class="relative">
        <KeyIcon class="absolute left-3 top-1/2 -translate-y-1/2 w-5 h-5 z-10" style="color: rgb(var(--md-on-surface-variant))" />
        <input
          v-model="configStore.apiKey"
          :type="showKey ? 'text' : 'password'"
          :placeholder="i18n.t('configApiKey')"
          class="input-outlined !pl-10 !pr-10"
        />
        <button class="absolute right-3 top-1/2 -translate-y-1/2 btn-icon !w-8 !h-8" @click="showKey = !showKey">
          <EyeSlashIcon v-if="showKey" class="w-4 h-4" />
          <EyeIcon v-else class="w-4 h-4" />
        </button>
      </div>
    </div>

    <!-- Model -->
    <div class="card-filled p-5 mb-4">
      <label class="text-label-md block mb-3" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('configSectionModel') }}</label>
      <div class="flex flex-col sm:flex-row items-center gap-3">
        <button
          class="btn-outlined shrink-0 text-sm"
          :disabled="(!isCloudflare() || configStore.aiProvider === 'custom') && (!configStore.endpoint || !configStore.apiKey)"
          @click="handleFetchModels"
        >
          <CloudArrowDownIcon class="w-4 h-4" />
          {{ configFetching ? '...' : i18n.t('configFetchModels') }}
        </button>
        <div class="flex-1 relative">
          <CpuChipIcon class="absolute left-3 top-1/2 -translate-y-1/2 w-5 h-5 z-10" style="color: rgb(var(--md-on-surface-variant))" />
          <select
            v-if="configStore.models.length > 0"
            v-model="configStore.model"
```

```
            class="input-outlined !pl-10 appearance-none cursor-pointer"
          >
            <option value="" disabled>{{ i18n.t('configSelectModel') }}</option>
            <option v-for="m in configStore.models" :key="m.id" :value="m.id">{{ m.id }}</option>
          </select>
          <input
            v-else
            v-model="configStore.model"
            :placeholder="i18n.t('configEnterModel')"
            class="input-outlined !pl-10"
          />
        </div>
      </div>
    </div>

    <Transition name="scale">
      <div
        v-if="configFetchError"
        class="mb-4 px-4 py-3 rounded-2xl text-sm flex items-center gap-2"
        style="background-color: rgb(var(--md-error-container)); color: rgb(var(--md-on-error-container))"
      >
        <span>{{ configFetchError }}</span>
      </div>
    </Transition>

    <!-- ========= Unified Save + CTA ========= -->
    <div class="flex items-center justify-center gap-3 mt-6">
      <button class="btn-filled" :disabled="!configStore.configured" @click="handleSave">
        <CheckIcon class="w-5 h-5" />
        {{ i18n.t('configSave') }}
      </button>

      <button
        class="btn-filled"
        :disabled="!configStore.configured"
        @click="router.push('/generate')"
      >
        <ArrowRightIcon class="w-5 h-5" />
        {{ i18n.t('configReadyCta') }}
      </button>

      <Transition name="fade">
        <div v-if="saveSuccess" class="flex items-center gap-2 text-sm font-medium" style="color: rgb(var(--md-primary))">
          <CheckCircleIcon class="w-5 h-5" /> {{ i18n.t('configSaved') }}
        </div>
      </Transition>
    </div>

    <Transition name="scale">
      <div
        v-if="saveError"
        class="mt-4 px-4 py-3 rounded-2xl text-sm"
        style="background-color: rgb(var(--md-error-container)); color: rgb(var(--md-on-error-container))"
      >
        {{ saveError }}
      </div>
    </Transition>
  </div>
</template>


======================================================================
FILE: frontend/src/views/GenerateView.vue
======================================================================

<script setup lang="ts">
import { ref, computed } from 'vue'
import { useExamStore } from '@/stores/exam'
import { useConfigStore } from '@/stores/config'
import { useI18nStore } from '@/stores/i18n'
import FileUploader from '@/components/generate/FileUploader.vue'
import ParamForm from '@/components/generate/ParamForm.vue'
import QuestionTable from '@/components/preview/QuestionTable.vue'
import { getFileInputs, fileInputsRef } from '@/stores/fileInput'
import { api } from '@/api'
import { generateCsvContent } from '@/api/http'
import { isAndroid } from '@/utils/platform'
import { SparklesIcon, ArrowDownTrayIcon, CheckCircleIcon, ShareIcon } from '@heroicons/vue/24/outline'

const examStore = useExamStore()
const configStore = useConfigStore()
const i18n = useI18nStore()

const isTauri = '__TAURI__' in window || '__TAURI_INTERNALS__' in window
const exportError = ref('')
const exportSuccess = ref('')
const exportFilePath = ref('')
const exporting = ref(false)
const exportingXlsx = ref(false)

const baseFileName = computed(() => examStore.sourceFileName || 'exameow_questions')

const canGenerate = computed(() =>
  fileInputsRef.value.length > 0 && examStore.questionTypes.length > 0 && examStore.totalCount > 0 && configStore.configured && !examStore.generating,
)

const progressPercent = computed(() => {
  const p = examStore.progress
  if (!p.total) return 0
  return Math.round((p.current / p.total) * 100)
})

const isBatched = computed(() => examStore.progress.total > 0)

async function handleGenerate() {
  const inputs = getFileInputs()
  if (inputs.length === 0) return
  try {
    await examStore.generate(inputs)
  } catch (_e) {}
}

function handleNewBatch() { examStore.reset() }

async function saveFile(filename: string, content: string | Uint8Array): Promise<string | null> {
  exportError.value = ''
  exportSuccess.value = ''
  try {
```

```
    const mod: any = await import('@tauri-apps/plugin-dialog')
    const path = await mod.save({ defaultPath: filename, filters: [{ name: 'File', extensions: [filename.split('.').pop() || '*'] }] })
    if (!path) return null
    if (typeof content === 'string') {
      await api.exportCsv(examStore.questions, path)
    } else {
      await api.exportXlsx(examStore.questions, path)
    }
    exportSuccess.value = i18n.t('previewExportSaved') + path
    exportFilePath.value = path
    return path
  } catch {}
  try {
    const b64 = typeof content === 'string' ? btoa(unescape(encodeURIComponent(content))) : btoa(String.fromCharCode(...content))
    const { tauriApi } = await import('@/api/bridge')
    const savedPath = await tauriApi.saveToDownloads(filename, b64)
    exportSuccess.value = i18n.t('previewExportSaved') + savedPath
    exportFilePath.value = savedPath
    return savedPath
  } catch (e: any) {
    exportError.value = 'Export failed: ' + (e.message || String(e))
    return null
  }
}

async function handleExportCsv() {
  exporting.value = true
  try {
    const defaultName = `${baseFileName.value}.csv`
    if (isTauri) {
      await saveFile(defaultName, generateCsvContent(examStore.questions))
    } else {
      await api.exportCsv(examStore.questions, undefined, defaultName)
    }
  } catch (e: any) { exportError.value = e.message || String(e) } finally { exporting.value = false }
}

async function handleExportXlsx() {
  exportingXlsx.value = true
  try {
    const defaultName = `${baseFileName.value}.xlsx`
    if (isTauri) {
      const base64 = await api.exportXlsxData(examStore.questions)
      const bytes = Uint8Array.from(atob(base64), (c) => c.charCodeAt(0))
      await saveFile(defaultName, bytes)
    } else {
      await api.exportXlsx(examStore.questions, undefined, defaultName)
    }
  } catch (e: any) { exportError.value = e.message || String(e) } finally { exportingXlsx.value = false }
}

async function handleShare() {
  try {
    const { shareFile } = await import('@choochmeque/tauri-plugin-sharekit-api')
    const mime = exportFilePath.value!.endsWith('.xlsx') ? 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' : 'text/csv'
    await shareFile('file://' + exportFilePath.value!, { mimeType: mime, title: exportFilePath.value!.split('/').pop() || 'File' })
  } catch (_e) {}
}
</script>

<template>
  <div>
    <h1 class="text-display-sm mb-1">{{ i18n.t('genTitle') }}</h1>
    <p class="text-body-lg mb-6" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('genSubtitle') }}</p>

    <div class="grid grid-cols-1 lg:grid-cols-2 gap-4 sm:gap-5 mb-6">
      <!-- File Upload Card -->
      <div class="card-outlined min-h-[180px] sm:min-h-[240px] flex items-center justify-center p-3 sm:p-4">
        <FileUploader :is-tauri="isTauri" />
      </div>
      <ParamForm />
    </div>

    <!-- Progress -->
    <Transition name="scale">
      <div v-if="!examStore.error && examStore.generating" class="card-filled p-5 mb-6">
        <div class="mb-3 flex items-center justify-between">
          <span class="text-title-sm">{{ examStore.progress.message || i18n.t('genGenerating') }}</span>
          <span v-if="isBatched" class="text-sm font-bold tabular-nums" style="color: rgb(var(--md-primary))">
            {{ examStore.progress.current }}/{{ examStore.progress.total }}
          </span>
        </div>
        <div v-if="!isBatched && examStore.generating" class="progress-indeterminate" />
        <div v-else class="w-full h-1 rounded-full overflow-hidden" :style="{ backgroundColor: 'rgba(var(--md-primary) / 0.12)' }">
          <div
            class="h-full rounded-full transition-all duration-500 ease-out"
            :style="{ backgroundColor: 'rgb(var(--md-primary))', width: progressPercent + '%' }"
          />
        </div>
        <div v-if="isBatched" class="mt-2 text-body-sm" style="color: rgb(var(--md-on-surface-variant))">
          {{ examStore.questions.length }} questions generated so far
        </div>
        <button
          class="btn-outlined mt-3 text-sm !h-10"
          @click="examStore.cancelGeneration()"
        >
          {{ i18n.t('genCancel') }}
        </button>
      </div>
    </Transition>

    <!-- Error -->
    <Transition name="scale">
      <div v-if="examStore.error && !examStore.generating" class="card-filled p-4 mb-6 border" :style="{ backgroundColor: 'rgba(var(--md-error) / 0.08)', bord
        <p class="text-body-md" style="color: rgb(var(--md-error))">{{ examStore.error }}</p>
      </div>
    </Transition>

    <!-- Generate Button -->
    <div class="flex flex-wrap items-center justify-center gap-2">
      <button
        class="btn-filled text-base !px-10 !h-12"
        :disabled="!canGenerate"
        @click="handleGenerate"
      >
        <SparklesIcon v-if="!examStore.generating" class="w-5 h-5" />
        <svg v-else class="animate-spin w-5 h-5" fill="none" viewBox="0 0 24 24">
          <circle class="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" stroke-width="4"/>
```

```
          <path class="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4z"/>
        </svg>
        {{ examStore.generating ? i18n.t('genGenerating') : i18n.t('genGenerateBtn') }}
      </button>
      <button
        v-if="examStore.generated"
        class="btn-tonal text-sm !h-12"
        @click="handleNewBatch"
      >
        {{ i18n.t('previewNewBatch') }}
      </button>
    </div>

    <!-- Export & Preview -->
    <template v-if="examStore.generated && !examStore.generating">
      <div class="mt-6 mb-4 flex flex-wrap items-center justify-between gap-3">
        <p class="text-body-lg" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('previewQuestionCount', { n: examStore.questions.length }) }}</p>
        <div class="flex flex-wrap gap-2">
          <button class="btn-tonal text-sm" :disabled="exporting" @click="handleExportCsv">
            <ArrowDownTrayIcon class="w-4 h-4" /> CSV
          </button>
          <button class="btn-filled text-sm" :disabled="exportingXlsx" @click="handleExportXlsx">
            <ArrowDownTrayIcon class="w-4 h-4" /> {{ exportingXlsx ? '...' : 'XLSX' }}
          </button>
        </div>
      </div>

      <Transition name="scale">
        <div v-if="exportError" class="mb-4 px-4 py-3 rounded-2xl text-sm" style="background-color: rgb(var(--md-error-container)); color: rgb(var(--md-on-err
      </Transition>
      <Transition name="scale">
        <div v-if="exportSuccess" class="mb-4 px-4 py-3 rounded-2xl text-sm flex items-center gap-2 break-all" style="background-color: rgba(var(--md-primary)
          <CheckCircleIcon class="w-5 h-5 shrink-0" />
          <span class="flex-1 min-w-0">{{ exportSuccess }}</span>
          <button v-if="exportFilePath && isAndroid()" class="btn-tonal !h-7 !px-3 !text-xs shrink-0" @click="handleShare">
            <ShareIcon class="w-3.5 h-3.5" /> {{ i18n.t('previewExportShare') }}
          </button>
        </div>
      </Transition>

      <QuestionTable :questions="examStore.questions" />
    </template>
  </div>
</template>


======================================================================
FILE: frontend/src/views/PhotoSearchView.vue
======================================================================

<script setup lang="ts">
import { ref, onBeforeUnmount } from 'vue'
import { useRouter } from 'vue-router'
import { useI18nStore } from '@/stores/i18n'
import { useImageSearch } from '@/composables/useImageSearch'
import { isMobileDevice } from '@/utils/platform'
import {
  ArrowLeftIcon,
  CameraIcon,
  PhotoIcon,
  ArrowPathIcon,
} from '@heroicons/vue/24/outline'
import { AdjustmentsHorizontalIcon } from '@heroicons/vue/24/outline'
import SearchPanel from '@/components/search/SearchPanel.vue'
import SearchSettingsPanel from '@/components/search/SearchSettingsPanel.vue'

const router = useRouter()
const i18n = useI18nStore()
const { phase, error, busy, recognize, cancel } = useImageSearch()

const cameraInput = ref<HTMLInputElement | null>(null)
const fileInput = ref<HTMLInputElement | null>(null)
const previewUrl = ref('')
const query = ref('')
const recognizedEmpty = ref(false)
const dragging = ref(false)
let currentFile: File | null = null

const showCamera = isMobileDevice()
const showSettings = ref(false)

function pickCamera() {
  cameraInput.value?.click()
}

function pickFile() {
  fileInput.value?.click()
}

function onFileChange(e: Event) {
  const input = e.target as HTMLInputElement
  const file = input.files?.[0]
  input.value = ''
  if (file) handleFile(file)
}

function onDrop(e: DragEvent) {
  dragging.value = false
  const file = e.dataTransfer?.files?.[0]
  if (file && file.type.startsWith('image/')) handleFile(file)
}

async function handleFile(file: File) {
  currentFile = file
  if (previewUrl.value) URL.revokeObjectURL(previewUrl.value)
  previewUrl.value = URL.createObjectURL(file)
  query.value = ''
  recognizedEmpty.value = false
  const text = await recognize(file)
  if (text !== null) {
    query.value = text
    recognizedEmpty.value = !text.trim()
  }
}
onBeforeUnmount(() => {
  cancel()
  if (previewUrl.value) URL.revokeObjectURL(previewUrl.value)
})
```

```
</script>

<template>
  <div>
    <div class="flex items-center gap-2 mb-1">
      <button class="btn-icon" @click="router.push('/search')">
        <ArrowLeftIcon class="w-5 h-5" />
      </button>
      <h1 class="text-display-sm flex-1">{{ i18n.t('searchModePhoto') }}</h1>
    </div>
    <p class="text-body-lg mb-4" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('searchModePhotoDesc') }}</p>

    <input ref="cameraInput" type="file" accept="image/*" capture="environment" class="hidden" @change="onFileChange" />
    <input ref="fileInput" type="file" accept="image/*" class="hidden" @change="onFileChange" />

    <div
      class="card-filled p-5 mb-4 transition-all"
      :style="dragging ? { outline: '2px dashed rgb(var(--md-primary))' } : {}"
      @dragover.prevent="dragging = true"
      @dragleave="dragging = false"
      @drop.prevent="onDrop"
    >
      <div class="flex items-start justify-end gap-2 mb-4">
        <button
          class="btn-tonal text-sm shrink-0 !px-3 !py-2"
          :style="{ color: showSettings ? 'rgb(var(--md-primary))' : 'rgb(var(--md-on-surface-variant))' }"
          @click="showSettings = !showSettings"
          :title="i18n.t('searchSettings')"
        >
          <AdjustmentsHorizontalIcon class="w-4 h-4" />
          <span class="hidden sm:inline">{{ i18n.t('searchSettings') }}</span>
        </button>
      </div>

      <template v-if="!previewUrl">
        <div class="flex flex-col items-center gap-4 text-center">
          <div
            class="w-16 h-16 rounded-full flex items-center justify-center"
            :style="{ backgroundColor: 'rgb(var(--md-primary-container))', color: 'rgb(var(--md-on-primary-container))' }"
          >
            <CameraIcon class="w-8 h-8" />
          </div>
          <div class="flex items-center gap-3">
            <button v-if="showCamera"
              class="px-8 py-3 rounded-full text-title-md font-medium transition-all duration-200 cursor-pointer"
              :style="{ backgroundColor: 'rgb(var(--md-primary))', color: 'rgb(var(--md-on-primary))' }"
              @click="pickCamera"
            >
              {{ i18n.t('searchPhotoTake') }}
            </button>
            <button
              class="px-8 py-3 rounded-full text-title-md font-medium transition-all duration-200 cursor-pointer"
              :style="{ backgroundColor: 'rgb(var(--md-secondary-container))', color: 'rgb(var(--md-on-secondary-container))' }"
              @click="pickFile"
            >
              {{ i18n.t('searchPhotoUpload') }}
            </button>
          </div>
        </div>
      </template>

      <template v-else>
        <img :src="previewUrl" class="max-h-72 mx-auto rounded-2xl mb-4 object-contain" />
        <div class="flex items-center justify-center gap-3 flex-wrap">
          <button v-if="showCamera" class="btn-tonal text-sm" :disabled="busy" @click="pickCamera">
            <CameraIcon class="w-4 h-4" />
            {{ i18n.t('searchPhotoTake') }}
          </button>
          <button class="btn-tonal text-sm" :disabled="busy" @click="pickFile">
            <ArrowPathIcon class="w-4 h-4" />
            {{ i18n.t('searchPhotoReselect') }}
          </button>
        </div>
      </template>

      <div v-if="showSettings" class="mt-4 pt-4" style="border-top: 1px solid rgb(var(--md-outline-variant) / 0.4)">
        <SearchSettingsPanel />
      </div>
    </div>

    <div v-if="busy" class="card-outlined p-4 mb-4 flex items-center justify-between">
      <span class="text-body-md" style="color: rgb(var(--md-on-surface-variant))">
        {{ phase === 'loading-model' ? i18n.t('searchPhotoLoadingModel') : i18n.t('searchPhotoRecognizing') }}
      </span>
      <button class="btn-outlined !h-8 text-xs !px-3" @click="cancel">{{ i18n.t('searchCancel') }}</button>
    </div>

    <div v-else-if="error" class="card-outlined p-4 mb-4">
      <p class="text-body-md mb-3" :style="{ color: 'rgb(var(--md-error))' }">{{ error }}</p>
      <div class="flex gap-2 flex-wrap">
        <button class="btn-tonal !h-8 text-xs !px-3" @click="currentFile && handleFile(currentFile)">
          {{ i18n.t('searchRetry') }}
        </button>
      </div>
    </div>

    <p v-if="recognizedEmpty && !busy && !error" class="text-body-sm mb-4" style="color: rgb(var(--md-on-surface-variant))">
      {{ i18n.t('searchPhotoEmpty') }}
    </p>

    <SearchPanel v-if="previewUrl && !busy" v-model:query="query" />
  </div>
</template>


================================================================
FILE: frontend/src/views/PracticeView.vue
================================================================

<script setup lang="ts">
import { ref, computed, onMounted, onUnmounted, nextTick, watch } from 'vue'
import { useI18nStore } from '@/stores/i18n'
import { usePracticeStore } from '@/stores/practice'
import { useWrongQuestionsStore } from '@/stores/wrongQuestions'
import { useConfigStore } from '@/stores/config'
import { api } from '@/api'
import { isCloudflare } from '@/utils/platform'
import type { JudgeResult } from '@exameow/shared'
```

```
import { useSwipeNavigation } from '@/composables/useSwipeNavigation'
import type { PracticeMode, MockExamConfig, WrongSort } from '@exameow/shared'
import BankListCard from '@/components/practice/BankListCard.vue'
import ImportDialog from '@/components/practice/ImportDialog.vue'
import ModeSelector from '@/components/practice/ModeSelector.vue'
import MockExamConfigComponent from '@/components/practice/MockExamConfig.vue'
import QuestionCard from '@/components/practice/QuestionCard.vue'
import ProgressBar from '@/components/practice/ProgressBar.vue'
import PracticeResult from '@/components/practice/PracticeResult.vue'
import AnswerSheet from '@/components/practice/AnswerSheet.vue'
import WrongQuestionsSortDialog from '@/components/practice/WrongQuestionsSortDialog.vue'
import PracticeModeToggle from '@/components/practice/PracticeModeToggle.vue'
import {
  ArrowLeftIcon,
  CheckIcon,
  XMarkIcon,
  PlayIcon,
  TrashIcon,
  ClockIcon,
  QueueListIcon,
  ArrowPathRoundedSquareIcon,
  ExclamationTriangleIcon,
} from '@heroicons/vue/24/outline'

const i18n = useI18nStore()
const practiceStore = usePracticeStore()
const wrongStore = useWrongQuestionsStore()
const configStore = useConfigStore()

type ViewState = 'browse' | 'select-mode' | 'mock-config' | 'practice' | 'result'

const viewState = ref<ViewState>('browse')
const selectedBankId = ref<string | null>(null)
const selectedMode = ref<PracticeMode | null>(null)
const mockConfig = ref<MockExamConfig>({ typeCounts: {} })
const showImportDialog = ref(false)
const showDeleteConfirm = ref(false)
const showClearSessionConfirm = ref(false)
const deleteBankId = ref<string | null>(null)
const showSubmitConfirm = ref(false)
const showAnswerSheet = ref(false)
const autoAdvancing = ref(false)
const aiJudging = ref(false)
const aiFeedback = ref<string | null>(null)
const aiJudgeError = ref<string | null>(null)
let judgeAbort: AbortController | null = null
const elapsedText = ref('')
const showWrongSortDialog = ref(false)
const wrongSort = ref<WrongSort>('count-desc')
const wrongToast = ref<string | null>(null)
const savedMainSession = ref<any>(null)
const flashcardMode = ref<'exam' | 'flashcard'>('exam')
const swipeContainer = ref<HTMLElement | null>(null)

let pendingDirection: 'next' | 'prev' | null = null

function isView(state: ViewState) {
  return viewState.value === state
}

function doNavigate(dir: 'next' | 'prev') {
  if (animating.value) return
  if (dir === 'prev') {
    if (practiceStore.isFirstQuestion) return
    practiceStore.prevQuestion()
  } else {
    if (practiceStore.isLastQuestion) {
      showSubmitConfirm.value = true
      return
    }
    practiceStore.nextQuestion()
  }
}

function triggerNav(dir: 'next' | 'prev', fromGesture: boolean) {
  if (animating.value) return
  if (dir === 'prev' && practiceStore.isFirstQuestion) return
  if (dir === 'next' && practiceStore.isLastQuestion) {
    showSubmitConfirm.value = true
    return
  }
  if (fromGesture) {
    pendingDirection = dir
  } else {
    doNavigate(dir)
    pendingDirection = null
  }
}

function goPrev() {
  if (practiceStore.isFirstQuestion) return
  triggerNav('prev', true)
  triggerSlide('prev', 0)
}

function goNext() {
  if (practiceStore.isLastQuestion) {
    showSubmitConfirm.value = true
    return
  }
  triggerNav('next', true)
  triggerSlide('next', 0)
}

const { slideOffset, animating, triggerSlide, finishAnimation, attach, detach } = useSwipeNavigation(
  (dir) => triggerNav(dir, true),
  (direction, offset, width) => {
    const target = direction === 'next'
      ? -width
      : width
    slideOffset.value = offset
    void (swipeContainer.value?.offsetWidth)
    slideOffset.value = target
  },
)

function onTransitionEnd() {
  if (animating.value && pendingDirection) {
```

```
    const dir = pendingDirection
    pendingDirection = null
    finishAnimation()
    doNavigate(dir)
  } else {
    finishAnimation()
  }
}

const isMockMode = computed(() => practiceStore.session?.mode === 'mock')

const sessionBankName = computed(() => {
  if (!practiceStore.session) return ''
  const bank = practiceStore.getBank(practiceStore.session.bankId)
  return bank?.name ?? i18n.t('practiceUnknownBank')
})

const sessionProgressText = computed(() => {
  if (!practiceStore.session) return ''
  const answered = practiceStore.session.questions.filter(q => q.submitted).length
  return i18n.t('practiceAnsweredCount', { a: answered, t: practiceStore.progress.total })
})

const sessionModeIcon = computed(() => {
  if (!practiceStore.session) return null
  switch (practiceStore.session.mode) {
    case 'sequential': return QueueListIcon
    case 'random': return ArrowPathRoundedSquareIcon
    case 'mock': return ClockIcon
    case 'wrong': return ExclamationTriangleIcon
  }
})

const isWrongMode = computed(() => practiceStore.session?.mode === 'wrong')

const sessionBankId = computed(() => practiceStore.session?.bankId ?? null)

const currentWrongCount = computed(() => {
  if (!practiceStore.session || !practiceStore.currentQuestion) return undefined
  if (practiceStore.session.mode !== 'wrong') return undefined
  const originalId = practiceStore.currentQuestion.question.id.replace(/-s\d+$/, '')
  return wrongStore.getWrongEntry(practiceStore.session.bankId, originalId)?.wrongCount
})

const resumeSessionHasWrong = computed(() => {
  if (!practiceStore.session) return false
  return wrongStore.hasWrongQuestions(practiceStore.session.bankId)
})

onMounted(() => {
  if (!configStore.configured) configStore.loadSaved()
  if (practiceStore.session) {
    wrongStore.syncSession(practiceStore.session)
  }
})

watch([viewState, () => practiceStore.session], ([state]) => {
  nextTick(() => {
    if (swipeContainer.value) {
      detach(swipeContainer.value)
    }
    if (state === 'practice' && swipeContainer.value) {
      attach(swipeContainer.value)
    }
  })
})

watch(
  [() => practiceStore.session?.currentIndex, () => practiceStore.session?.startedAt],
  () => {
    judgeAbort?.abort()
    judgeAbort = null
    aiJudging.value = false
    aiFeedback.value = null
    aiJudgeError.value = null
  },
)

onUnmounted(() => {
  judgeAbort?.abort()
  if (swipeContainer.value) {
    detach(swipeContainer.value)
  }
})

function resumeSession() {
  viewState.value = 'practice'
  autoAdvancing.value = false
}

function confirmClearSession() {
  practiceStore.clearSession()
  showClearSessionConfirm.value = false
  viewState.value = 'browse'
  selectedBankId.value = null
  selectedMode.value = null
}

const sortedBanks = computed(() => {
  return [...practiceStore.banks].sort((a, b) => b.createdAt - a.createdAt)
})

const availableTypes = computed(() => {
  if (!selectedBankId.value) return []
  const bank = practiceStore.getBank(selectedBankId.value)
  if (!bank) return []
  const counts: Record<string, number> = {}
  for (const q of bank.questions) {
    counts[q.type] = (counts[q.type] || 0) + 1
  }
  const typeKeys: Record<string, string> = {
    single_choice: 'typeSingle',
    multi_choice: 'typeMulti',
    true_false: 'typeTrueFalse',
    fill_blank: 'typeFillBlank',
    short_answer: 'typeShortAnswer',
  }
  return Object.entries(counts).map(([type, count]) => ({
```

```
    type: type as any,
    label: i18n.t(typeKeys[type] as any),
    count,
  }))
})

const canStartMockExam = computed(() => {
  return Object.values(mockConfig.value.typeCounts).some(c => c > 0)
})

function selectBank(id: string) {
  selectedBankId.value = id
  viewState.value = 'select-mode'
}

function handleModeSelect(mode: PracticeMode) {
  selectedMode.value = mode
  if (mode === 'wrong') {
    showWrongSortDialog.value = true
  } else if (mode === 'mock') {
    viewState.value = 'mock-config'
    mockConfig.value = { typeCounts: {} }
  } else {
    startPractice(mode)
  }
}

function generateMockExam() {
  if (!canStartMockExam.value || !selectedBankId.value) return
  startPractice('mock')
}

function startPractice(mode: PracticeMode) {
  if (!selectedBankId.value) return
  practiceStore.startSession(selectedBankId.value, mode, mode === 'mock' ? mockConfig.value : undefined)
  viewState.value = 'practice'
  autoAdvancing.value = false
}

function handleStartWrongPractice(sort: WrongSort) {
  showWrongSortDialog.value = false
  startWrongPractice(sort)
}

function startWrongPractice(sort: WrongSort) {
  if (!selectedBankId.value) return
  const wrongQs = wrongStore.getWrongQuestions(selectedBankId.value, sort)
  if (wrongQs.length === 0) return
  if (practiceStore.session && practiceStore.session.mode !== 'wrong') {
    savedMainSession.value = JSON.parse(JSON.stringify(practiceStore.session))
  }
  wrongSort.value = sort
  practiceStore.startSession(selectedBankId.value, 'wrong', undefined, wrongQs)
  viewState.value = 'practice'
  autoAdvancing.value = false
  selectedMode.value = 'wrong'
}

function handleWrongPracticeFromCard() {
  if (!practiceStore.session) return
  selectedBankId.value = practiceStore.session.bankId
  showWrongSortDialog.value = true
}

function handleRemoveWrong() {
  if (!practiceStore.session || !practiceStore.currentQuestion) return
  const originalId = practiceStore.currentQuestion.question.id.replace(/-s\d+$/, '')
  wrongStore.removeWrong(practiceStore.session.bankId, originalId)
  showToast('wrongRemoved')
  const isEmpty = practiceStore.removeCurrentQuestion()
  if (isEmpty) {
    elapsedText.value = practiceStore.formatTime(practiceStore.getElapsedTime())
    viewState.value = 'result'
  }
}

function handleManageWrong(bankId: string) {
  selectedBankId.value = bankId
  showWrongSortDialog.value = true
}

function showToast(msg: string) {
  wrongToast.value = msg
  setTimeout(() => {
    if (wrongToast.value === msg) {
      wrongToast.value = null
    }
  }, 2500)
}

function handleImportDone(count: number) {
  showImportDialog.value = false
  if (count > 0) {
    showToast(i18n.t('practiceImportSuccess', { n: count }))
  }
}

function handleSelect(answer: string | null) {
  practiceStore.setAnswer(answer)
}

function handleSubmit(answer: string | null) {
  const isCorrect = practiceStore.submitAnswer(answer)

  if (practiceStore.currentQuestion && sessionBankId.value) {
    const originalId = practiceStore.currentQuestion.question.id.replace(/-s\d+$/, '')
    if (isCorrect === true) {
      const removed = wrongStore.recordCorrect(sessionBankId.value, originalId)
      if (removed) {
        showToast('wrongAutoRemoved')
      }
    } else if (isCorrect === false) {
      wrongStore.recordWrong(sessionBankId.value, originalId)
    }
  }

  if (isCorrect === true && !isMockMode.value) {
    autoAdvancing.value = true
```

```
    setTimeout(() => {
      autoAdvancing.value = false
      if (practiceStore.isLastQuestion) {
        finalizeSession()
      } else {
        goNext()
      }
    }, 600)
  }
}

function applyGrade(correct: boolean) {
  practiceStore.selfCheck(correct)

  if (practiceStore.currentQuestion && sessionBankId.value) {
    const originalId = practiceStore.currentQuestion.question.id.replace(/-s\d+$/, '')
    if (correct) {
      const removed = wrongStore.recordCorrect(sessionBankId.value, originalId)
      if (removed) {
        showToast('wrongAutoRemoved')
      }
    } else {
      wrongStore.recordWrong(sessionBankId.value, originalId)
    }
  }
}

function handleSelfCheck(correct: boolean) {
  applyGrade(correct)

  if (correct && !isMockMode.value) {
    autoAdvancing.value = true
    setTimeout(() => {
      autoAdvancing.value = false
      if (practiceStore.isLastQuestion) {
        finalizeSession()
      } else {
        goNext()
      }
    }, 600)
  }
}

function handleRegrade(correct: boolean) {
  applyGrade(correct)
}

function handleAiCancel() {
  judgeAbort?.abort()
}

async function handleAiJudge() {
  const item = practiceStore.currentQuestion
  if (!item || !item.userAnswer || aiJudging.value || item.submitted) return

  if (!configStore.configured) {
    await configStore.loadSaved()
    if (!configStore.configured) {
      aiJudgeError.value = i18n.t('searchNotConfigured')
      return
    }
  }

  aiJudging.value = true
  aiJudgeError.value = null
  aiFeedback.value = null
  judgeAbort = new AbortController()
  const language = i18n.locale === 'zh' ? 'Chinese' : 'English'
  const qIndex = practiceStore.session?.currentIndex
  const q = item.question
  const params = {
    stem: q.stem,
    reference_answer: q.answer,
    analysis: q.analysis || undefined,
    user_answer: item.userAnswer,
  }

  try {
    const config = configStore.getConfig()
    let result: JudgeResult
    if (isCloudflare() && configStore.aiProvider === 'custom') {
      const { judgeViaCustomAI } = await import('@/utils/answerClient')
      result = await judgeViaCustomAI(params, language, config, judgeAbort.signal)
    } else {
      result = await api.judgeAnswer(params, language, config, judgeAbort.signal)
    }
    if (practiceStore.session?.currentIndex !== qIndex) return
    aiFeedback.value = result.feedback
    applyGrade(result.correct)
  } catch (e: any) {
    if (e?.name !== 'AbortError') aiJudgeError.value = e?.message || String(e)
  } finally {
    aiJudging.value = false
    judgeAbort = null
  }
}

function finalizeSession() {
  practiceStore.finishSession()
  elapsedText.value = practiceStore.formatTime(practiceStore.getElapsedTime())
  viewState.value = 'result'
}

function handleSubmitAll() {
  finalizeSession()
  showSubmitConfirm.value = false
}

function handleRetry() {
  const s = practiceStore.session
  if (!s) return
  if (s.mode === 'wrong') {
    practiceStore.clearSession()
    startWrongPractice(wrongSort.value)
    return
  }
  practiceStore.clearSession()
  practiceStore.startSession(s.bankId, s.mode, s.mockConfig)
```

```
  viewState.value = 'practice'
  autoAdvancing.value = false
}

function handleHome() {
  practiceStore.clearSession()
  if (savedMainSession.value) {
    practiceStore.session = savedMainSession.value
    localStorage.setItem('exameow-practice-session', JSON.stringify(savedMainSession.value))
    savedMainSession.value = null
  }
  selectedBankId.value = null
  selectedMode.value = null
  mockConfig.value = { typeCounts: {} }
  showWrongSortDialog.value = false
  viewState.value = 'browse'
}

function handleDelete(id: string) {
  deleteBankId.value = id
  showDeleteConfirm.value = true
}

function confirmDelete() {
  if (deleteBankId.value) {
    practiceStore.removeBank(deleteBankId.value)
    if (selectedBankId.value === deleteBankId.value) {
      selectedBankId.value = null
    }
  }
  showDeleteConfirm.value = false
  deleteBankId.value = null
}

function handleBack() {
  if (viewState.value === 'select-mode') {
    viewState.value = 'browse'
    selectedMode.value = null
  } else if (viewState.value === 'mock-config') {
    viewState.value = 'select-mode'
    selectedMode.value = null
  } else if (viewState.value === 'practice') {
    if (practiceStore.session?.mode === 'wrong') {
      practiceStore.session = null
      if (savedMainSession.value) {
        practiceStore.session = savedMainSession.value
        localStorage.setItem('exameow-practice-session', JSON.stringify(savedMainSession.value))
        savedMainSession.value = null
      }
    }
    viewState.value = 'browse'
    selectedBankId.value = null
    selectedMode.value = null
  } else if (viewState.value === 'result') {
    viewState.value = 'browse'
    selectedBankId.value = null
    selectedMode.value = null
  }
}
</script>

<template>
  <div>
    <!-- Header -->
    <div class="flex items-center justify-between mb-4">
      <div>
        <h1 class="text-display-sm mb-1">{{ i18n.t('practiceTitle') }}</h1>
        <p class="text-body-lg" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
          {{ viewState === 'browse'
            ? i18n.t('practiceSubtitle')
            : viewState === 'result'
              ? i18n.t('practiceResultTitle')
              : practiceStore.session && practiceStore.progress.total > 0
                ? i18n.t('practiceProgress', { c: practiceStore.progress.current, t: practiceStore.progress.total })
                : ''
          }}
        </p>
      </div>
      <button
        v-if="viewState !== 'browse'"
        class="btn-icon"
        @click="handleBack"
      >
        <ArrowLeftIcon class="w-5 h-5" />
      </button>
    </div>

    <!-- Browse View -->
    <template v-if="viewState === 'browse'">
      <!-- Resume Session Card -->
      <div
        v-if="practiceStore.hasSession"
        class="card-elevated p-3 sm:p-5 mb-4 border-2 transition-all duration-200"
        :style="{ borderColor: 'rgb(var(--md-primary) / 0.3)' }"
      >
        <div class="flex items-start gap-3 sm:gap-4">
          <div
            class="w-10 h-10 sm:w-12 sm:h-12 rounded-xl sm:rounded-2xl flex items-center justify-center shrink-0"
            :style="{ backgroundColor: 'rgb(var(--md-primary-container))' }"
          >
            <component :is="sessionModeIcon" class="w-5 h-5 sm:w-6 sm:h-6" :style="{ color: 'rgb(var(--md-on-primary-container))' }" />
          </div>
          <div class="flex-1 min-w-0">
            <div class="text-title-sm truncate" :style="{ color: 'rgb(var(--md-on-surface))' }">
              {{ i18n.t('practiceContinuePractice') }}
            </div>
            <div class="text-body-sm mt-0.5 truncate" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
              {{ sessionBankName }} · {{ sessionProgressText }}
            </div>
            <div class="h-1.5 rounded-full overflow-hidden mt-2 w-full" :style="{ backgroundColor: 'rgba(var(--md-primary) / 0.12)' }">
              <div
                class="h-full rounded-full transition-all"
                :style="{
                  backgroundColor: 'rgb(var(--md-primary))',
                  width: `${practiceStore.session ? Math.round((practiceStore.session.questions.filter(q => q.submitted).length / practiceStore.progress.total
                }"
              />
            </div>
```

```
      </div>
    </div>
    <div class="flex items-center gap-2 mt-3">
      <button
        class="btn-icon !w-7 !h-7 sm:!w-8 sm:!h-8 shrink-0"
        :style="{ color: 'rgb(var(--md-on-surface-variant))' }"
        @click="showClearSessionConfirm = true"
      >
        <TrashIcon class="w-4 h-4" />
      </button>
      <button
        v-if="resumeSessionHasWrong"
        class="btn-tonal !h-8 sm:!h-9 text-xs sm:text-sm !px-3 sm:!px-4 shrink-0"
        :style="{
          borderColor: 'rgb(var(--md-error))',
          color: 'rgb(var(--md-error))',
        }"
        @click="handleWrongPracticeFromCard"
      >
        <ExclamationTriangleIcon class="w-3.5 h-3.5 sm:w-4 sm:h-4" />
        {{ i18n.t('practiceWrongPractice') }}
      </button>
      <button class="btn-filled !h-8 sm:!h-9 text-xs sm:text-sm !px-3 sm:!px-4 shrink-0 ml-auto" @click="resumeSession">
        <PlayIcon class="w-3.5 h-3.5 sm:w-4 sm:h-4" />
        {{ i18n.t('practiceContinue') }}
      </button>
    </div>
  </div>

  <BankListCard
    :banks="sortedBanks"
    :selected-id="selectedBankId"
    @select="selectBank"
    @delete="handleDelete"
    @import="showImportDialog = true"
    @manage-wrong="handleManageWrong"
  />
</template>

<!-- Mode Selection View -->
<template v-if="isView('select-mode')">
  <template v-if="selectedBankId">
    <div class="card-outlined p-4 mb-4">
      <div class="text-sm truncate" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
        {{ practiceStore.getBank(selectedBankId)?.name }}
      </div>
      <div class="text-title-md truncate" :style="{ color: 'rgb(var(--md-on-surface))' }">
        {{ practiceStore.getBank(selectedBankId)?.questions.length ? i18n.t('practiceQuestionUnit', { n: practiceStore.getBank(selectedBankId)!.questions.
      </div>
    </div>
  </template>
  <ModeSelector
    v-model="selectedMode"
    :has-wrong-questions="selectedBankId ? wrongStore.hasWrongQuestions(selectedBankId) : false"
    @confirm="handleModeSelect"
  />
</template>

<!-- Mock Exam Config View -->
<template v-if="isView('mock-config')">
  <MockExamConfigComponent
    :available-types="availableTypes"
    :config="mockConfig"
    @update:config="mockConfig = $event"
    @generate="generateMockExam"
  />
</template>

<!-- Practice View -->
<template v-if="isView('practice') && practiceStore.session && practiceStore.currentQuestion">
  <div ref="swipeContainer" class="space-y-4 cursor-grab select-none">
    <!-- Progress Bar + Mode Toggle -->
    <div class="card-outlined p-3">
      <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-2">
        <ProgressBar
          :mode="practiceStore.session.mode"
          :current="practiceStore.progress.current"
          :total="practiceStore.progress.total"
          :answered-count="practiceStore.answeredCount"
          @open-sheet="showAnswerSheet = true"
          class="flex-1 min-w-0"
        />
        <PracticeModeToggle
          v-model="flashcardMode"
        />
      </div>
    </div>

    <!-- Question Card -->
    <div
      :style="{
        transform: `translateX(${slideOffset}px)`,
        transition: animating ? 'transform 0.28s cubic-bezier(0.2, 0, 0, 1)' : 'none',
      }"
      @transitionend="onTransitionEnd"
    >
      <QuestionCard
        :key="practiceStore.session.currentIndex"
        :question="practiceStore.currentQuestion.question"
        :mode="practiceStore.session.mode"
        :user-answer="practiceStore.currentQuestion.userAnswer"
        :is-correct="practiceStore.currentQuestion.isCorrect"
        :submitted="practiceStore.currentSubmitted"
        :question-number="practiceStore.progress.current"
        :auto-advancing="autoAdvancing"
        :wrong-count="currentWrongCount"
        :is-wrong-mode="isWrongMode"
        :flashcard-mode="flashcardMode === 'flashcard'"
        :ai-configured="configStore.configured"
        :ai-judging="aiJudging"
        :ai-feedback="aiFeedback"
        :ai-judge-error="aiJudgeError"
        @ai-judge="handleAiJudge"
        @ai-cancel="handleAiCancel"
        @regrade="handleRegrade"
        @submit="handleSubmit"
        @select="handleSelect"
        @self-check="handleSelfCheck"
```

```
        @remove-wrong="handleRemoveWrong"
      />
    </div>

    <!-- Navigation -->
    <div class="flex items-center gap-3">
      <button
        class="btn-tonal flex-1"
        :disabled="practiceStore.isFirstQuestion"
        @click="goPrev"
      >
        {{ i18n.t('practicePrevBtn') }}
      </button>

      <button class="btn-filled flex-1" @click="goNext" :disabled="autoAdvancing">
        <template v-if="practiceStore.isLastQuestion">
          <CheckIcon class="w-4 h-4" />
          {{ i18n.t('practiceSubmitBtn') }}
        </template>
        <template v-else>
          {{ i18n.t('practiceNextBtn') }}
          <span class="text-xs opacity-60">({{ practiceStore.answeredCount }}/{{ practiceStore.progress.total }})</span>
        </template>
      </button>
    </div>
  </div>
</template>

<!-- Result View -->
<template v-if="isView('result') && practiceStore.session">
  <PracticeResult
    :session="practiceStore.session"
    :score="practiceStore.score"
    :auto-graded-count="practiceStore.autoGradedCount"
    :elapsed-text="elapsedText"
    @retry="handleRetry"
    @home="handleHome"
  />
</template>

<!-- Import Dialog -->
<Transition name="scale">
  <div v-if="showImportDialog" class="scrim flex items-center justify-center p-4" @click.self="showImportDialog = false">
    <div class="card-elevated w-full max-w-lg max-h-[80vh] overflow-y-auto p-5">
      <ImportDialog @close="showImportDialog = false" @imported="handleImportDone" />
    </div>
  </div>
</Transition>

<!-- Answer Sheet -->
<Transition name="scale">
  <AnswerSheet
    v-if="showAnswerSheet"
    @close="showAnswerSheet = false"
  />
</Transition>

<!-- Wrong Questions Sort Dialog -->
<WrongQuestionsSortDialog
  v-if="showWrongSortDialog"
  :wrong-count="selectedBankId ? wrongStore.getWrongCount(selectedBankId) : 0"
  @start="handleStartWrongPractice"
  @close="showWrongSortDialog = false"
/>

<!-- Toast -->
<Transition name="scale">
  <div
    v-if="wrongToast"
    class="fixed bottom-[calc(88px+env(safe-area-inset-bottom))] sm:bottom-6 left-1/2 -translate-x-1/2 px-4 py-2 rounded-full text-sm font-medium z-50 sha
    :style="{
      backgroundColor: 'rgb(var(--md-inverse-surface))',
      color: 'rgb(var(--md-inverse-on-surface))',
    }"
  >
    {{ wrongToast }}
  </div>
</Transition>

<!-- Delete Confirm Dialog -->
<Transition name="scale">
  <div v-if="showDeleteConfirm" class="scrim flex items-center justify-center p-4" @click.self="showDeleteConfirm = false">
    <div class="card-elevated w-full max-w-sm p-5 text-center">
      <XMarkIcon class="w-10 h-10 mx-auto mb-3" :style="{ color: 'rgb(var(--md-error))' }" />
      <div class="text-title-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface))' }">
        {{ i18n.t('practiceDeleteConfirm') }}
      </div>
      <div class="flex gap-3 mt-4">
        <button class="btn-outlined flex-1" @click="showDeleteConfirm = false">{{ i18n.t('btnBack') }}</button>
        <button class="btn-filled flex-1" :style="{ backgroundColor: 'rgb(var(--md-error))', color: 'rgb(var(--md-on-error))' }" @click="confirmDelete">
          {{ i18n.t('practiceDeleteBank') }}
        </button>
      </div>
    </div>
  </div>
</Transition>

<!-- Clear Session Dialog -->
<Transition name="scale">
  <div v-if="showClearSessionConfirm" class="scrim flex items-center justify-center p-4" @click.self="showClearSessionConfirm = false">
    <div class="card-elevated w-full max-w-sm p-5 text-center">
      <TrashIcon class="w-10 h-10 mx-auto mb-3" :style="{ color: 'rgb(var(--md-error))' }" />
      <div class="text-title-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface))' }">
        {{ i18n.t('practiceClearProgress') }}
      </div>
      <p class="text-body-sm mb-4" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
        {{ i18n.t('practiceClearProgressMsg') }}
      </p>
      <div class="flex gap-3">
        <button class="btn-outlined flex-1" @click="showClearSessionConfirm = false">{{ i18n.t('practiceCancel') }}</button>
        <button
          class="btn-filled flex-1"
          :style="{ backgroundColor: 'rgb(var(--md-error))', color: 'rgb(var(--md-on-error))' }"
          @click="confirmClearSession"
        >
          {{ i18n.t('practiceClear') }}
        </button>
      </div>
```

```
        </div>
      </div>
    </Transition>

    <!-- Submit Confirm Dialog -->
    <Transition name="scale">
      <div v-if="showSubmitConfirm" class="scrim flex items-center justify-center p-4" @click.self="showSubmitConfirm = false">
        <div class="card-elevated w-full max-w-sm p-5 text-center">
          <div class="text-title-sm mb-1" :style="{ color: 'rgb(var(--md-on-surface))' }">
            {{ i18n.t('practiceSubmitConfirm') }}
          </div>
          <div v-if="practiceStore.hasUnanswered" class="text-body-sm mt-2 mb-3" :style="{ color: 'rgb(var(--md-error))' }">
            {{ i18n.t('practiceUnansweredCount', { n: practiceStore.progress.total - practiceStore.answeredCount }) }}
          </div>
          <p class="text-body-sm mb-4" :style="{ color: 'rgb(var(--md-on-surface-variant))' }">
            {{ i18n.t('practiceSubmitConfirmMsg') }}
          </p>
          <div class="flex gap-3">
            <button class="btn-outlined flex-1" @click="showSubmitConfirm = false">{{ i18n.t('btnBack') }}</button>
            <button class="btn-filled flex-1" @click="handleSubmitAll">
              <CheckIcon class="w-4 h-4" />
              {{ i18n.t('practiceSubmitBtn') }}
            </button>
          </div>
        </div>
      </div>
    </Transition>
  </div>
</template>


======================================================================
FILE: frontend/src/views/PreviewView.vue
======================================================================

<script setup lang="ts">
import { ref, computed } from 'vue'
import { useRouter } from 'vue-router'
import { useExamStore } from '@/stores/exam'
import { useI18nStore } from '@/stores/i18n'
import { api } from '@/api'
import { generateCsvContent } from '@/api/http'
import QuestionTable from '@/components/preview/QuestionTable.vue'
import { ArrowLeftIcon, ArrowDownTrayIcon, DocumentTextIcon, CheckCircleIcon, ShareIcon } from '@heroicons/vue/24/outline'
import { isAndroid } from '@/utils/platform'

const router = useRouter()
const examStore = useExamStore()
const i18n = useI18nStore()
const isTauri = '__TAURI__' in window || '__TAURI_INTERNALS__' in window
const exportError = ref('')
const exportSuccess = ref('')
const exportFilePath = ref('')
const exporting = ref(false)
const exportingXlsx = ref(false)

const baseFileName = computed(() => {
  const name = examStore.sourceFileName
  if (name) return name
  return 'exameow_questions'
})

async function saveFile(filename: string, content: string | Uint8Array): Promise<string | null> {
  try {
    const mod: any = await import('@tauri-apps/plugin-dialog')
    const path = await mod.save({ defaultPath: filename, filters: [{ name: 'File', extensions: [filename.split('.').pop() || '*'] }] })
    if (!path) return null
    if (typeof content === 'string') {
      await api.exportCsv(examStore.questions, path)
    } else {
      await api.exportXlsx(examStore.questions, path)
    }
    exportSuccess.value = i18n.t('previewExportSaved') + path
    exportFilePath.value = path
    return path
  } catch {}
  try {
    const b64 = typeof content === 'string' ? btoa(unescape(encodeURIComponent(content))) : btoa(String.fromCharCode(...content))
    const { tauriApi } = await import('@/api/bridge')
    const savedPath = await tauriApi.saveToDownloads(filename, b64)
    exportSuccess.value = i18n.t('previewExportSaved') + savedPath
    exportFilePath.value = savedPath
    return savedPath
  } catch (e: any) {
    exportError.value = 'Export failed: ' + (e.message || String(e))
    return null
  }
}

async function handleShare() {
  try {
    const { openPath } = await import('@tauri-apps/plugin-opener')
    await openPath(exportFilePath.value!)
  } catch (e: any) {
    exportError.value = 'Open failed: ' + (e.message || String(e))
  }
}

async function handleExport() {
  exportError.value = ''
  exportSuccess.value = ''
  exporting.value = true
  try {
    const defaultName = `${baseFileName.value}.csv`
    if (isTauri) {
      const content = generateCsvContent(examStore.questions)
      await saveFile(defaultName, content)
    } else {
      await api.exportCsv(examStore.questions, undefined, defaultName)
    }
  } catch (e: any) { exportError.value = e.message || String(e) } finally { exporting.value = false }
}

async function handleExportXlsx() {
  exportError.value = ''
  exportSuccess.value = ''
  exportingXlsx.value = true
  try {
```

```
    const defaultName = `${baseFileName.value}.xlsx`
    if (isTauri) {
      const base64 = await api.exportXlsxData(examStore.questions)
      const bytes = Uint8Array.from(atob(base64), (c) => c.charCodeAt(0))
      await saveFile(defaultName, bytes)
    } else {
      await api.exportXlsx(examStore.questions, undefined, defaultName)
    }
  } catch (e: any) { exportError.value = e.message || String(e) } finally { exportingXlsx.value = false }
}

function handleNewBatch() { examStore.reset(); router.push('/generate') }
</script>

<template>
  <div>
    <!-- Empty State -->
    <div v-if="!examStore.generated">
      <h1 class="text-display-sm mb-1">{{ i18n.t('previewTitle') }}</h1>
      <p class="text-body-lg mb-6" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('previewSubtitleEmpty') }}</p>
      <div class="card-outlined text-center py-14 px-6">
        <div class="w-[72px] h-[72px] rounded-[28px] flex items-center justify-center mx-auto mb-5 elevation-1"
             style="background: linear-gradient(135deg, rgb(var(--md-primary)), rgb(var(--md-tertiary)))">
          <DocumentTextIcon class="w-9 h-9 text-white" />
        </div>
        <h3 class="text-title-lg mb-2">{{ i18n.t('previewEmptyTitle') }}</h3>
        <p class="text-body-md mb-6" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('previewEmptyText') }}</p>
        <button class="btn-filled" @click="router.push('/generate')">
          {{ i18n.t('previewGotoGenerate') }}
        </button>
      </div>
    </div>

    <!-- Results -->
    <div v-else>
      <div class="flex flex-col sm:flex-row flex-wrap items-start sm:items-center gap-3 mb-6">
        <div class="flex-1">
          <h1 class="text-display-sm mb-1">{{ i18n.t('previewTitle') }}</h1>
          <p class="text-body-lg" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('previewQuestionCount', { n: examStore.questions.length }) }}</p>
        </div>
        <div class="flex flex-wrap gap-2">
          <button class="btn-tonal text-sm" @click="handleNewBatch">
            <ArrowLeftIcon class="w-4 h-4" /> {{ i18n.t('previewNewBatch') }}
          </button>
          <button class="btn-tonal text-sm" :disabled="exporting" @click="handleExport">
            <ArrowDownTrayIcon class="w-4 h-4" /> CSV
          </button>
          <button class="btn-filled text-sm" :disabled="exportingXlsx" @click="handleExportXlsx">
            <ArrowDownTrayIcon class="w-4 h-4" /> {{ exportingXlsx ? '...' : 'XLSX' }}
          </button>
        </div>
      </div>

      <Transition name="scale">
        <div
          v-if="exportError"
          class="mb-4 px-4 py-3 rounded-2xl text-sm"
          style="background-color: rgb(var(--md-error-container)); color: rgb(var(--md-on-error-container))"
        >
          {{ exportError }}
        </div>
      </Transition>

      <Transition name="scale">
        <div
          v-if="exportSuccess"
          class="mb-4 px-4 py-3 rounded-2xl text-sm flex items-center gap-2 break-all"
          style="background-color: rgba(var(--md-primary) / 0.12); color: rgb(var(--md-primary))"
        >
          <CheckCircleIcon class="w-5 h-5 shrink-0" />
          <span class="flex-1 min-w-0">{{ exportSuccess }}</span>
          <button
            v-if="exportFilePath && isAndroid()"
            class="btn-tonal !h-7 !px-3 !text-xs shrink-0"
            @click="handleShare"
          >
            <ShareIcon class="w-3.5 h-3.5" /> {{ i18n.t('previewExportShare') }}
          </button>
        </div>
      </Transition>

      <QuestionTable :questions="examStore.questions" />
    </div>
  </div>
</template>


================================================================
FILE: frontend/src/views/ScreenRecordView.vue
================================================================

<script setup lang="ts">
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { useI18nStore } from '@/stores/i18n'
import { useScreenRecordStore } from '@/stores/screenRecord'
import { isAndroid, isDesktopTauri } from '@/utils/platform'
import { VideoCameraIcon, ArrowLeftIcon, AdjustmentsHorizontalIcon } from '@heroicons/vue/24/outline'
import SearchSettingsPanel from '@/components/search/SearchSettingsPanel.vue'

const router = useRouter()
const i18n = useI18nStore()
const store = useScreenRecordStore()

const supported = isDesktopTauri() || isAndroid()
const showSettings = ref(false)

async function startRecording() {
  store.startRecording()
  const { useScreenRecord } = await import('@/composables/useScreenRecord')
  const { start } = useScreenRecord()
  try {
    await start()
  } catch (e) {
    store.stopRecording()
    console.warn('[录屏搜题] start failed:', e)
    const msg = String(e)
    if (msg.includes('overlay_permission')) {
```

```
      alert(i18n.t('searchScreenRecordOverlayPerm'))
    } else if (msg.includes('projection_denied')) {
      alert(i18n.t('searchScreenRecordProjectionDenied'))
    } else if (msg.includes('screen_permission_required')) {
      alert(i18n.t('searchScreenRecordScreenPerm'))
    }
  }
}

function goBack() {
  router.push('/search')
}
</script>

<template>
  <div>
    <div class="flex items-center gap-2 mb-1">
      <button class="btn-icon" @click="goBack">
        <ArrowLeftIcon class="w-5 h-5" />
      </button>
      <h1 class="text-display-sm flex-1">{{ i18n.t('searchModeScreenRecord') }}</h1>
    </div>
    <p class="text-body-lg mb-4" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('searchScreenRecordDesc') }}</p>

    <div v-if="!supported" class="card-filled p-6 text-center">
      <VideoCameraIcon class="w-12 h-12 mx-auto mb-3" style="color: rgb(var(--md-on-surface-variant))" />
      <p class="text-title-md mb-2">{{ i18n.t('searchModeScreenRecord') }}</p>
      <p class="text-body-md" style="color: rgb(var(--md-on-surface-variant))">
        {{ i18n.t('searchScreenRecordNotSupported') }}
      </p>
    </div>

    <template v-else>
      <div class="card-filled p-6 flex flex-col gap-4">
        <div class="flex items-start justify-end gap-2">
          <button
            class="btn-tonal text-sm shrink-0 !px-3 !py-2"
            :style="{ color: showSettings ? 'rgb(var(--md-primary))' : 'rgb(var(--md-on-surface-variant))' }"
            @click="showSettings = !showSettings"
            :title="i18n.t('searchSettings')"
          >
            <AdjustmentsHorizontalIcon class="w-4 h-4" />
            <span class="hidden sm:inline">{{ i18n.t('searchSettings') }}</span>
          </button>
        </div>

        <div class="flex flex-col items-center gap-4 text-center">
          <div
            class="w-16 h-16 rounded-full flex items-center justify-center"
            :style="{ backgroundColor: 'rgb(var(--md-primary-container))', color: 'rgb(var(--md-on-primary-container))' }"
          >
            <VideoCameraIcon class="w-8 h-8" />
          </div>
          <button
            class="px-8 py-3 rounded-full text-title-md font-medium transition-all duration-200 cursor-pointer"
            :style="{
              backgroundColor: 'rgb(var(--md-primary))',
              color: 'rgb(var(--md-on-primary))',
            }"
            @click="startRecording"
          >
            {{ i18n.t('searchScreenRecordStart') }}
          </button>
        </div>

        <div v-if="showSettings" class="pt-4" style="border-top: 1px solid rgb(var(--md-outline-variant) / 0.4)">
          <SearchSettingsPanel />
        </div>
      </div>
    </template>
  </div>
</template>


======================================================================
FILE: frontend/src/views/SearchHomeView.vue
======================================================================

<script setup lang="ts">
import { useRouter } from 'vue-router'
import { useI18nStore } from '@/stores/i18n'
import { isTauri, isDesktopTauri, isMobileDevice, isAndroid } from '@/utils/platform'
import {
  DocumentTextIcon,
  CameraIcon,
  VideoCameraIcon,
  ComputerDesktopIcon,
} from '@heroicons/vue/24/outline'

const router = useRouter()
const i18n = useI18nStore()

const isTauriMobile = isTauri() && isMobileDevice()

const modes = [
  { key: 'text', titleKey: 'searchModeText', descKey: 'searchModeTextDesc', icon: DocumentTextIcon, available: true, supported: true, path: '/search/text' },
  { key: 'photo', titleKey: 'searchModePhoto', descKey: 'searchModePhotoDesc', icon: CameraIcon, available: true, supported: true, path: '/search/photo' },
  { key: 'screenRecord', titleKey: 'searchModeScreenRecord', descKey: 'searchModeScreenRecordDesc', icon: ComputerDesktopIcon, available: true, supported: isD
  { key: 'cameraLive', titleKey: 'searchModeCameraLive', descKey: 'searchModeCameraLiveDesc', icon: VideoCameraIcon, available: true, supported: isTauriMobile
] as const

function open(mode: (typeof modes)[number]) {
  if (mode.available && mode.supported && mode.path) router.push(mode.path)
}
</script>

<template>
  <div>
    <h1 class="text-display-sm mb-1">{{ i18n.t('searchTitle') }}</h1>
    <p class="text-body-lg mb-6" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('searchSubtitle') }}</p>

    <div class="grid grid-cols-1 sm:grid-cols-2 gap-4 sm:gap-5">
      <button
        v-for="mode in modes"
        :key="mode.key"
        class="card-filled p-5 text-left flex items-start gap-4 transition-all duration-200"
        :class="mode.available && mode.supported ? 'cursor-pointer hover:shadow-md' : 'cursor-not-allowed opacity-50'"
        :disabled="!mode.available || !mode.supported"
```

```
        @click="open(mode)"
      >
        <div
          class="w-12 h-12 rounded-2xl flex items-center justify-center shrink-0"
          :style="{ backgroundColor: 'rgb(var(--md-secondary-container))', color: 'rgb(var(--md-on-secondary-container))' }"
        >
          <component :is="mode.icon" class="w-6 h-6" />
        </div>
        <div class="min-w-0">
          <div class="flex items-center gap-2 flex-wrap">
            <span class="text-title-md">{{ i18n.t(mode.titleKey) }}</span>
            <span
              v-if="!mode.supported"
              class="text-[11px] font-medium px-2 py-0.5 rounded-full"
              :style="{ backgroundColor: 'rgb(var(--md-surface-container-highest))', color: 'rgb(var(--md-on-surface-variant))' }"
            >
              {{ i18n.t('searchNotSupported') }}
            </span>
            <span
              v-else-if="!mode.available"
              class="text-[11px] font-medium px-2 py-0.5 rounded-full"
              :style="{ backgroundColor: 'rgb(var(--md-surface-container-highest))', color: 'rgb(var(--md-on-surface-variant))' }"
            >
              {{ i18n.t('searchComingSoon') }}
            </span>
          </div>
          <p class="text-body-sm mt-1" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t(mode.descKey) }}</p>
        </div>
      </button>
    </div>
  </div>
</template>


======================================================================
FILE: frontend/src/views/TextSearchView.vue
======================================================================

<script setup lang="ts">
import { useRouter } from 'vue-router'
import { useI18nStore } from '@/stores/i18n'
import { ArrowLeftIcon } from '@heroicons/vue/24/outline'
import SearchPanel from '@/components/search/SearchPanel.vue'

const router = useRouter()
const i18n = useI18nStore()
</script>

<template>
  <div>
    <!-- Header -->
    <div class="flex items-center gap-2 mb-1">
      <button class="btn-icon" @click="router.push('/search')">
        <ArrowLeftIcon class="w-5 h-5" />
      </button>
      <h1 class="text-display-sm">{{ i18n.t('searchModeText') }}</h1>
    </div>
    <p class="text-body-lg mb-4" style="color: rgb(var(--md-on-surface-variant))">{{ i18n.t('searchModeTextDesc') }}</p>

    <SearchPanel />
  </div>
</template>
```